Contents

Phân hệ 1: Identity & Access Management (Quản lý Định danh & Người dùng).....3

Phân hệ lõi (Core Module). Mọi phân hệ khác đều phải trỏ khóa ngoại (Foreign Key) về đây..... 3

Các Thực thể ERD dự kiến (Core Entities):.....3

1. Tổng quan: Mục đích chính của sơ đồ này là gì?..... 3

2. Các thực thể đại diện cho đối tượng/quy trình nào & Chi tiết Thực thể..... 4

3. Phân tích Mối quan hệ & Quy tắc nghiệp vụ (Business Rules)..... 4

4. Kịch bản thực tế (Use Case) chạy xuyên suốt sơ đồ.....5

Mermaid:..... 6

Phân hệ 2: Telehealth & Clinical Records (Khám bệnh trực tuyến & Bệnh án)..... 9

Phân hệ xử lý lịch trình, cuộc gọi và lưu trữ hồ sơ y tế cá nhân..... 9

Các Thực thể ERD dự kiến (Core Entities):.....10

1. Tổng quan: Mục đích chính của sơ đồ này là gì?..... 10

2. Chi tiết Thực thể: Đại diện cho quy trình nào trong thực tế?.....10

3. Mối quan hệ & Các Quy tắc Nghiệp vụ (Business Rules) bị ràng buộc..... 11

4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống..... 12

Mermaid:..... 12

Phân hệ 3: E-Pharmacy & Inventory (Nhà thuốc trực tuyến)..... 17

Các Use Case bao hàm (5 UCs):..... 17

Các Thực thể ERD dự kiến (Core Entities):.....17

1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?.....17

2. Chi tiết Thực thể: Đại diện cho đối tượng/quy trình nào trong thực tế?..... 18

3. Mối quan hệ (Relationship) & Quy tắc Nghiệp vụ (Business Rules).....18

4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống..... 19

erDiagram..... 20

Phân hệ 4: Billing & Finance (Tài chính & Thanh toán).....23

Các Use Case bao hàm (5 UCs):..... 24

Các Thực thể ERD dự kiến (Core Entities):.....24

1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?.....24
2. Chi tiết Thực thể: Đại diện cho quy trình/chứng từ nào trong thực tế?.....24
3. Mối quan hệ (Relationship) & Các Quy tắc Nghiệp vụ (Business Rules)..... 25
4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống..... 26
Mermaid:..... 27
Phân hệ 5: CMS & Health Utilities (Quản trị Nội dung & Tiện ích)..... 30
Các Use Case bao hàm (6 UCs):..... 30
1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?.....31
2. Chi tiết Thực thể: Đại diện cho quy trình/đối tượng nào trong thực tế?..... 31
3. Mối quan hệ (Relationship) & Quy tắc Nghiệp vụ (Business Rules).....31
4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống..... 33
Mermaid:..... 33
Phân hệ 6: Third-Party & System Ops (Tích hợp & Vận hành lõi)..... 37
Các Use Case bao hàm (4 UCs):..... 37
Các Thực thể ERD dự kiến (Core Entities):.....37
1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?.....37
2. Chi tiết Thực thể: Đại diện cho quy trình/chứng từ nào trong thực tế?.....37
3. Mối quan hệ (Relationship) & Quy tắc Nghiệp vụ (Business Rules).....38
4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống..... 39
erDiagram..... 39
Conceptual ERD Mermaid.....57
Class diagram: Boundary / Control / Entity..... 57
UC 1: register..... 57
UC2: Login.....67
UC-03: Logout — BCE Class Diagram + Sequence Diagram..... 74
UC-04: Recover Password — Class Diagram + Sequence Diagram..... 78
UC-05: Change Password — Class Diagram + Sequence Diagram..... 85
UC-06: Manage User Profile — Class Diagram + Sequence Diagram.....93

Phân hệ 1: Identity & Access Management (Quản lý Định danh & Người dùng)

Phân hệ lõi (Core Module). Mọi phân hệ khác đều phải trỏ khóa ngoại (Foreign Key) về đây.

Các Use Case bao hàm (7 UCs):

UC-01: Register

UC-02: Login

UC-03: Logout

UC-04: Recover Password

UC-05: Change Password

UC-06: Manage User Profile

UC-32: Manage Users & Approvals (Phần duyệt hồ sơ Bác sĩ)

Các Thực thể ERD dự kiến (Core Entities):

Account (Lưu thông tin đăng nhập, Password Hash, Role).

Patient_Profile (Thông tin cá nhân, Địa chỉ giao hàng).

Dependent_Profile (Người phụ thuộc/Người thân của Patient).

Doctor_Profile (Chuyên khoa, Bằng cấp y khoa, Điểm đánh giá).

Admin_Profile / Pharmacist_Profile.

1. Tổng quan: Mục đích chính của sơ đồ này là gì?

Trong thực tế, sơ đồ này đóng vai trò là **"Trạm Gác An Ninh" (Security Checkpoint) và "Bộ phận Nhân sự/Chăm sóc Khách hàng"** của toàn bộ phòng khám/bệnh viện WebMed.

Nó giải quyết 3 bài toán cốt lõi:

1. **Ai đang đứng trước cửa? (Authentication):** Xác minh danh tính qua Email/SĐT và Mật khẩu.
2. **Họ được phép làm gì? (Authorization):** Phân loại người vào là Bệnh nhân, Bác sĩ hay Quản trị viên.

3. **Họ là ai trong đời thực? (Identity):** Lưu trữ hồ sơ nhân thân, bằng cấp y khoa, địa chỉ nhà, và cả những người phụ thuộc (con cái/cha mẹ) mà họ chịu trách nhiệm giám hộ.

Mọi phân hệ khác (Khám bệnh, Mua thuốc) muốn hoạt động đều phải "hỏi" phân hệ này để biết người đang thao tác là ai.

---

2. Các thực thể đại diện cho đối tượng/quy trình nào & Chi tiết Thực thể

Hãy tưởng tượng bạn bước vào một bệnh viện, các thực thể này chính là những "món đồ" hoặc "tờ giấy" bạn phải làm việc:

- **`ACCOUNT` (Tài khoản):** Đại diện cho **"Chiếc chìa khóa"**. Nó không quan tâm bạn tên gì, nó chỉ giữ `email`, `password_hash` (mật khẩu), và trạng thái khóa/mở (`locked_until`). Nếu bạn gõ sai pass quá 5 lần (`failed_login_attempts`), nó sẽ khóa cửa không cho bạn vào.
- **`ROLE` & `ACCOUNT_ROLE` (Vai trò):** Đại diện cho **"Thẻ đeo ngực"**. Thẻ này quy định bạn là Khách (Patient) hay Bác sĩ (Doctor). Một người có thể đeo nhiều thẻ (vừa là bác sĩ khám bệnh, vừa là bệnh nhân đi mua thuốc).
- **`USER_PROFILE` (Hồ sơ cơ bản):** Đại diện cho **"Tờ khai nhân thân"**. Bất kể bạn đeo thẻ gì, bạn đều phải khai tờ này gồm `full_name` (Họ tên) và `date_of_birth` (Ngày sinh).
- **`DOCTOR_PROFILE` (Hồ sơ Bác sĩ):** Đại diện cho **"Kẹp hồ sơ chuyên môn"**. Chỉ những ai nộp đơn làm bác sĩ mới phải đính kèm tờ này, ghi rõ Số chứng chỉ hành nghề (`medical_license_number`) và nộp bản sao bằng cấp (`credentials_url`).
- **`DEPENDENT_PROFILE` (Người phụ thuộc):** Đại diện cho **"Sổ hộ khẩu thu nhỏ"**. Dùng để khai báo những người không thể tự dùng app (trẻ em, người già) để người nắm `ACCOUNT` (ví dụ: Bố/Mẹ) có thể đặt lịch khám thay cho họ.
- **`DELIVERY_ADDRESS` (Địa chỉ): "Sổ địa chỉ"** dùng để shipper giao thuốc tới nhà.
- **`USER_SESSION` (Phiên làm việc):** Đại diện cho **"Vé vào cửa có thời hạn"**. Khi bạn đăng nhập thành công, bạn nhận 1 vé (`session_token`). Tránh việc bạn copy vé đưa cho người khác, vé này có giờ hết hạn (`expires_at`).
- **`OTP_TOKEN` (Mã OTP):** Đại diện cho **"Tin nhắn SMS xác thực"** gửi về điện thoại bạn khi đăng ký hoặc quên mật khẩu.
- **`AUDIT_LOG` (Nhật ký):** Đại diện cho **"Camera an ninh"**. Ghi lại mọi hành động nhạy cảm: Ai (`actor`) đã duyệt hồ sơ hoặc khóa nick của ai (`target`) vào lúc nào (`timestamp`).

---

3. Phân tích Mối quan hệ & Quy tắc nghiệp vụ (Business Rules)

Các đường nối trong sơ đồ chính là các **Ràng buộc pháp lý / Quy định của phòng khám**:

1. Mối quan hệ 1-1 (Một - Một): `ACCOUNT ||--|| USER_PROFILE`

- **Quy tắc nghiệp vụ:** "Không chấp nhận khách hàng vô danh".

- **Giải thích:** Bắt buộc 1 Tài khoản đăng nhập phải có đúng 1 Hồ sơ nhân thân. Bạn không thể có email/pass để mua thuốc mà hệ thống lại không biết bạn tên gì, sinh năm bao nhiêu.

2. Mối quan hệ 1-0..1 (Một - Có hoặc Không): `ACCOUNT ||--o| DOCTOR_PROFILE`

- **Quy tắc nghiệp vụ:** "Kế thừa có điều kiện".
- **Giải thích:** 1 Tài khoản có thể có hoặc không có Hồ sơ Bác sĩ. Nếu bạn là Bệnh nhân, kẹp hồ sơ này của bạn là khoảng trống. Quy tắc này giúp hệ thống không bắt ép Bệnh nhân phải nhập "Số chứng chỉ y khoa".

3. Mối quan hệ 1-N (Một - Nhiều): `ACCOUNT ||--o{ DEPENDENT_PROFILE`

- **Quy tắc nghiệp vụ:** "Giới hạn độ tuổi & Quyền giám hộ pháp lý".
- **Giải thích:** Chiều tác động đi từ Account -> Dependent. Trẻ em dưới 18 tuổi không được phép tự tạo `ACCOUNT` độc lập. Đứa trẻ phải được tạo dưới dạng `DEPENDENT_PROFILE` và bị ràng buộc bởi `caregiver_account_id` (ID của tài khoản Bố/Mẹ). Nếu tài khoản Bố/Mẹ bị xóa, hồ sơ đứa trẻ cũng không thể tự đứng độc lập.

4. Mối quan hệ N-N được triệt tiêu (Nhiều - Nhiều): Bảng `ACCOUNT_ROLE`

- **Quy tắc nghiệp vụ:** "Linh hoạt chức vụ".
- **Giải thích:** Thay vì nối trực tiếp Account với Role, ta dùng bảng trung gian. Tại sao? Vì một `ACCOUNT` (Bác sĩ Nguyễn Văn A) có thể đóng 2 `ROLE` (Vừa là Doctor, vừa là Patient). Sự thiết kế này giúp Bác sĩ A không cần dùng 2 số điện thoại khác nhau để tạo 2 nick trên cùng một app.

---

4. Kịch bản thực tế (Use Case) chạy xuyên suốt sơ đồ

Hãy tưởng tượng một ngày của **Bác sĩ Dũng** trên hệ thống WebMed:

1. **Đăng ký (UC-01):** Dũng tải app, nhập SĐT và mật khẩu. Hệ thống tạo một dòng trong bảng **`ACCOUNT`**. Để xác minh SĐT là thật, hệ thống sinh ra một mã 6 số lưu vào **`OTP_TOKEN`**.
2. **Khai báo hồ sơ:** Sau khi nhập mã OTP đúng, Dũng khai báo Tên/Tuổi (lưu vào **`USER_PROFILE`**) và tải lên bằng Cử nhân Y khoa (lưu vào **`DOCTOR_PROFILE`**). Hệ thống cấp cho Dũng chức danh Bác sĩ (gắn vào **`ACCOUNT_ROLE`**). Lúc này, nick của Dũng đang ở trạng thái "Pending" (Chờ duyệt).
3. **Duyệt hồ sơ (UC-32):** Admin đăng nhập, xem xét bằng cấp của Dũng hợp lệ bèn bấm "Approve". Hành động của Admin lập tức bị "Camera" **`AUDIT_LOG`** ghi lại: *Admin X đã duyệt hồ sơ cho Account Dũng vào lúc 10:00 AM*.
4. **Đăng nhập & Sử dụng (UC-02):** Hôm sau, Dũng đăng nhập. Hệ thống phát cho Dũng một vé **`USER_SESSION`** có thời hạn 30 ngày để dùng app.

5. **Mở rộng:** Dũng có một cậu con trai 5 tuổi bị ốm. Dũng không tạo tài khoản cho con mà vào app, thêm tên con vào bảng **`DEPENDENT_PROFILE`**, đồng thời thêm địa chỉ nhà vào **`DELIVERY_ADDRESS`** để tiện mua thuốc ship về nhà.

Mermaid:

```
erDiagram
  ACCOUNT ||--o{ ACCOUNT_ROLE : has
  ROLE ||--o{ ACCOUNT_ROLE : assigned_to
  ACCOUNT ||--|| USER_PROFILE : owns
  ACCOUNT ||--o| DOCTOR_PROFILE : extends_as
  ACCOUNT ||--o{ DEPENDENT_PROFILE : cares_for
  ACCOUNT ||--o{ DELIVERY_ADDRESS : manages
  ACCOUNT ||--o{ USER_SESSION : creates
  ACCOUNT ||--o{ OTP_TOKEN : generates
  ACCOUNT ||--o{ AUDIT_LOG : tracks_actions

  ACCOUNT {
    int account_id PK
    string email
    string phone_number
    string password_hash
    string status
    int failed_login_attempts
    datetime locked_until
  }

  ROLE {
```

```
    int role_id PK
    string role_name
}

ACCOUNT_ROLE {
    int account_id PK, FK
    int role_id PK, FK
}

USER_PROFILE {
    int profile_id PK
    int account_id FK
    string full_name
    date date_of_birth
}

DOCTOR_PROFILE {
    int doctor_profile_id PK
    int account_id FK
    string medical_license_number
    string credentials_url
    string rejection_reason
}

DEPENDENT_PROFILE {
    int dependent_id PK
```

```
    int caregiver_account_id FK
    string full_name
    date date_of_birth
    string relationship
}

DELIVERY_ADDRESS {
    int address_id PK
    int account_id FK
    string address_line
    boolean is_default
}

USER_SESSION {
    int session_id PK
    int account_id FK
    string session_token
    datetime expires_at
    boolean is_active
}

OTP_TOKEN {
    int token_id PK
    int account_id FK
    string otp_code
    string purpose
```

```
    datetime expires_at
    boolean is_used
  }

  AUDIT_LOG {
    int log_id PK
    int actor_account_id FK
    int target_account_id FK
    string action_type
    datetime timestamp
    string ip_address
  }
```

Phân hệ 2: Telehealth & Clinical Records (Khám bệnh trực tuyến & Bệnh án)

Phân hệ xử lý lịch trình, cuộc gọi và lưu trữ hồ sơ y tế cá nhân.

Các Use Case bao hàm (9 UCs):

UC-14: Find/Search Doctor

UC-15: View Doctor Information

UC-16: View Patient Medical History (Góc nhìn Bác sĩ)

UC-17: Book Consultation

UC-18: Manage Doctor Schedule

UC-19: Provide prescription

UC-28: View Medical History (Góc nhìn Bệnh nhân)

UC-30: Submit Feedback

UC-33: Manage booking consultation (Admin hủy/dời lịch)

Các Thực thể ERD dự kiến (Core Entities):

Doctor_Schedule (Khung giờ làm việc trống của Bác sĩ).

Appointment (Trạng thái lịch khám: Pending, Confirmed, Completed...).

Medical_Record (Ghi chú lâm sàng, Chẩn đoán bệnh).

E_Prescription (Đơn thuốc điện tử - Nối sang Phân hệ 3).

Feedback_Review (Đánh giá sao, Nhận xét).

1. Tổng quan: Mục đích chính của sơ đồ này là gì?

Sơ đồ này được thiết kế để số hóa và tự động hóa **toàn bộ vòng đời của một ca khám bệnh (Consultation Lifecycle)**. Nó giải quyết 3 bài toán kinh doanh cốt lõi:

1. **Khớp lệnh Cung - Cầu (Matchmaking):** Kết nối nhu cầu khám của bệnh nhân với chuyên môn và thời gian rảnh của bác sĩ.
2. **Quản lý Vận hành (Operations):** Đảm bảo không có tình trạng trùng lịch (Double-booking) và theo dõi trạng thái ca khám (Chờ khám, Đã khám, Đã hủy).
3. **Quản lý Hồ sơ Y tế (Clinical Integrity):** Lưu trữ bệnh án, đơn thuốc một cách hợp pháp, có tính kế thừa và tuyệt đối bảo mật.

---

2. Chi tiết Thực thể: Đại diện cho quy trình nào trong thực tế?

Hãy hình dung quy trình vận hành của một bệnh viện, các thực thể này đóng vai trò như sau:

- **`SPECIALTY` & `DOCTOR_SPECIALTY`:** Đại diện cho **"Bảng tên Khoa"**. Giúp bệnh nhân biết bác sĩ này chữa Tim mạch hay Da liễu để tìm kiếm (UC-14).
- **`DOCTOR_SCHEDULE`:** Đại diện cho **"Kho hàng Thời gian"** của bác sĩ. Mỗi slot (VD: 8:00 - 8:30) là một "món hàng" được đưa lên kệ để bệnh nhân chọn mua.
- **`APPOINTMENT`:** Đại diện cho **"Tờ Phiếu Khám" (Hợp đồng khám bệnh)**. Nó ghi nhận ai khám (`patient_id`), khám cho ai (`dependent_id`), bác sĩ nào (`doctor_id`), khung giờ nào (`slot_id`), và lý do đến khám là gì.
- **`MEDICAL_RECORD`:** Đại diện cho **"Sổ Y bạ / Hồ sơ Bệnh án"**. Đây là văn bản pháp lý chỉ được bác sĩ viết ra *sau khi* ca khám đã kết thúc, ghi rõ triệu chứng, chẩn đoán và lời dặn.
- **`LAB_ATTACHMENT`:** Đại diện cho **"Kẹp giấy tờ đính kèm"**. Nơi lưu các file ảnh chụp X-Quang, siêu âm, xét nghiệm máu của bệnh nhân.
- **`PRESCRIPTION` & `MEDICATION` & `PRESCRIPTION_ITEM`:** Đại diện cho **"Tờ Đơn thuốc"**.
  - `PRESCRIPTION`: Tờ giấy bìa ghi thông tin chung (Ngày kê, hạn dùng).
  - `MEDICATION`: Cuốn "Từ điển Thuốc" của phòng khám.
  - `PRESCRIPTION_ITEM`: Từng dòng chữ bác sĩ viết trên tờ đơn (Tên thuốc + Uống 2 viên/ngày).

- **`FEEDBACK`:** Đại diện cho **"Hòm thư góp ý"**. Nơi bệnh nhân đánh giá sao (Rating) sau khi khám xong.

---

3. Mối quan hệ & Các Quy tắc Nghiệp vụ (Business Rules) bị ràng buộc

Đây là phần thú vị nhất, các đường nối trong Mermaid chính là những "Điều luật vận hành" cực kỳ chặt chẽ:

1. Ràng buộc Chống Trùng Lịch (1-1/1-0): `DOCTOR_SCHEDULE ||--o| APPOINTMENT`

- **Quy tắc nghiệp vụ:** "Một khung giờ rảnh chỉ phục vụ đúng 1 ca khám".
- **Giải thích:** Nếu slot 9:00 sáng đã bị một `APPOINTMENT` móc vào (reserved_by), hệ thống sẽ khóa slot đó lại. Người thứ 2 vào xem sẽ không thấy slot này nữa, triệt tiêu hoàn toàn rủi ro bác sĩ phải tiếp 2 bệnh nhân cùng lúc.

2. Ràng buộc Pháp lý Y khoa (1-1/1-0): `APPOINTMENT ||--o| MEDICAL_RECORD`

- **Quy tắc nghiệp vụ:** "Không có lịch khám hợp lệ thì không có bệnh án".
- **Giải thích:** Bệnh án (`MEDICAL_RECORD`) không thể tự nhiên sinh ra. Nó bắt buộc phải bắt nguồn từ một cuộc hẹn (`APPOINTMENT`). Thêm nữa, quan hệ là 1-1, nghĩa là một cuộc hẹn chỉ sinh ra ĐÚNG 1 bệnh án duy nhất. Tránh việc bác sĩ chẩn đoán 2 bệnh án trái ngược nhau cho cùng 1 lần khám.

3. Ràng buộc Chống Spam Đánh giá (1-1/1-0): `APPOINTMENT ||--o| FEEDBACK`

- **Quy tắc nghiệp vụ:** "Khám xong mới được đánh giá, và chỉ đánh giá 1 lần".
- **Giải thích:** Bạn không thể review một bác sĩ nếu bạn chưa từng book lịch người đó. Bảng `FEEDBACK` lấy `appointment_id` làm khóa ngoại (FK) và khóa này là duy nhất (Unique). Do đó, đối thủ cạnh tranh không thể dùng 1 nick ảo để spam 100 cái review 1 sao cho bác sĩ được.

4. Ràng buộc Tính chính xác của Đơn thuốc (N-N): `PRESCRIPTION_ITEM`

- **Quy tắc nghiệp vụ:** "Lưu giữ khoảnh khắc kê đơn".
- **Giải thích:** Tại sao không nối thẳng Đơn thuốc (`PRESCRIPTION`) với Thuốc (`MEDICATION`)? Vì một tờ đơn có nhiều loại thuốc, một loại thuốc xuất hiện trong nhiều tờ đơn (Nhiều - Nhiều). Bảng trung gian `PRESCRIPTION_ITEM` ra đời để lưu lại `dosage` (Liều lượng: 2 viên) và `frequency` (Tần suất: Sáng/Tối). Việc này đảm bảo nếu ngày mai Admin có xóa/sửa tên thuốc trong danh mục, thì toa thuốc cũ của bệnh nhân vẫn giữ nguyên bằng chứng pháp lý bác sĩ đã dặn uống thế nào.

5. Ủy quyền Đặt lịch (1-N): DEPENDENT_PROFILE ||--o{ APPOINTMENT

- **Quy tắc nghiệp vụ:** "Khám cho ai thì bệnh án lưu vào người đó".

- **Giải thích:** Khi người Mẹ (`PATIENT_PROFILE`) đặt lịch, bà ấy là người trả tiền (Account books). Nhưng người đi khám lại là đứa con nhỏ (`DEPENDENT_PROFILE`). Thiết kế này giúp Bác sĩ biết chính xác đối tượng ngồi trước camera là một em bé, và bệnh án sau này sẽ lưu đúng vào lịch sử y tế của em bé đó chứ không bị lẫn vào hồ sơ của người mẹ.

---

4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống

Hãy xem các thực thể này phối hợp với nhau mượt mà như thế nào qua câu chuyện của **Chị Mai và bé Bi (5 tuổi)**:

1. **Tìm kiếm (UC-14):** Nửa đêm bé Bi sốt cao, chị Mai mở app tìm bác sĩ Khoa Nhi (`SPECIALTY`). Hệ thống lọc ra Bác sĩ Dũng đang có một khung giờ trống lúc 8:00 sáng mai trong **`DOCTOR_SCHEDULE`**.
2. **Đặt lịch (UC-17):** Chị Mai đặt lịch cho bé Bi. Hệ thống tạo một **`APPOINTMENT`** với `patient_id` là chị Mai (để đóng tiền), nhưng `dependent_id` là bé Bi (để bác sĩ biết ai bệnh). Khung giờ 8:00 sáng lập tức bị khóa lại.
3. **Khám bệnh & Bệnh án (UC-16, UC-19):** 8:00 sáng, bác sĩ Dũng video call với hai mẹ con. Khám xong, bác sĩ gõ chẩn đoán "Viêm họng cấp" vào **`MEDICAL_RECORD`** (móc nối với cuộc hẹn lúc 8:00). Đồng thời, bác sĩ kê một tờ **`PRESCRIPTION`** gồm 2 món **`PRESCRIPTION_ITEM`** (Thuốc hạ sốt và Vitamin C lấy từ danh mục **`MEDICATION`**).
4. **Hậu mãi (UC-30):** Chiều hôm đó, chị Mai thấy bé Bi đỡ sốt nên vào app, thả 5 sao kèm lời cảm ơn vào bảng **`FEEDBACK`** (hệ thống lưu lại review này ứng với mã cuộc hẹn của sáng nay).

Mermaid:

```
erDiagram

  DOCTOR_PROFILE ||--o{ DOCTOR_SPECIALTY : has_specialty

  SPECIALTY ||--o{ DOCTOR_SPECIALTY : assigned_to

  DOCTOR_PROFILE ||--o{ DOCTOR_SCHEDULE : configures_slots

  DOCTOR_PROFILE ||--o{ APPOINTMENT : assigned_to

  PATIENT_PROFILE ||--o{ DEPENDENT_PROFILE : manages

  PATIENT_PROFILE ||--o{ APPOINTMENT : books

  DEPENDENT_PROFILE ||--o{ APPOINTMENT : booked_for

  DOCTOR_SCHEDULE ||--o| APPOINTMENT : reserved_by

  APPOINTMENT ||--o| MEDICAL_RECORD : generates
```

```
APPOINTMENT ||--o| FEEDBACK : receives
MEDICAL_RECORD ||--o{ LAB_ATTACHMENT : contains
MEDICAL_RECORD ||--o| PRESCRIPTION : issues
PRESCRIPTION ||--o{ PRESCRIPTION_ITEM : includes
MEDICATION ||--o{ PRESCRIPTION_ITEM : prescribed_as

DOCTOR_PROFILE {
   int doctor_id PK
   string bio
   int experience_years
   decimal consultation_fee
   float average_rating
   int total_reviews
}

SPECIALTY {
   int specialty_id PK
   string specialty_name
}

DOCTOR_SPECIALTY {
   int doctor_id PK, FK
   int specialty_id PK, FK
}

DOCTOR_SCHEDULE {
```

```
    int slot_id PK
    int doctor_id FK
    date slot_date
    time start_time
    time end_time
    string status
}

APPOINTMENT {
    int appointment_id PK
    int patient_id FK
    int dependent_id FK
    int doctor_id FK
    int slot_id FK
    string consultation_type
    string reason_for_visit
    string status
    string cancellation_reason
}

MEDICAL_RECORD {
    int record_id PK
    int appointment_id FK
    string patient_symptoms
    string diagnosis
    string medical_advice
```

```
    datetime created_at
}

LAB_ATTACHMENT {
    int attachment_id PK
    int record_id FK
    string file_url
    string file_type
}

PRESCRIPTION {
    int prescription_id PK
    int record_id FK
    int doctor_id FK
    int patient_id FK
    datetime issue_date
    datetime expiry_date
    string qr_code_url
}

MEDICATION {
    int medication_id PK
    string medication_name
    boolean is_active
}
```

```
PRESCRIPTION_ITEM {
    int item_id PK
    int prescription_id FK
    int medication_id FK
    string dosage
    string frequency
    int duration_days
}

FEEDBACK {
    int feedback_id PK
    int appointment_id FK
    int rating_score
    string review_text
}
```

Phân hệ 3: E-Pharmacy & Inventory (Nhà thuốc trực tuyến)

Phân hệ thương mại điện tử (E-commerce) quản lý vòng đời đơn hàng và tồn kho.

Các Use Case bao hàm (5 UCs):

UC-20: Manage Shopping Cart

UC-21: Order Medicine

UC-22: Verify Prescription (Duyệt đơn Rx)

UC-23: Manage Order Processing (Đóng gói, Giao hàng)

UC-24: Track Order Status

Các Thực thể ERD dự kiến (Core Entities):

Product_Medicine (Master data của thuốc: Tên, Giá, Loại Rx hay OTC).

Inventory (Số lượng tồn kho vật lý).

Shopping_Cart & Cart_Item (Giỏ hàng tạm thời).

Medicine_Order (Thông tin giao hàng, Trạng thái đơn).

Order_Line_Item (Chi tiết từng món thuốc trong đơn).

1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?

Sơ đồ này được thiết kế để giải quyết 3 bài toán lõi của một Nhà thuốc trực tuyến:

1. **Thương mại điện tử chuẩn mực:** Quản lý vòng đời mua sắm từ lúc bỏ vào giỏ hàng (Cart) đến lúc chốt đơn và theo dõi giao hàng (Tracking).
2. **Quản trị Tồn kho thời gian thực (Real-time Inventory):** Ngăn chặn tuyệt đối tình trạng "Bán vượt mức" (Overselling) - khách đặt mua thành công nhưng kho lại hết thuốc.
3. **Gác cổng Y tế (Rx Gatekeeping):** Kiểm soát nghiêm ngặt việc bán Thuốc kê đơn (Rx). Không có đơn thuốc hợp lệ, hoặc Dược sĩ chưa duyệt, thì tuyệt đối không được phép thanh toán.

---

2. Chi tiết Thực thể: Đại diện cho đối tượng/quy trình nào trong thực tế?

Hãy tưởng tượng bạn bước vào một nhà thuốc lớn, các thực thể này chính là những vật dụng và quy trình tại đó:

- **`PRODUCT_MEDICINE` (Danh mục thuốc):** Đại diện cho **"Cuốn Catalog Sản phẩm"**. Nó chứa thông tin bất biến: Tên thuốc, Hình ảnh, Giá bán. Đặc biệt là cột cờ `is_rx_required` (Thuốc này có bắt buộc kê đơn hay không?).
- **`INVENTORY` (Tồn kho):** Đại diện cho **"Kệ hàng trong kho"**. Nó giữ thông số sống còn: `qty_available` (Số hộp thuốc đang cất trên kệ) và `qty_reserved` (Số hộp thuốc đang được nhân viên cầm ra quầy chờ khách trả tiền).
- **`SHOPPING_CART` & `CART_ITEM`:** Đại diện cho **"Chiếc Giỏ Nhựa"** bạn cầm đi vòng quanh siêu thị. Bạn có thể bỏ thuốc vào (`CART_ITEM`), đổi ý bỏ ra, hoặc tăng giảm số lượng thoải mái.
- **`PRESCRIPTION_DOCUMENT`:** Đại diện cho **"Tờ Đơn thuốc cầm tay"**. Đây có thể là đơn thuốc bác sĩ vừa khám cấp cho bạn (E-Prescription) hoặc tờ giấy bạn tự chụp ảnh tải lên app.
- **`MEDICINE_ORDER` & `ORDER_LINE_ITEM`:** Đại diện cho **"Tờ Hóa đơn tính tiền"** (Receipt). Khi bạn ra quầy thanh toán, nhân viên chốt sổ. Những gì ghi trên hóa đơn này là bất biến (Mã đơn, phí ship, số lượng mua, giá tiền tại thời điểm mua).
- **`ORDER_TRACKING`:** Đại diện cho **"Mã vận đơn bưu điện"**. Ghi nhật ký từng bước: Đã đóng gói lúc 8h, Đang giao lúc 9h, Đã nhận lúc 10h.

3. Mối quan hệ (Relationship) & Quy tắc Nghiệp vụ (Business Rules)

Các đường nối PK-FK ở đây ràng buộc những quy tắc kinh doanh sắc bén nhất của E-commerce:

1. Ràng buộc "Giỏ hàng vĩnh cửu" (1-1): `PATIENT_PROFILE ||--|| SHOPPING_CART`

- **Quy tắc nghiệp vụ:** Bệnh nhân không bị mất đồ đã chọn khi vô tình đóng app.
- **Giải thích:** Quan hệ 1-1 nghĩa là mỗi tài khoản chỉ có đúng 1 giỏ hàng duy nhất và nó được lưu cứng trong Database (chứ không lưu tạm ở trình duyệt). Bạn đang chọn dở bằng điện thoại, mở máy tính lên giỏ hàng vẫn còn nguyên.

2. Quy tắc "Bảo vệ Lịch sử Kế toán" (N-N): Bảng `ORDER_LINE_ITEM`

- **Quy tắc nghiệp vụ:** Giá thuốc hôm nay tăng không được làm sai lệch hóa đơn của tháng trước.
- **Giải thích:** Thuốc (Product) và Đơn hàng (Order) là quan hệ Nhiều-Nhiều. Bảng chi tiết `ORDER_LINE_ITEM` ra đời để chứa thuộc tính cực kỳ quan trọng: `unit_price_at_purchase`.
    - Nếu không có cột này, hóa đơn sẽ gọi giá từ bảng Product. Tháng 1 bạn mua Panadol giá 10k. Tháng 2 Admin tăng giá Panadol lên 15k. Bạn mở lại lịch sử hóa đơn tháng 1 sẽ thấy mình bị tính 15k (sai lệch kế toán). Việc lưu cứng giá tại thời điểm mua vào bảng này là bắt buộc!

3. Tách biệt Tồn kho (1-1): PRODUCT_MEDICINE ||--|| INVENTORY

- **Quy tắc nghiệp vụ:** Đảm bảo hiệu suất (Performance) và chống bán lố (Over-selling).
- **Giải thích:** Tại sao không gộp số lượng tồn kho vào bảng Product luôn? Vì người dùng có thói quen "Search" (Tìm kiếm thuốc) liên tục. Nếu gộp chung, mỗi lần có người mua thuốc, database sẽ khóa (lock) dòng dữ liệu đó để trừ tồn kho, làm những người đang search bị đứng máy. Việc tách bảng `INVENTORY` giúp hệ thống chịu tải cao hơn. Cột `qty_reserved` (tạm giữ) đảm bảo khi bạn bấm "Thanh toán", thuốc được "giữ chỗ" để người khác không mua mất trong 10 phút bạn nhập thẻ tín dụng.

4. Ràng buộc "Gác cổng Y khoa" (1-N): PRESCRIPTION_DOCUMENT ||--o{ MEDICINE_ORDER

- **Quy tắc nghiệp vụ:** Mua kháng sinh thì phải trình đơn thuốc (BR-12).
- **Giải thích:** Khóa ngoại `document_id` trong bảng `MEDICINE_ORDER` có thể rỗng (nếu bạn mua vitamin/OTC). Nhưng nếu giỏ hàng có chứa thuốc `is_rx_required = True`, hệ thống ép buộc phải có ID của đơn thuốc nối vào đây. Đơn hàng này sẽ bị treo (Pending) để Dược sĩ (`PHARMACIST_PROFILE`) vào kiểm duyệt trước khi cho phép thanh toán.

5. Ghi vết Hành trình (1-N): MEDICINE_ORDER ||--o{ ORDER_TRACKING

- **Quy tắc nghiệp vụ:** Minh bạch quá trình vận chuyển.

- **Giải thích:** Một đơn hàng (1) sẽ có nhiều (N) cột mốc trạng thái. Việc thiết kế bảng `ORDER_TRACKING` riêng giúp Frontend dễ dàng vẽ ra một cái "Timeline" (Đường thời gian có các chấm tròn) cho bệnh nhân theo dõi (UC-24) y hệt như mua hàng Shopee.

---

4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống

Hãy xem cách dữ liệu luân chuyển qua kịch bản: **Anh Hoàng bị ho và mua thuốc Kháng sinh (Thuốc Rx)**:

1. **Quản lý Giỏ hàng (UC-20):** Anh Hoàng dạo siêu thị thuốc, tìm thấy Kháng sinh Augmentin (`PRODUCT_MEDICINE` có `is_rx_required = True`). Anh bấm Thêm vào giỏ. Hệ thống chèn 1 dòng vào **`CART_ITEM`** nối với **`SHOPPING_CART`** của anh Hoàng.
2. **Đặt hàng & Chống vượt mức (UC-21):** Anh bấm Checkout. Hệ thống check **`INVENTORY`**: Kho còn 10 hộp (`qty_available`). Hệ thống lập tức trừ đi 1 hộp, chuyển sang `qty_reserved` (tạm giữ cho Hoàng).
3. **Nộp đơn thuốc (Gác cổng):** Vì là thuốc Rx, app yêu cầu Hoàng chụp ảnh toa thuốc bác sĩ vừa kê hôm qua. Ảnh này được tải lên lưu vào **`PRESCRIPTION_DOCUMENT`**.
4. **Chốt đơn:** Hệ thống dọn sạch Giỏ hàng. Tạo ra một **`MEDICINE_ORDER`** (trạng thái: *Chờ duyệt đơn thuốc*) và **`ORDER_LINE_ITEM`** (lưu cứng giá Augmentin là 100k).
5. **Dược sĩ duyệt (UC-22):** Dược sĩ Tâm (`PHARMACIST_PROFILE`) đăng nhập, xem ảnh toa thuốc của Hoàng hợp lệ, bèn bấm "Approve". Đơn hàng chuyển sang trạng thái *Chờ thanh toán*. (Hệ thống ghi 1 dòng vào **`ORDER_TRACKING`**).
6. **Xử lý & Giao hàng (UC-23, UC-24):** Hoàng thanh toán xong, Dược sĩ Tâm đóng gói thuốc và bấm "Giao hàng". Hệ thống tiếp tục ném một dòng vào **`ORDER_TRACKING`**. Hoàng mở app ra xem và thấy chấm tròn hiển thị "Đơn hàng đang trên đường tới nhà".

erDiagram

```
PATIENT_PROFILE ||--|| SHOPPING_CART : owns

SHOPPING_CART ||--o{ CART_ITEM : contains

PRODUCT_MEDICINE ||--o{ CART_ITEM : added_as

PRODUCT_MEDICINE ||--|| INVENTORY : stocked_in


PATIENT_PROFILE ||--o{ PRESCRIPTION_DOCUMENT : uploads_or_owns

PRESCRIPTION_DOCUMENT ||--o{ MEDICINE_ORDER : validates_rx_for

PHARMACIST_PROFILE ||--o{ MEDICINE_ORDER : verifies
```

```
PATIENT_PROFILE ||--o{ MEDICINE_ORDER : places
MEDICINE_ORDER ||--o{ ORDER_LINE_ITEM : includes
PRODUCT_MEDICINE ||--o{ ORDER_LINE_ITEM : purchased_as

MEDICINE_ORDER ||--o{ ORDER_TRACKING : logs_milestone

SHOPPING_CART {
   int cart_id PK
   int patient_id FK
   datetime last_updated
}

CART_ITEM {
   int cart_item_id PK
   int cart_id FK
   int product_id FK
   int quantity
   datetime added_at
}

PRODUCT_MEDICINE {
   int product_id PK
   string product_name
   string product_image_url
   decimal unit_price
   boolean is_rx_required
```

```
}

INVENTORY {
    int inventory_id PK
    int product_id FK
    int qty_available
    int qty_reserved
}

PRESCRIPTION_DOCUMENT {
    int document_id PK
    int patient_id FK
    string file_url
    date issue_date
    boolean is_e_prescription
}

MEDICINE_ORDER {
    int order_id PK
    int patient_id FK
    int address_id FK
    int document_id FK
    int pharmacist_id FK
    decimal subtotal_amount
    decimal shipping_fee
    decimal total_amount
```

```
    string status
    string cancellation_reason
    datetime created_at
}

ORDER_LINE_ITEM {
    int line_item_id PK
    int order_id FK
    int product_id FK
    int quantity
    decimal unit_price_at_purchase
}

ORDER_TRACKING {
    int tracking_id PK
    int order_id FK
    string status_milestone
    int updated_by FK
    datetime created_at
}

PATIENT_PROFILE {
    int patient_id PK
}

PHARMACIST_PROFILE {
```

```
    int pharmacist_id PK
  }
```

Phân hệ 4: Billing & Finance (Tài chính & Thanh toán)

Phân hệ nhạy cảm nhất, nơi dòng tiền ra/vào hệ thống được ghi nhận lại.

Các Use Case bao hàm (5 UCs):

UC-25: Make Payment (Luồng gốc)

UC-26: Pay for Doctor Services

UC-27: Pay for Medicine

UC-34: Process Provider Payout (Trả lương Bác sĩ)

UC-35: Process Refund (Hoàn tiền)

Các Thực thể ERD dự kiến (Core Entities):

Transaction_Log (Ghi nhận mọi giao dịch: TransactionID, Số tiền, Trạng thái).

Escrow_Ledger (Sổ cái lưu trữ tiền đang bị giam của Bác sĩ theo BR-17).

Refund_Request (Yêu cầu hoàn tiền theo BR-15, BR-18).

Payout_Batch (Lịch sử chuyển tiền cho Bác sĩ).

1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?

Sơ đồ Tài chính này không chỉ đơn thuần là nơi cộng trừ tiền bạc, mà nó sinh ra để bảo vệ nền tảng khỏi 3 rủi ro cốt lõi:

1. **Minh bạch Dòng tiền (Audit & Compliance):** Ghi nhận chính xác dòng tiền vào (Inbound) và dòng tiền ra (Outbound) khớp đến từng đồng, phục vụ đối soát với Cổng thanh toán (VNPay, Momo, Stripe).
2. **Bảo vệ Quyền lợi Bệnh nhân (Escrow Mechanism):** Đảm bảo bệnh nhân trả tiền trước nhưng Bác sĩ chưa được nhận tiền ngay. Tiền sẽ bị "giam" lại cho đến khi ca khám hoàn tất tốt đẹp.

3. **Phòng chống Rửa tiền & Gian lận (AML & Anti-Fraud):** Đảm bảo tiền hoàn (Refund) phải trả về đúng thẻ ngân hàng đã quẹt ban đầu. Đảm bảo lương Bác sĩ được chuyển đúng số tài khoản đăng ký, không để Admin sửa bậy để biển thủ.

---

2. Chi tiết Thực thể: Đại diện cho quy trình/chứng từ nào trong thực tế?

Hãy tưởng tượng bạn đang xem sổ sách của Kế toán trưởng, các thực thể này là những loại chứng từ sau:

- **`TRANSACTION_LOG`:** Đại diện cho **"Sổ Sao Kê Ngân Hàng"**. Đây là sổ gốc chứa mọi giao dịch (Tiền khám, Tiền thuốc). Các thuộc tính quan trọng: `amount` (số tiền), `payment_method` (thẻ Visa, Momo), và đặc biệt là `gateway_txn_ref` (Mã giao dịch ngân hàng trả về làm bằng chứng đã thu tiền).
- **`ESCROW_LEDGER`:** Đại diện cho **"Két Sắt Tạm Giữ"** (Tài khoản Đảm bảo). Tiền nạp vào để khám bệnh sẽ bị nhốt ở đây. Trạng thái (`status`) của nó sẽ quyết định số phận dòng tiền: "Held" (Đang giam), "Released" (Đã nhả cho Bác sĩ), hoặc "Refunded" (Đã trả lại Bệnh nhân).
- **`PAYOUT_REQUEST`:** Đại diện cho **"Lệnh Ủy Nhiệm Chi / Bảng Lương"**. Đến cuối tháng, Admin tạo lệnh này để ngân hàng chuyển tiền thật vào tài khoản của Bác sĩ. Thuộc tính `receiving_bank_name` và `receiving_account_number` là **bất biến** (lưu cứng lúc tạo lệnh) để đối phó gian lận.
- **`ESCROW_PAYOUT_ITEM`:** Đại diện cho **"Bảng Kê Chi Tiết Lương"**. Đi kèm với Lệnh Ủy Nhiệm Chi 10 triệu đồng, phải có bảng kê giải thích rõ: 10 triệu này được cộng gộp từ 20 ca khám nào.
- **`REFUND_REQUEST`:** Đại diện cho **"Phiếu Đề Nghị Hoàn Tiền"**. Khi ca khám bị hủy, Kế toán lập phiếu này, ghi rõ lý do (`reason`) và số tiền cần hoàn (`refund_amount`).

---

3. Mối quan hệ (Relationship) & Các Quy tắc Nghiệp vụ (Business Rules)

Sơ đồ này là một tác phẩm nghệ thuật về kế toán (Double-entry bookkeeping). Các đường nối chính là các "Luật Kế Toán":

1. Phân luồng Doanh thu (1-1/1-0): `TRANSACTION_LOG ||--o| ESCROW_LEDGER`

- **Quy tắc nghiệp vụ:** "Tách bạch tiền giữ hộ và doanh thu trực tiếp".
- **Giải thích:** Một giao dịch thanh toán (Transaction) có thể sinh ra 1 khoản giam tiền (Escrow), hoặc KHÔNG CÓ (0). Tại sao?
    - Nếu bệnh nhân mua THUỐC, tiền chạy thẳng vào doanh thu Nhà thuốc -> Không sinh ra Escrow.
    - Nếu bệnh nhân đặt LỊCH KHÁM, tiền đó không phải của nền tảng mà là của Bác sĩ -> Hệ thống bắt buộc sinh ra 1 Escrow tương ứng.

2. Ràng buộc Trách nhiệm (1-1): `APPOINTMENT ||--|| ESCROW_LEDGER`

- **Quy tắc nghiệp vụ:** "Có làm thì mới có ăn - Không khám thì không có tiền".
- **Giải thích:** Mỗi một lịch khám (`APPOINTMENT`) bị khóa chặt 1-1 với một khoản tiền trong `ESCROW_LEDGER`. Trạng thái của Lịch khám sẽ điều khiển Trạng thái của Tiền. Khám xong (Completed) thì tiền mới được thả (Released).

3. Triệt tiêu N-N để gom bill: PAYOUT_REQUEST -- ESCROW_PAYOUT_ITEM -- ESCROW_LEDGER

- **Quy tắc nghiệp vụ:** "Chuyển khoản 1 lần cho hàng chục ca khám".
- **Giải thích:** Bác sĩ A khám 100 ca trong tháng, tạo ra 100 khoản tiền trong `ESCROW_LEDGER`. Admin không rảnh để bấm chuyển khoản 100 lần. Admin sẽ tạo ra ĐÚNG 1 cái `PAYOUT_REQUEST` (Lệnh chi). Bảng trung gian `ESCROW_PAYOUT_ITEM` sẽ làm nhiệm vụ gom 100 cái mã Escrow kia móc vào 1 cái Lệnh chi duy nhất.

4. Luật Chống Rửa Tiền AML (1-N): TRANSACTION_LOG ||--o{ REFUND_REQUEST

- **Quy tắc nghiệp vụ:** "Tiền từ đâu đến, phải quay về đúng chỗ đó" (BR-15, BR-18).
- **Giải thích:** Khóa ngoại `original_transaction_id` trong phiếu Hoàn tiền bắt buộc phải trỏ ngược về dòng Transaction nạp tiền ban đầu. Admin không có quyền gõ một số tài khoản mới để chuyển tiền hoàn. API ngân hàng sẽ đọc mã gốc này và tự động trả tiền ngược về đúng thẻ Visa/Momo mà bệnh nhân đã quẹt. Đây là tiêu chuẩn bảo mật PCI-DSS tối cao.

---

4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống

Hãy theo dõi đường đi của tờ 500.000 VNĐ từ túi **Bệnh nhân Hoàng** vào túi **Bác sĩ Dũng**:

1. **Thanh toán (UC-25, UC-26):** Hoàng dùng thẻ Visa đặt lịch khám bác sĩ Dũng giá 500k.

   -> Bảng **`TRANSACTION_LOG`** ghi nhận 500k vào hệ thống (Status: Success).

   -> Ngay lập tức, bảng **`ESCROW_LEDGER`** giam 500k này lại dưới tên Bác sĩ Dũng (Status: Held).

2. **Khám xong:** Cuộc gọi video kết thúc. Hệ thống tự động update **`ESCROW_LEDGER`** của Dũng (Status: Released - sẵn sàng rút).
3. **Trả lương (UC-34):** Cuối tuần, Admin vào xem. Thấy Dũng có 10 ca (tương đương 5 triệu đang Released). Admin bấm "Tạo lệnh thanh toán".

   -> Bảng **`PAYOUT_REQUEST`** sinh ra 1 dòng trị giá 5 triệu, ghi cứng số tài khoản Techcombank của Dũng.

-> Bảng **`ESCROW_PAYOUT_ITEM`** sinh ra 10 dòng, nối 10 ca khám của Dũng vào cái Lệnh 5 triệu này để làm chứng từ giải trình.

4. **Hủy lịch & Hoàn tiền (UC-35 - Rẽ nhánh):** Giả sử ở Bước 1, Hoàng đột ngột có việc bận nên bấm Hủy lịch.

   -> Admin duyệt Hủy. Bảng **`REFUND_REQUEST`** sinh ra 1 dòng 500k, trỏ thẳng vào cái Transaction Visa gốc của Hoàng.

   -> Cổng thanh toán (Stripe/VNPay) nhận lệnh, tự động đảo chiều (reverse) trả 500k về thẻ Visa cho Hoàng. Khoản giam trong **`ESCROW_LEDGER`** bị chuyển thành "Refunded" (Bác sĩ Dũng không nhận được đồng nào).

Mermaid:

```
erDiagram

  PATIENT_PROFILE ||--o{ TRANSACTION_LOG : initiates

  APPOINTMENT ||--o{ TRANSACTION_LOG : triggers

  MEDICINE_ORDER ||--o{ TRANSACTION_LOG : generates


  TRANSACTION_LOG ||--o| ESCROW_LEDGER : funds

  APPOINTMENT ||--|| ESCROW_LEDGER : locked_in

  DOCTOR_PROFILE ||--o{ ESCROW_LEDGER : earns


  DOCTOR_PROFILE ||--o{ PAYOUT_REQUEST : receives

  ADMIN_ACCOUNT ||--o{ PAYOUT_REQUEST : processes

  PAYOUT_REQUEST ||--o{ ESCROW_PAYOUT_ITEM : contains

  ESCROW_LEDGER ||--o| ESCROW_PAYOUT_ITEM : included_in


  TRANSACTION_LOG ||--o{ REFUND_REQUEST : source_of

  ADMIN_ACCOUNT ||--o{ REFUND_REQUEST : initiates_refund
```

```
TRANSACTION_LOG {
    int transaction_id PK
    int patient_id FK
    int appointment_id FK
    int order_id FK
    string payment_method
    decimal amount
    string gateway_txn_ref
    string status
    datetime created_at
}

ESCROW_LEDGER {
    int escrow_id PK
    int doctor_id FK
    int appointment_id FK
    int inbound_transaction_id FK
    decimal amount
    string status
    datetime created_at
}

PAYOUT_REQUEST {
    int payout_id PK
    int doctor_id FK
    int admin_id FK
```

```
    decimal total_amount
    string receiving_bank_name
    string receiving_account_number
    string gateway_payout_ref
    string status
    datetime processed_at
}

ESCROW_PAYOUT_ITEM {
    int payout_id PK, FK
    int escrow_id PK, FK
}

REFUND_REQUEST {
    int refund_id PK
    int original_transaction_id FK
    int admin_id FK
    decimal refund_amount
    string reason
    string gateway_refund_ref
    string status
    datetime created_at
    datetime processed_at
}

PATIENT_PROFILE {
```

```
    int patient_id PK
  }
  APPOINTMENT {
    int appointment_id PK
  }
  MEDICINE_ORDER {
    int order_id PK
  }
  DOCTOR_PROFILE {
    int doctor_id PK
  }
  ADMIN_ACCOUNT {
    int admin_id PK
  }
```

Phân hệ 5: CMS & Health Utilities (Quản trị Nội dung & Tiện ích)

Phân hệ chứa dữ liệu công khai (Public) và các công cụ tra cứu không ràng buộc khắt khe về định danh.

Các Use Case bao hàm (6 UCs):

UC-07: View Health A-Z

UC-08: Search Health A-Z

UC-09: Check Symptoms

UC-10: Health Calculator

UC-11: View News & Community Programs

UC-31: Manage Content (Admin quản trị nội dung)

Các Thực thể ERD dự kiến (Core Entities):

Content_Article (Tin tức, Bài viết Y khoa).

Disease_Dictionary (Dữ liệu từ điển bệnh lý Health A-Z).

Calculator_History (Lưu kết quả BMI, BMR nếu user muốn lưu).

1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?

Sơ đồ này được thiết kế để phục vụ 3 chiến lược kinh doanh cốt lõi:

- **Thu hút và Giữ chân người dùng (Inbound Marketing):** Cung cấp các bài viết y khoa, tin tức và bộ công cụ tính toán (BMI, BMR) hoàn toàn miễn phí, thậm chí không cần đăng nhập.
- **Hệ thống Phân loại Bệnh nhân (Triage Engine):** Giúp bệnh nhân tự kiểm tra triệu chứng (Symptom Checker). Từ đó hệ thống đưa ra lời khuyên "Có nên đi khám bác sĩ ngay không?", gián tiếp thúc đẩy doanh thu cho Phân hệ 2 (Đặt lịch khám).
- **Quản trị Rủi ro Y khoa (Medical Liability) & Truy vết Admin:** Mọi nội dung xuất bản đều phải được kiểm duyệt. Bất kỳ ai (Admin nào) tạo mới, chỉnh sửa bài viết hay ẩn bình luận đều bị hệ thống "bắt quả tang" và lưu trữ lịch sử tuyệt đối để đối chiếu pháp lý khi xảy ra sự cố.

2. Chi tiết Thực thể: Đại diện cho quy trình/đối tượng nào trong thực tế?

Hãy tưởng tượng Phân hệ này là một Thư viện Y khoa & Phòng Khám Sàng Lọc được kiểm soát an ninh nghiêm ngặt:

- **`CONTENT_CATEGORY`:** Đại diện cho "Các Tủ sách phân loại". Giúp người dùng dễ tìm bài viết (Tủ Dinh dưỡng, Tủ Bệnh lý, Tủ Mẹ & Bé).
- **`CMS_ARTICLE`:** Đại diện cho "Tờ Báo / Bản tin Y tế". Chứa các bài viết do chuyên gia soạn thảo. Các cột quan trọng: `title`, `status` (Bản nháp, Chờ duyệt, Đã xuất bản), và cặp khóa quyền lực `created_by_admin_id` (Ai viết), `updated_by_admin_id` (Ai sửa lần cuối).
- **`DISEASE_DICTIONARY`:** Đại diện cho "Cuốn Bách khoa toàn thư Y học". Là tri thức y khoa chuẩn mực (Định nghĩa bệnh, Nguyên nhân, Cách chữa). Tương tự Article, nó cũng bị giám sát bởi cặp khóa Audit của Admin.
- **`SYMPTOM`:** Đại diện cho "Bảng danh sách các dấu hiệu bệnh" (Ho, Sốt, Đau đầu, Buồn nôn).
- **`CONTENT_VERSION_HISTORY`:** Đại diện cho "Kho lưu trữ Hồ sơ cũ" (Archive). Nó lưu lại chính xác từng câu từng chữ của bài viết *trước khi* bị ai đó ấn nút sửa, kèm theo đích danh kẻ đã sửa (`modified_by_admin_id`).
- **`CALCULATOR_HISTORY`:** Đại diện cho "Cuốn sổ tay Sức khỏe cá nhân". Nơi hệ thống ghi nhớ các chỉ số cơ thể (Chiều cao, Cân nặng, BMI) nếu bệnh nhân muốn lưu lại theo dõi.

3. Mối quan hệ (Relationship) & Quy tắc Nghiệp vụ (Business Rules)

Sơ đồ này chứa những quy tắc cực kỳ quan trọng về quản lý tri thức và kiểm duyệt:

*1. Ma trận Bộ não AI (Quan hệ N-N triệt tiêu): **DISEASE_DICTIONARY** -- **DISEASE_SYMPTOM** -- **SYMPTOM***

- **Quy tắc nghiệp vụ:** "Một bệnh có nhiều triệu chứng, một triệu chứng là dấu hiệu của nhiều bệnh".
- **Giải thích:** Bảng trung gian `DISEASE_SYMPTOM` chính là bộ não của tính năng Check Symptoms (UC-09). Nó chứa thuộc tính cốt lõi: `matching_probability` (Trọng số xác suất).
    - *Ví dụ:* Triệu chứng "Sốt" nối với "Cảm cúm" (Xác suất 90%), nhưng nối với "Sốt xuất huyết" (Xác suất 99%). Dựa vào bảng này, thuật toán mới biết nên gợi ý người dùng bị bệnh gì lên đầu tiên.

*2. Ràng buộc Chống Trách nhiệm Pháp lý (1-N): Bảng Nội dung ||--o{ **CONTENT_VERSION_HISTORY***

- **Quy tắc nghiệp vụ:** "Không bao giờ được xóa dấu vết Y khoa".
- **Giải thích:** Trong y tế, nếu hôm nay bài viết ghi "Uống 10 viên Paracetamol", ngày mai có người nhập viện, Admin hoảng sợ vào sửa lại thành "Uống 2 viên". Bảng `CONTENT_VERSION_HISTORY` sẽ lưu một bản chụp (`content_snapshot_json`) của cái nội dung "10 viên" đó kèm theo thời gian và đích danh Admin nào đã sửa (`modified_by_admin_id`). Đây là bằng chứng không thể chối cãi trước tòa.

*3. Ràng buộc Kiểm soát Quyền lực Admin (1-N): **ADMIN_ACCOUNT** móc nối vào các bảng Nội dung*

- **Quy tắc nghiệp vụ:** "Quyền CRUD đi kèm Trách nhiệm giải trình".
- **Giải thích:** Hệ thống không dùng một ID tác giả chung chung. Mọi bài viết đều bị khóa chặt bởi 2 luồng: Kẻ tạo ra nó ban đầu (`created_by_admin_id`) và kẻ vừa chạm vào nó lần cuối cùng (`updated_by_admin_id`).

*4. Quyền Lực Kiểm Duyệt Minh Bạch (1-N): **ADMIN_ACCOUNT ||--o{ FEEDBACK***

- **Quy tắc nghiệp vụ:** "Bảo vệ danh dự Bác sĩ nhưng chống lạm quyền".
- **Giải thích:** Mặc dù `FEEDBACK` sinh ra từ Phân hệ Khám bệnh, nhưng Admin có quyền ẩn chúng. Đường nối này không chỉ đổi `status` thành "Hidden", mà còn ép hệ thống ghi lại `moderated_by_admin_id` (Đích danh Admin nào đã ẩn?) và `moderated_at` (Ẩn lúc nào?). Điều này ngăn chặn việc Admin lén lút che giấu bình luận xấu cho bác sĩ người nhà.

*5. Ràng buộc Lưu trữ Phù du (1-N / 0-N): **PATIENT_PROFILE ||--o{ CALCULATOR_HISTORY***

- **Quy tắc nghiệp vụ:** "Dữ liệu tính toán là tạm thời, trừ khi khách hàng muốn giữ lại".
- **Giải thích:** Khóa ngoại `patient_id` trong bảng này có thể là rỗng (Bản số `o{` - zero or many). Một người vãng lai không cần tạo tài khoản vẫn có thể lên app tính BMI (Hệ thống tính xong rồi xóa, không lưu DB). Nhưng nếu họ đã đăng nhập và bấm "Save", hệ thống mới ghim `patient_id` của họ vào kết quả để lưu trữ.

4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống

Hãy xem cách các luồng dữ liệu công cộng này tương tác qua câu chuyện của **Chị Linh và Bác sĩ Dũng (Admin)**:

1. **Tạo Tri thức (UC-31):** Bác sĩ Dũng (đăng nhập quyền Admin ID: 5) viết một trang Bách khoa toàn thư về bệnh "Sốt Xuất Huyết". Hệ thống lưu vào bảng `DISEASE_DICTIONARY` và tự động ghim ID 5 của Dũng vào cột `created_by_admin_id`.
2. **Huấn luyện thuật toán (UC-31):** BS Dũng vào bảng `DISEASE_SYMPTOM`, gán bệnh "Sốt Xuất Huyết" với 2 mã `SYMPTOM` là "Sốt cao" (Xác suất 95%) và "Phát ban đỏ" (Xác suất 80%).
3. **Cập nhật kiến thức & Lưu vết:** Vài tháng sau, Admin Lan (ID: 8) vào sửa lại bài viết của Dũng. Ngay khi bấm Save:
   - Hệ thống lén tạo 1 dòng trong `CONTENT_VERSION_HISTORY` để cất bản cũ đi (ghim `modified_by_admin_id` là 8 - Lan).
   - Cập nhật bản mới ra trang chủ (giữ nguyên người tạo là Dũng, nhưng đổi `updated_by_admin_id` thành 8 - Lan).
4. **Sàng lọc tự động (UC-09):** Chị Linh (chưa đăng nhập) thấy mệt, mở tính năng Check Symptoms. Chị tick chọn "Sốt cao" và "Phát ban đỏ". Hệ thống chui vào bảng `DISEASE_SYMPTOM` tính toán, phát hiện trùng khớp cao nhất, hiển thị: *"Cảnh báo: Bạn có nguy cơ mắc Sốt xuất huyết. Vui lòng đặt lịch khám bác sĩ ngay!"*
5. **Tiện ích (UC-10):** Lo lắng về sức khỏe, Linh đăng nhập tài khoản. Chị đo BMI và bấm "Save". Hệ thống ném 1 dòng chứa Chiều cao, Cân nặng, BMI vào bảng `CALCULATOR_HISTORY` khóa chặt với `patient_id` của Linh.

Mermaid:

```
erDiagram

  CONTENT_CATEGORY ||--o{ CMS_ARTICLE : classifies

  ADMIN_ACCOUNT ||--o{ CMS_ARTICLE : authors_and_moderates

  ADMIN_ACCOUNT ||--o{ DISEASE_DICTIONARY : authors

  CMS_ARTICLE ||--o{ CONTENT_VERSION_HISTORY : tracks_versions_of

  DISEASE_DICTIONARY ||--o{ CONTENT_VERSION_HISTORY : tracks_versions_of

  ADMIN_ACCOUNT ||--o{ CONTENT_VERSION_HISTORY : modifies

  DISEASE_DICTIONARY ||--o{ DISEASE_SYMPTOM : exhibits

  SYMPTOM ||--o{ DISEASE_SYMPTOM : indicates
```

```
PATIENT_PROFILE ||--o{ CALCULATOR_HISTORY : explicitly_saves
ADMIN_ACCOUNT ||--o{ FEEDBACK : moderates_status

CONTENT_CATEGORY {
   int category_id PK
   string category_name
}

CMS_ARTICLE {
   int article_id PK
   int category_id FK
   int author_id FK
   string title
   string cover_image_url
   string body_content
   string status
   datetime published_at
}

CONTENT_VERSION_HISTORY {
   int version_id PK
   int reference_id FK
   string content_type
   string content_snapshot_json
   int modified_by FK
   datetime modified_at
```

```
}

DISEASE_DICTIONARY {
    int disease_id PK
    int author_id FK
    string disease_title
    string symptoms_text
    string causes
    string treatments
    string prevention
    string status
}

SYMPTOM {
    int symptom_id PK
    string symptom_name
}

DISEASE_SYMPTOM {
    int disease_id PK, FK
    int symptom_id PK, FK
    decimal matching_probability
}

CALCULATOR_HISTORY {
    int calc_id PK
```

```
    int patient_id FK
    string biological_gender
    int age
    decimal height_cm
    decimal weight_kg
    string activity_level
    decimal bmi_score
    string bmi_classification
    decimal bmr_kcal
    decimal tdee_kcal
    datetime saved_at
}

ADMIN_ACCOUNT {
    int admin_id PK
}

PATIENT_PROFILE {
    int patient_id PK
}

FEEDBACK {
    int feedback_id PK
    string status
}
```

Phân hệ 6: Third-Party & System Ops (Tích hợp & Vận hành lõi)

Phân hệ chứa các luồng giao tiếp với bên thứ 3 và báo cáo hệ thống tổng.

Các Use Case bao hàm (4 UCs):

UC-12: View Insurance Guide

UC-13: Purchase Insurance

UC-29: Request Ambulance

UC-36: View System Reports

Các Thực thể ERD dự kiến (Core Entities):

Insurance_Policy (Gói bảo hiểm từ đối tác).

Ambulance_Dispatch (Lịch sử gọi xe cấp cứu, Tọa độ GPS).

System_Audit_Log (Bảng Log ghi lại thao tác của Admin và System để phục vụ báo cáo UC-36).

1. Tổng quan: Mục đích chính của sơ đồ này là để giải quyết bài toán gì?

Sơ đồ này được thiết kế để xử lý 3 luồng vận hành tối cao:

1. **Mở rộng Hệ sinh thái (Ecosystem Integration):** WebMed không tự kinh doanh Bảo hiểm hay có sẵn đội xe Cấp cứu, mà đóng vai trò là "Sàn giao dịch" (Broker). Hệ thống phải kết nối mượt mà với các Đối tác thứ 3 (PTI, Bảo Việt, Trung tâm 115).
2. **Tính Bất biến Pháp lý (Legal Immutability):** Đảm bảo các hợp đồng bảo hiểm khi xuất ra phải tuân thủ tuyệt đối quy định của Pháp luật, không bị ảnh hưởng bởi những thay đổi thông tin cá nhân trên app sau này.
3. **Giám sát Toàn diện (System Monitoring):** Cung cấp nguyên liệu (dữ liệu log) để tạo ra các Dashboard Báo cáo BI (Business Intelligence) và theo dõi dấu vết của hệ thống Quản trị.

---

2. Chi tiết Thực thể: Đại diện cho quy trình/chứng từ nào trong thực tế?

Hãy hình dung đây là khu vực **Hành chính - Pháp lý** của hệ thống:

- **`INSURANCE_PROVIDER`:** Đại diện cho **"Danh bạ Đối Tác"**. Lưu trữ thông tin định danh của các công ty bảo hiểm (Logo, Tên).
- **`INSURANCE_PACKAGE`:** Đại diện cho **"Cuốn Catalog Bảo hiểm"**. Chứa các thông số quyền lợi như hạn mức nội trú (`inpatient_limit`), nha khoa (`dental_care`), và giới hạn

độ tuổi (`min_age`, `max_age`). Dữ liệu này thường được đồng bộ tự động từ API của Đối tác.

- **`INSURANCE_POLICY_RECORD`:** Đại diện cho **"Tờ Hợp đồng / Giấy chứng nhận Điện tử"**. Đây là một văn bản có giá trị pháp lý, xác nhận quyền lợi bảo hiểm của một cá nhân cụ thể.
- **`AMBULANCE_DISPATCH`:** Đại diện cho **"Sổ điều phối Xe cấp cứu"**. Nó lưu lại khoảnh khắc "ngàn cân treo sợi tóc": Ai gọi, tọa độ GPS ở đâu, xe nào đang tới, và dự kiến mấy phút nữa tới (`eta_minutes`).
- **`SYSTEM_AUDIT_LOG`:** Đại diện cho **"Hộp đen Máy bay"** của hệ thống. Nó ghi lại mọi thao tác "động trời" của Admin (Ví dụ: Xuất file báo cáo tài chính, Xóa người dùng, Thay đổi cấu hình hệ thống).

---

3. Mối quan hệ (Relationship) & Quy tắc Nghiệp vụ (Business Rules)

Đây là nơi thể hiện trình độ thiết kế của một System Architect. Có những quy tắc không được phép sai lệch:

1. Khái niệm "Chụp nhanh Dữ liệu" (Data Snapshot): `PATIENT_PROFILE ||--o{ INSURANCE_POLICY_RECORD`

- **Quy tắc nghiệp vụ:** "Hợp đồng pháp lý không được phép thay đổi dù người dùng có đổi tên trên App".
- **Giải thích:** Bạn có thắc mắc tại sao bảng `INSURANCE_POLICY_RECORD` đã có khóa ngoại `purchaser_patient_id` rồi, mà vẫn phải đẻ thêm các cột `beneficiary_full_name`, `beneficiary_dob`, `national_id` không?
    - *Lý do:* Giả sử hôm nay bạn mua bảo hiểm bằng tên "Nguyễn Văn A". Ngày mai bạn vào app WebMed, đổi tên profile thành "A Đẹp Trai". Nếu Hợp đồng bảo hiểm truy xuất tên từ `PATIENT_PROFILE`, nó sẽ in ra "Hợp đồng của A Đẹp Trai" $\rightarrow$ Đối tác bảo hiểm sẽ từ chối đền bù vì sai tên pháp lý. Việc lưu cứng (Snapshot) các trường tên, CMND thẳng vào bảng Policy ngay lúc mua là quy tắc bất di bất dịch của ngành Tài chính!

2. Ràng buộc "Thấy tiền mới xuất Giấy" (1-1): `TRANSACTION_LOG ||--|| INSURANCE_POLICY_RECORD`

- **Quy tắc nghiệp vụ:** "Không có bằng chứng thanh toán, không có hợp đồng".
- **Giải thích:** Mỗi một chứng nhận bảo hiểm bắt buộc phải được móc nối với ĐÚNG 1 mã giao dịch thành công (Transaction ID). Nếu đối tác Bảo hiểm có tranh chấp yêu cầu đối soát, hệ thống chỉ cần dùng khóa ngoại này để chìa ra mã Giao dịch của VNPay/Stripe làm bằng chứng.

3. Khóa Kết nối Ngoại vi (External PK): Bảng `AMBULANCE_DISPATCH`

- **Quy tắc nghiệp vụ:** "Giao tiếp 2 chiều với hệ thống đối tác".

- **Giải thích:** Cột `partner_dispatch_id` không phải do WebMed sinh ra. Khi người dùng bấm nút gọi xe, WebMed bắn API sang hệ thống của Trung tâm Cấp cứu. Trung tâm này sẽ trả về một mã ID (Ví dụ: `AMB-999`). Ta lưu mã này lại để liên tục ping (hỏi) bên đối tác: *"Xe `AMB-999` đi đến đâu rồi? Cập nhật lại cột `eta_minutes` cho tôi"*.

4. Dữ liệu mỏ vàng (Data Mining): ADMIN_ACCOUNT ||--o{ SYSTEM_AUDIT_LOG

- **Quy tắc nghiệp vụ:** "Mọi hành động truy xuất báo cáo đều bị theo dõi".
- **Giải thích:** Khi Ban giám đốc yêu cầu xem UC-36 (View System Reports - Xem báo cáo tổng), hệ thống sẽ phải quét dữ liệu từ Phân hệ 2, 3, 4 để vẽ biểu đồ doanh thu. Nhưng đồng thời, chính cái hành động "Bấm nút Xuất Báo cáo" của Admin cũng lập tức bị ghi vào bảng `SYSTEM_AUDIT_LOG` để đề phòng việc nhân viên lén xuất trộm dữ liệu công ty mang bán.

---

4. Kịch bản thực tế (Use Case) chạy xuyên suốt hệ thống

Hãy xem kịch bản tích hợp API đỉnh cao của phân hệ này:

1. **Khẩn cấp (UC-29):** Một buổi tối, Dũng phát hiện người thân bị ngất. Mở app WebMed, Dũng bấm "Gọi Cấp cứu".
2. **Giao tiếp API:** Ứng dụng lấy tọa độ `gps_latitude` / `longitude` của Dũng, lưu thành 1 dòng vào bảng **`AMBULANCE_DISPATCH`** và đẩy API sang hệ thống Cấp cứu 115.
3. **Theo dõi thời gian thực:** Trạm 115 nhận cuốc, cử tài xế. API trả về mã điều phối và thời gian `eta_minutes` = 12 phút. App của Dũng liên tục đếm ngược thời gian xe đến dựa vào dữ liệu này.
4. **Mua Bảo hiểm (UC-13):** Vài ngày sau, nhận thấy tầm quan trọng của sức khỏe, Dũng vào mục Bảo hiểm. Dũng chọn mua gói "Chăm sóc Toàn diện" của Bảo Việt (**`INSURANCE_PACKAGE`**).
5. **Thanh toán & Chốt Hợp đồng:** Dũng trả 2 triệu VND qua thẻ. Bảng **`TRANSACTION_LOG`** ghi nhận tiền vào. Lập tức, một bản ghi **`INSURANCE_POLICY_RECORD`** được sinh ra, "đóng băng" toàn bộ thông tin ngày sinh, CCCD hiện tại của Dũng và nối với mã giao dịch 2 triệu kia làm bằng chứng pháp lý.
6. **Vận hành (UC-36):** Cuối tháng, Quản lý Vận hành (Admin) vào xuất báo cáo: *"Tháng này có bao nhiêu cuốc xe cấp cứu? Bán được bao nhiêu gói bảo hiểm?"*. Hệ thống vẽ biểu đồ, đồng thời ghi 1 dòng vào **`SYSTEM_AUDIT_LOG`**: *"Admin X vừa xuất file Báo cáo Kinh doanh tháng 10"*.

erDiagram

```
INSURANCE_PROVIDER ||--o{ INSURANCE_PACKAGE : offers

INSURANCE_PACKAGE ||--o{ INSURANCE_POLICY_RECORD : issued_as
```

```
PATIENT_PROFILE ||--o{ INSURANCE_POLICY_RECORD : purchases
TRANSACTION_LOG ||--|| INSURANCE_POLICY_RECORD : pays_for

PATIENT_PROFILE ||--o{ AMBULANCE_DISPATCH : requests

ADMIN_ACCOUNT ||--o{ SYSTEM_AUDIT_LOG : performs_action

INSURANCE_PROVIDER {
   int provider_id PK
   string provider_name
   string provider_logo_url
}

INSURANCE_PACKAGE {
   int package_id PK
   int provider_id FK
   string package_name
   string package_category
   decimal annual_premium
   int min_age
   int max_age
   decimal inpatient_limit
   decimal outpatient_limit
   decimal maternity_allowance
   decimal dental_care
```

```
    string exclusions_text
}

INSURANCE_POLICY_RECORD {
    int policy_record_id PK
    int package_id FK
    int purchaser_patient_id FK
    int transaction_id FK
    string beneficiary_full_name
    date beneficiary_dob
    string beneficiary_national_id
    string beneficiary_gender
    string relationship
    string e_certificate_url
    string status
}

AMBULANCE_DISPATCH {
    int dispatch_id PK
    int patient_id FK
    decimal gps_latitude
    decimal gps_longitude
    string precise_address
    string contact_phone
    string situation_description
    string partner_dispatch_id
```

```
    string driver_name
    int eta_minutes
    string status
    datetime requested_at
}

SYSTEM_AUDIT_LOG {
    int log_id PK
    int admin_id FK
    string module_name
    string action_type
    string action_details_text
    string ip_address
    datetime created_at
}

PATIENT_PROFILE {
    int patient_id PK
}

TRANSACTION_LOG {
    int transaction_id PK
}

ADMIN_ACCOUNT {
    int admin_id PK
```

```
    }

erDiagram

    %% ============================================
    %% SUBSYSTEM 1 - IDENTITY & ACCESS MANAGEMENT
    %% ============================================

    ACCOUNT {
        string email
        string phone_number
        string status
        int failed_login_attempts
    }

    ROLE {
        string role_name
    }

    USER_PROFILE {
        string full_name
        date date_of_birth
    }

    DOCTOR_PROFILE {
        string medical_license_number
```

```
    string approval_status
    string rejection_reason
}

DEPENDENT_PROFILE {
    string full_name
    date date_of_birth
    string relationship
}

DELIVERY_ADDRESS {
    string address_line
    boolean is_default
}

USER_SESSION {
    string session_token
    datetime expires_at
    boolean is_active
}

OTP_TOKEN {
    string purpose
    datetime expires_at
    boolean is_used
}
```

```
AUDIT_LOG {
    string action_type
    string ip_address
    datetime timestamp
}

%% ==============================================
%% SUBSYSTEM 2 - TELEHEALTH & CLINICAL RECORDS
%% ==============================================

SPECIALTY {
    string specialty_name
}

DOCTOR_SCHEDULE {
    date slot_date
    time start_time
    time end_time
    string status
}

APPOINTMENT {
    string consultation_type
    string reason_for_visit
    string status
```

```
    string cancellation_reason
}

MEDICAL_RECORD {
    string patient_symptoms
    string diagnosis
    string medical_advice
    datetime created_at
}

LAB_ATTACHMENT {
    string file_type
    string file_url
}

PRESCRIPTION {
    date issue_date
    date expiry_date
}

MEDICATION {
    string medication_name
    boolean is_active
}

PRESCRIPTION_ITEM {
```

```
    string dosage
    string frequency
    int duration_days
}

FEEDBACK {
    int rating_score
    string review_text
}

%% ==============================================
%% SUBSYSTEM 3 - E-PHARMACY & INVENTORY
%% ==============================================

PRODUCT_MEDICINE {
    string product_name
    decimal unit_price
    boolean is_rx_required
}

INVENTORY {
    int qty_available
    int qty_reserved
}

SHOPPING_CART {
```

```
    datetime last_updated
}

CART_ITEM {
    int quantity
}

PRESCRIPTION_DOCUMENT {
    date issue_date
    boolean is_e_prescription
}

MEDICINE_ORDER {
    decimal subtotal_amount
    decimal shipping_fee
    decimal total_amount
    string status
    string cancellation_reason
}

ORDER_LINE_ITEM {
    int quantity
    decimal unit_price_at_purchase
}

ORDER_TRACKING {
```

```
    string status_milestone
    datetime created_at
}

%% ==============================================
%% SUBSYSTEM 4 - BILLING & FINANCE
%% ==============================================

TRANSACTION_LOG {
    decimal amount
    string payment_method
    string gateway_txn_ref
    string status
    datetime created_at
}

ESCROW_LEDGER {
    decimal amount
    string status
    datetime created_at
}

PAYOUT_REQUEST {
    decimal total_amount
    string receiving_bank_name
    string receiving_account_number
```

```
    string status
    datetime processed_at
}

REFUND_REQUEST {
    decimal refund_amount
    string reason
    string status
    datetime created_at
}

%% ============================================
%% SUBSYSTEM 5 - CMS & HEALTH UTILITIES
%% ============================================

CONTENT_CATEGORY {
    string category_name
}

CMS_ARTICLE {
    string title
    string status
}

DISEASE_DICTIONARY {
    string disease_name
```

```
    string description
    string causes
    string treatment
}

SYMPTOM {
    string symptom_name
}

DISEASE_SYMPTOM {
    float matching_probability
}

CONTENT_VERSION_HISTORY {
    string content_snapshot_json
    datetime modified_at
}

CALCULATOR_HISTORY {
    string calculator_type
    float input_height
    float input_weight
    float result_value
    datetime calculated_at
}
```

```
%% ==============================================
%% RELATIONSHIPS - SUBSYSTEM 1
%% ==============================================

ACCOUNT ||--o{ ROLE : "has (M-N via ACCOUNT_ROLE)"
ACCOUNT ||--|| USER_PROFILE : "owns"
ACCOUNT ||--o| DOCTOR_PROFILE : "extends_as"
ACCOUNT ||--o{ DEPENDENT_PROFILE : "cares_for"
ACCOUNT ||--o{ DELIVERY_ADDRESS : "manages"
ACCOUNT ||--o{ USER_SESSION : "creates"
ACCOUNT ||--o{ OTP_TOKEN : "generates"
ACCOUNT ||--o{ AUDIT_LOG : "recorded_in"

%% ==============================================
%% RELATIONSHIPS - SUBSYSTEM 2
%% ==============================================

DOCTOR_PROFILE ||--o{ SPECIALTY : "specializes_in (M-N via DOCTOR_SPECIALTY)"
DOCTOR_PROFILE ||--o{ DOCTOR_SCHEDULE : "configures"
DOCTOR_PROFILE ||--o{ APPOINTMENT : "assigned_to"
USER_PROFILE ||--o{ APPOINTMENT : "books"
DEPENDENT_PROFILE ||--o{ APPOINTMENT : "booked_for"
DOCTOR_SCHEDULE ||--o| APPOINTMENT : "reserved_by"
APPOINTMENT ||--o| MEDICAL_RECORD : "generates"
APPOINTMENT ||--o| FEEDBACK : "receives"
MEDICAL_RECORD ||--o{ LAB_ATTACHMENT : "contains"
```

```
MEDICAL_RECORD ||--o| PRESCRIPTION : "issues"
PRESCRIPTION ||--o{ PRESCRIPTION_ITEM : "includes"
MEDICATION ||--o{ PRESCRIPTION_ITEM : "prescribed_as"

%% ==============================================
%% RELATIONSHIPS - SUBSYSTEM 3
%% ==============================================

USER_PROFILE ||--|| SHOPPING_CART : "owns"
SHOPPING_CART ||--o{ CART_ITEM : "contains"
PRODUCT_MEDICINE ||--o{ CART_ITEM : "added_as"
PRODUCT_MEDICINE ||--|| INVENTORY : "stocked_in"
USER_PROFILE ||--o{ PRESCRIPTION_DOCUMENT : "uploads"
PRESCRIPTION_DOCUMENT ||--o{ MEDICINE_ORDER : "validates_rx_for"
DOCTOR_PROFILE ||--o{ MEDICINE_ORDER : "verifies_rx"
USER_PROFILE ||--o{ MEDICINE_ORDER : "places"
DELIVERY_ADDRESS ||--o{ MEDICINE_ORDER : "ships_to"
MEDICINE_ORDER ||--o{ ORDER_LINE_ITEM : "includes"
PRODUCT_MEDICINE ||--o{ ORDER_LINE_ITEM : "purchased_as"
MEDICINE_ORDER ||--o{ ORDER_TRACKING : "logs_milestone"

%% ==============================================
%% RELATIONSHIPS - SUBSYSTEM 4
%% ==============================================

USER_PROFILE ||--o{ TRANSACTION_LOG : "initiates"
```

```
APPOINTMENT ||--o{ TRANSACTION_LOG : "triggers"
MEDICINE_ORDER ||--o{ TRANSACTION_LOG : "generates"
TRANSACTION_LOG ||--o| ESCROW_LEDGER : "funds"
APPOINTMENT ||--|| ESCROW_LEDGER : "locked_in"
DOCTOR_PROFILE ||--o{ ESCROW_LEDGER : "earns"
DOCTOR_PROFILE ||--o{ PAYOUT_REQUEST : "receives"
PAYOUT_REQUEST ||--o{ ESCROW_LEDGER : "batches (M-N via ESCROW_PAYOUT_ITEM)"
TRANSACTION_LOG ||--o{ REFUND_REQUEST : "source_of"
ACCOUNT ||--o{ PAYOUT_REQUEST : "processed_by_admin"
ACCOUNT ||--o{ REFUND_REQUEST : "initiated_by_admin"

%% ============================================
%% RELATIONSHIPS - SUBSYSTEM 5
%% ============================================

CONTENT_CATEGORY ||--o{ CMS_ARTICLE : "classifies"
ACCOUNT ||--o{ CMS_ARTICLE : "authors_and_moderates"
ACCOUNT ||--o{ DISEASE_DICTIONARY : "authors"
CMS_ARTICLE ||--o{ CONTENT_VERSION_HISTORY : "tracks_versions"
DISEASE_DICTIONARY ||--o{ CONTENT_VERSION_HISTORY : "tracks_versions"
ACCOUNT ||--o{ CONTENT_VERSION_HISTORY : "modifies"
DISEASE_DICTIONARY ||--o{ DISEASE_SYMPTOM : "exhibits"
SYMPTOM ||--o{ DISEASE_SYMPTOM : "indicates"
USER_PROFILE ||--o{ CALCULATOR_HISTORY : "saves"
ACCOUNT ||--o{ FEEDBACK : "moderated_by_admin"
```

```
%% ==============================================
%% SUBSYSTEM 6 - THIRD-PARTY & SYSTEM OPS
%% ==============================================

INSURANCE_PROVIDER {
   string provider_name
   string provider_logo_url
}

INSURANCE_PACKAGE {
   string package_name
   string package_category
   decimal annual_premium
   int min_age
   int max_age
   decimal inpatient_limit
   decimal outpatient_limit
   decimal dental_care
   string exclusions_text
}

INSURANCE_POLICY_RECORD {
   string beneficiary_full_name
   date beneficiary_dob
   string beneficiary_national_id
   string beneficiary_gender
```

```
    string relationship
    string status
}

AMBULANCE_DISPATCH {
    decimal gps_latitude
    decimal gps_longitude
    string precise_address
    string contact_phone
    string situation_description
    string partner_dispatch_id
    int eta_minutes
    string status
    datetime requested_at
}

SYSTEM_AUDIT_LOG {
    string module_name
    string action_type
    string action_details_text
    string ip_address
    datetime created_at
}
```

Conceptual ERD Mermaid

```
%% ==========================================
```

```
%% RELATIONSHIPS - SUBSYSTEM 6
%% ==========================================

INSURANCE_PROVIDER ||--o{ INSURANCE_PACKAGE : "offers"
INSURANCE_PACKAGE ||--o{ INSURANCE_POLICY_RECORD : "issued_as"
USER_PROFILE ||--o{ INSURANCE_POLICY_RECORD : "purchases (snapshot on buy)"
TRANSACTION_LOG ||--|| INSURANCE_POLICY_RECORD : "pays_for"
USER_PROFILE ||--o{ AMBULANCE_DISPATCH : "requests"
ACCOUNT ||--o{ SYSTEM_AUDIT_LOG : "performs_action"
```

Class diagram: Boundary / Control / Entity

UC 1: register

classDiagram

```
direction TB

class RegisterRoleSelectionUI {
  <<Boundary>>
  +displayRoleOptions()
  +onRoleSelected(role)
}

class RegistrationFormUI {
  <<Boundary>>
  +displayForm(role)
  +displayValidationErrors(errors)
  +onSubmit(formData)
  +togglePasswordVisibility()
```

```
    +openTermsModal()
}

class OTPVerificationUI {
    <<Boundary>>
    +displayOTPInput()
    +onVerifyOTP(otp)
    +onResendOTP()
    +startCountdown(seconds)
    +displayError(message)
}

class RegistrationResultUI {
    <<Boundary>>
    +showSuccessAndRedirect()
    +showPendingNotice()
}

class EmailSMSGateway {
    <<Boundary>>
    +sendOTPEmail(email, otp)
    +sendOTPSMS(phone, otp)
}

class RegistrationController {
    <<Control>>
```

```
    -currentRole String
    +selectRole(role)
    +submitRegistrationForm(data)
    +verifyOTP(otp)
    +resendOTP()
    -routeByRole(role)
}

class AccountValidationService {
    <<Control>>
    +validateAge(dob) bool
    +validateUniqueContact(email, phone) bool
    +validatePasswordComplexity(password) bool
    +validateTermsAccepted(accepted) bool
}

class OTPService {
    <<Control>>
    +generateOTP(accountId) String
    +sendOTP(contact, otp)
    +verifyOTP(accountId, code) bool
    +invalidatePrevious(accountId)
}

class Account {
    <<Entity>>
```

```
    +email String
    +phoneNumber String
    +passwordHash String
    +status String
    +failedLoginAttempts int
    +save()
    +findByEmail(email)
    +findByPhone(phone)
}

class UserProfile {
    <<Entity>>
    +fullName String
    +dateOfBirth Date
    +save()
}

class DoctorProfile {
    <<Entity>>
    +medicalLicenseNumber String
    +credentialsUrl String
    +approvalStatus String
    +save()
}

class Role {
```

```
    <<Entity>>
    +roleName String
    +findByName(name) Role
}

class AccountRole {
    <<Entity>>
    +accountId int
    +roleId int
    +save()
}

class OTPToken {
    <<Entity>>
    +otpCode String
    +purpose String
    +expiresAt DateTime
    +isUsed bool
    +save()
    +markAsUsed()
    +findLatest(accountId) OTPToken
}

RegisterRoleSelectionUI ..> RegistrationController : calls
RegistrationFormUI ..> RegistrationController : calls
OTPVerificationUI ..> RegistrationController : calls
```

```
RegistrationController ..> RegistrationResultUI : directs

RegistrationController ..> AccountValidationService : uses
RegistrationController ..> OTPService : uses
OTPService ..> EmailSMSGateway : sends via

RegistrationController ..> Account : creates
RegistrationController ..> UserProfile : creates
RegistrationController ..> DoctorProfile : creates
RegistrationController ..> AccountRole : assigns

AccountValidationService ..> Account : queries
OTPService ..> OTPToken : manages
```

Sequence dia

```
sequenceDiagram
  actor User
  participant RoleUI as RegisterRoleSelectionUI
  participant FormUI as RegistrationFormUI
  participant OTPUI as OTPVerificationUI
  participant ResultUI as RegistrationResultUI
  participant RegCtrl as RegistrationController
  participant ValSvc as AccountValidationService
  participant OTPSvc as OTPService
  participant Gateway as EmailSMSGateway
  participant Account as Account
```

```
participant UserProfile as UserProfile
participant DoctorProfile as DoctorProfile
participant Role as Role
participant AccountRole as AccountRole
participant OTPToken as OTPToken

User->>RoleUI: click "Register"
RoleUI->>User: displayRoleOptions()
User->>RoleUI: onRoleSelected(role)
RoleUI->>RegCtrl: selectRole(role)
RegCtrl->>FormUI: displayForm(role)

User->>FormUI: onSubmit(formData)
FormUI->>RegCtrl: submitRegistrationForm(formData)

RegCtrl->>ValSvc: validateAge(dob)
alt Age < 18 [BR-01]
   ValSvc-->>RegCtrl: false
   RegCtrl->>FormUI: displayValidationErrors("Must be 18+")
   FormUI->>User: show inline error
end

RegCtrl->>ValSvc: validateUniqueContact(email, phone)
alt Email/Phone already exists [BR-02]
   ValSvc->>Account: findByEmail(email) / findByPhone(phone)
   Account-->>ValSvc: found
```

```
    ValSvc-->>RegCtrl: false
    RegCtrl->>FormUI: displayValidationErrors("Already registered")
    FormUI->>User: show inline error
end

RegCtrl->>ValSvc: validatePasswordComplexity(password)
alt Password too weak [BR-03]
    ValSvc-->>RegCtrl: false
    RegCtrl->>FormUI: displayValidationErrors("Password policy failed")
    FormUI->>User: show inline error
end

RegCtrl->>ValSvc: validateTermsAccepted(accepted)
alt T&C not checked
    ValSvc-->>RegCtrl: false
    RegCtrl->>FormUI: displayValidationErrors("Must accept T&C")
    FormUI->>User: show inline error
end

Note over RegCtrl: All validations passed

RegCtrl->>OTPSvc: generateOTP(tempAccountId)
OTPSvc->>OTPToken: save(otpCode, purpose, expiresAt)
OTPSvc->>Gateway: sendOTPEmail(email, otp) / sendOTPSMS(phone, otp)
Gateway-->>User: deliver OTP
```

```
RegCtrl->>OTPUI: displayOTPInput()
OTPUI->>User: show OTP screen + startCountdown(900s)

alt User resends OTP [Alt 9.a]
   User->>OTPUI: onResendOTP()
   OTPUI->>RegCtrl: resendOTP()
   RegCtrl->>OTPSvc: invalidatePrevious(accountId)
   OTPSvc->>OTPToken: markAsUsed()
   RegCtrl->>OTPSvc: generateOTP(accountId)
   OTPSvc->>OTPToken: save(newOtpCode, ...)
   OTPSvc->>Gateway: sendOTPEmail/SMS(contact, newOtp)
   Gateway-->>User: deliver new OTP
end

User->>OTPUI: onVerifyOTP(otp)
OTPUI->>RegCtrl: verifyOTP(otp)
RegCtrl->>OTPSvc: verifyOTP(accountId, code)
OTPSvc->>OTPToken: findLatest(accountId)

alt Invalid or expired OTP [Exc 10.a]
   OTPSvc-->>RegCtrl: false
   RegCtrl->>OTPUI: displayError("Invalid or expired OTP")
   OTPUI->>User: show error + allow retry
end

OTPSvc->>OTPToken: markAsUsed()
```

```
OTPSvc-->>RegCtrl: true

Note over RegCtrl: Create account entities

RegCtrl->>Account: save(email, phone, passwordHash, status)
RegCtrl->>UserProfile: save(fullName, dateOfBirth)
RegCtrl->>Role: findByName(role)
Role-->>RegCtrl: roleId
RegCtrl->>AccountRole: save(accountId, roleId)

RegCtrl->>RegCtrl: routeByRole(role)

alt Role = Patient/Caregiver
  Note over Account: status = "Active"
  RegCtrl->>ResultUI: showSuccessAndRedirect()
  ResultUI->>User: success message → redirect to Login
else Role = Doctor/Pharmacist [BR-07]
  Note over Account: status = "Pending"
  RegCtrl->>DoctorProfile: save(licenseNumber, credentialsUrl, approvalStatus="Pending")
  RegCtrl->>ResultUI: showPendingNotice()
  ResultUI->>User: "Account pending Admin approval"
end
```

UC2: Login

Class dia

classDiagram

```
direction TB

class LoginFormUI {
    <<Boundary>>
    +displayLoginForm()
    +onSubmit(credentials)
    +displayInlineError(message)
    +displayLockedError(minutesLeft)
    +displayPendingNotice()
    +redirectToDashboard(role)
    +redirectToForgotPassword()
}

class LoginController {
    <<Control>>
    +submitLogin(email, password, rememberMe)
    +handleForgotPassword()
    -determineRedirectByRole(account) String
}

class AuthenticationService {
    <<Control>>
    +authenticate(email, password) AuthResult
    +checkAccountStatus(account) StatusResult
    +incrementFailedAttempts(accountId)
    +resetFailedAttempts(accountId)
```

```
    +isAccountLocked(account) bool
}

class SessionService {
    <<Control>>
    +createSession(accountId, rememberMe) String
    +setSessionExpiry(sessionToken, rememberMe)
}

class AuditService {
    <<Control>>
    +logLoginSuccess(accountId, ip)
    +logLoginFailed(accountId, ip)
}

class Account {
    <<Entity>>
    +email String
    +phoneNumber String
    +passwordHash String
    +status String
    +failedLoginAttempts int
    +lockedUntil DateTime
    +findByEmail(email) Account
    +findByPhone(phone) Account
    +save()
```

```
}

class AccountRole {
    <<Entity>>
    +accountId int
    +roleId int
    +findRolesByAccount(accountId) List~Role~
}

class Role {
    <<Entity>>
    +roleName String
    +findByName(name) Role
}

class UserSession {
    <<Entity>>
    +sessionToken String
    +expiresAt DateTime
    +isActive bool
    +save()
}

class AuditLog {
    <<Entity>>
    +actionType String
```

```
    +ipAddress String
    +timestamp DateTime
    +save()
}

LoginFormUI ..> LoginController : calls
LoginController ..> AuthenticationService : uses
LoginController ..> SessionService : uses
LoginController ..> AuditService : uses
LoginController ..> LoginFormUI : directs

AuthenticationService ..> Account : queries
AuthenticationService ..> AccountRole : queries
AccountRole ..> Role : references
SessionService ..> UserSession : creates
AuditService ..> AuditLog : writes
```

Sequence dia

```
sequenceDiagram
    actor User
    participant LoginUI as LoginFormUI <<Boundary>>
    participant LoginCtrl as LoginController <<Control>>
    participant AuthSvc as AuthenticationService <<Control>>
    participant SessionSvc as SessionService <<Control>>
    participant AuditSvc as AuditService <<Control>>
    participant AccountEnt as Account <<Entity>>
```

```
participant AccountRoleEnt as AccountRole <<Entity>>
participant UserSessionEnt as UserSession <<Entity>>
participant AuditLogEnt as AuditLog <<Entity>>

User->>LoginUI: navigate to Login page
LoginUI-->>User: displayLoginForm()

User->>LoginUI: onSubmit(email, password, rememberMe)
LoginUI->>LoginCtrl: submitLogin(email, password, rememberMe)

LoginCtrl->>AuthSvc: authenticate(email, password)

AuthSvc->>AccountEnt: findByEmail(email)
AccountEnt-->>AuthSvc: account | null

alt Account not found [Exc 5a]
   AuthSvc-->>LoginCtrl: AuthResult(INVALID_CREDENTIALS)
   LoginCtrl->>AuditSvc: logLoginFailed(null, ip)
   AuditSvc->>AuditLogEnt: save(action=LOGIN_FAILED)
   LoginCtrl->>LoginUI: displayInlineError("Incorrect email or password")
   LoginUI-->>User: show inline error
end

AuthSvc->>AuthSvc: isAccountLocked(account)

alt Account locked [Exc 5b - BR-05]
```

```
    AuthSvc-->>LoginCtrl: AuthResult(ACCOUNT_LOCKED, minutesLeft)
    LoginCtrl->>LoginUI: displayLockedError(minutesLeft)
    LoginUI-->>User: "Account temporarily locked for 30 minutes"
end

AuthSvc->>AuthSvc: verifyPasswordHash(password, account.passwordHash)

alt Password mismatch [Exc 5a]
    AuthSvc->>AuthSvc: incrementFailedAttempts(accountId)
    AuthSvc->>AccountEnt: save(failedLoginAttempts++)
    AuthSvc-->>LoginCtrl: AuthResult(INVALID_CREDENTIALS)
    LoginCtrl->>AuditSvc: logLoginFailed(accountId, ip)
    AuditSvc->>AuditLogEnt: save(action=LOGIN_FAILED)
    LoginCtrl->>LoginUI: displayInlineError("Incorrect email or password")
    LoginUI-->>User: show inline error + increment counter
end

Note over AuthSvc: Credentials valid — check account status

AuthSvc->>AuthSvc: checkAccountStatus(account)
AuthSvc->>AccountEnt: save(failedLoginAttempts=0)
AuthSvc-->>LoginCtrl: AuthResult(SUCCESS, account)

LoginCtrl->>AccountRoleEnt: findRolesByAccount(accountId)
AccountRoleEnt-->>LoginCtrl: List~Role~
```

```
LoginCtrl->>LoginCtrl: determineRedirectByRole(account)

alt Role = Doctor/Pharmacist AND status = Pending [Exc 6a - BR-07]
    LoginCtrl->>SessionSvc: createSession(accountId, rememberMe)
    SessionSvc->>UserSessionEnt: save(sessionToken, expiresAt, isActive=true)
    UserSessionEnt-->>SessionSvc: session saved
    SessionSvc-->>LoginCtrl: sessionToken
    LoginCtrl->>AuditSvc: logLoginSuccess(accountId, ip)
    AuditSvc->>AuditLogEnt: save(action=LOGIN_SUCCESS)
    LoginCtrl->>LoginUI: displayPendingNotice()
    LoginUI-->>User: "Account pending Admin approval"
else Role = Patient / approved Doctor / Admin
    LoginCtrl->>SessionSvc: createSession(accountId, rememberMe)
    SessionSvc->>SessionSvc: setSessionExpiry(rememberMe)
    Note over SessionSvc: rememberMe=true → 30 days, false → BR-04 15min inactivity
    SessionSvc->>UserSessionEnt: save(sessionToken, expiresAt, isActive=true)
    UserSessionEnt-->>SessionSvc: session saved
    SessionSvc-->>LoginCtrl: sessionToken
    LoginCtrl->>AuditSvc: logLoginSuccess(accountId, ip)
    AuditSvc->>AuditLogEnt: save(action=LOGIN_SUCCESS)
    LoginCtrl->>LoginUI: redirectToDashboard(role)
    LoginUI-->>User: navigate to Dashboard/Home
end

alt User clicks "Forgot Password?" [Alt 3a]
    User->>LoginUI: click "Forgot Password?"
```

```
    LoginUI->>LoginCtrl: handleForgotPassword()
    LoginCtrl-->>LoginUI: redirect to Password Recovery flow
    LoginUI-->>User: navigate to UC-04
  end
```

UC-03: Logout — BCE Class Diagram + Sequence Diagram

```
classDiagram
  class NavigationBarUI {
    <<Boundary>>
    +renderLogoutButton() void
    +onLogoutClicked() void
    +clearCookies() void
    +clearLocalStorage() void
    +showSessionExpiredNotice() void
    +showLocalLogoutNotice() void
    +redirectTo(page: String) void
  }

  class LoginPageUI {
    <<Boundary>>
    +render() void
  }

  class LogoutController {
    <<Control>>
    +handleLogoutRequest(sessionToken: String) void
```

```
    +checkSessionValidity(sessionToken: String) Boolean
    +invalidateServerSession(sessionToken: String) Boolean
    +clearLocalSession() void
    +handleAutoTimeout() void
}

class UserSession {
    <<Entity>>
    -session_id: int
    -account_id: int
    -session_token: String
    -expires_at: DateTime
    -is_active: Boolean
    +isExpired() Boolean
    +isActive() Boolean
    +invalidateToken() void
    +terminateSession() void
}

class Account {
    <<Entity>>
    -account_id: int
    -email: String
    -phone_number: String
    -status: String
}
```

```
    NavigationBarUI --> LogoutController : sendLogoutRequest
    LogoutController --> UserSession : manages session
    LogoutController --> LoginPageUI : redirects to
    UserSession --> Account : belongs to


sequenceDiagram
    actor User
    participant NavUI as NavigationBarUI
    participant LogoutCtrl as LogoutController
    participant SessionEnt as UserSession
    participant LoginUI as LoginPageUI

    alt Normal Flow - User clicks Logout
        User ->> NavUI: clicks "Logout"
        NavUI ->> LogoutCtrl: handleLogoutRequest(sessionToken)
    else Alt 1a - Auto-Timeout BR-04 - 15 min inactivity
        Note over LogoutCtrl: Inactivity timer reaches 15 minutes
        LogoutCtrl ->> LogoutCtrl: handleAutoTimeout()
    end

    LogoutCtrl ->> SessionEnt: checkSessionValidity(sessionToken)

    alt Exception 1a - Session already expired
        SessionEnt -->> LogoutCtrl: result: isExpired = true
```

```
    LogoutCtrl -->> NavUI: notifySessionAlreadyExpired()
    NavUI -->> User: "Your session has already expired.<br/>Please log in again."
    NavUI ->> LoginUI: redirectTo()
    LoginUI -->> User: Login page rendered

  else Normal - Session is still active
    SessionEnt -->> LogoutCtrl: result: isActive = true
    LogoutCtrl ->> SessionEnt: invalidateToken()

    alt Exception 2a - Server connection error
      SessionEnt -->> LogoutCtrl: error: serverUnreachable
      LogoutCtrl -->> NavUI: forceLocalLogout()
      NavUI ->> NavUI: clearCookies()
      NavUI ->> NavUI: clearLocalStorage()
      NavUI -->> User: "You have been securely logged out locally.<br/>Please close your
browser."

    else Normal - Server reachable
      SessionEnt -->> LogoutCtrl: success: tokenInvalidated
      LogoutCtrl ->> SessionEnt: terminateSession()
      SessionEnt -->> LogoutCtrl: success: sessionTerminated
      LogoutCtrl -->> NavUI: clearLocalData()
      NavUI ->> NavUI: clearCookies()
      NavUI ->> NavUI: clearLocalStorage()
      LogoutCtrl -->> NavUI: redirectToLogin()
      NavUI ->> LoginUI: redirectTo()
```

```
        LoginUI -->> User: Login page rendered
    end
end
```

UC-04: Recover Password — Class Diagram + Sequence Diagram

```
classDiagram
  direction TB

  class ForgotPasswordPageUI {
    <<Boundary>>
    +renderContactInputForm() void
    +onSubmitContact(contact: String) void
    +showGenericNotice() void
  }

  class OtpVerificationPageUI {
    <<Boundary>>
    +renderOtpForm() void
    +onSubmitOtp(otp: String) void
    +onResendOtpClicked() void
    +showOtpError(msg: String) void
    +showResendConfirmation() void
  }

  class NewPasswordPageUI {
    <<Boundary>>
```

```
    +renderNewPasswordForm() void
    +onSubmitNewPassword(newPwd: String,\nconfirmPwd: String) void
    +showPolicyErrors(errors: List) void
    +showSuccessAndRedirect() void
}

class EmailSmsGateway {
    <<Boundary>>
    +sendOtp(contact: String,\notpCode: String) Boolean
    +sendResetLink(contact: String,\nresetToken: String) Boolean
}

class PasswordRecoveryController {
    <<Control>>
    +initRecovery(contact: String) void
    +generateOtp(accountId: int) OtpToken
    +generateResetLink(accountId: int) String
    +resendOtp(accountId: int) void
    +verifyOtp(otpCode: String,\naccountId: int) Boolean
    +validatePasswordPolicy(pwd: String) Boolean
    +updatePassword(accountId: int,\nnewPwd: String) void
    +logRecoveryEvent(accountId: int) void
}

class Account {
    <<Entity>>
```

```
    -account_id: int
    -email: String
    -phone_number: String
    -password_hash: String
    -status: String
    +findByContact(contact: String) Account
    +updatePasswordHash(newHash: String) void
}

class OtpToken {
    <<Entity>>
    -token_id: int
    -account_id: int
    -otp_code: String
    -purpose: String
    -expires_at: DateTime
    -is_used: Boolean
    +isValid() Boolean
    +isExpired() Boolean
    +markAsUsed() void
    +invalidate() void
}

class AuditLog {
    <<Entity>>
    -log_id: int
```

```
    -actor_account_id: int
    -action_type: String
    -timestamp: DateTime
    -ip_address: String
    +recordEvent(actorId: int,\naction: String) void
  }

  ForgotPasswordPageUI --> PasswordRecoveryController : initiates recovery
  OtpVerificationPageUI --> PasswordRecoveryController : verifies OTP / resends
  NewPasswordPageUI --> PasswordRecoveryController : submits new password
  PasswordRecoveryController --> Account : queries & updates
  PasswordRecoveryController --> OtpToken : creates & validates
  PasswordRecoveryController --> AuditLog : records recovery event
  PasswordRecoveryController --> EmailSmsGateway : sends OTP or reset link

sequenceDiagram
  actor User
  participant ForgotUI as ForgotPasswordPageUI
  participant OtpUI as OtpVerificationPageUI
  participant NewPwdUI as NewPasswordPageUI
  participant RecoveryCtrl as PasswordRecoveryController
  participant AccountEnt as Account
  participant OtpEnt as OtpToken
  participant AuditEnt as AuditLog
  participant Gateway as EmailSmsGateway
```

```
User ->> +ForgotUI: clicks "Forgot Password?"
ForgotUI -->> -User: renders registered contact input form

User ->> +ForgotUI: enters email/phone & submits
ForgotUI ->> +RecoveryCtrl: initRecovery(contact)
RecoveryCtrl ->> +AccountEnt: findByContact(contact)

alt Exception 4a — Account Not Found
  AccountEnt -->> -RecoveryCtrl: result: null (not found)
  RecoveryCtrl -->> -ForgotUI: triggerGenericNotice()
  ForgotUI -->> -User: "If this contact is registered,\nyou will receive a recovery code shortly."

else Normal — Account exists & is active
  AccountEnt -->> -RecoveryCtrl: account object (accountId)
  RecoveryCtrl ->> +OtpEnt: create(accountId, "PASSWORD_RECOVERY") (BR-05: valid 15min)
  OtpEnt -->> -RecoveryCtrl: otpToken (code, expiresAt)

  alt Alt Flow 5a — Send Reset Link via Email
    RecoveryCtrl ->> RecoveryCtrl: generateResetLink(accountId)
    RecoveryCtrl ->> +Gateway: sendResetLink(contact, resetToken)
    Gateway -->> -RecoveryCtrl: sent: true
  else Normal — Send OTP via Email/SMS
    RecoveryCtrl ->> +Gateway: sendOtp(contact, otpCode)
    Gateway -->> -RecoveryCtrl: sent: true
  end
```

```
RecoveryCtrl -->> -ForgotUI: otpDispatched: true
ForgotUI -->> -User: renders OTP verification screen

User ->> +OtpUI: enters received OTP & submits

alt Alt Flow 7a — User clicks "Resend OTP"
   OtpUI ->> +RecoveryCtrl: resendOtp(accountId)
   RecoveryCtrl ->> +OtpEnt: invalidate(oldTokenId)
   OtpEnt -->> -RecoveryCtrl: old OTP invalidated
   RecoveryCtrl ->> +OtpEnt: create(accountId, "PASSWORD_RECOVERY")
   OtpEnt -->> -RecoveryCtrl: new otpToken generated
   RecoveryCtrl ->> +Gateway: sendOtp(contact, newOtpCode)
   Gateway -->> -RecoveryCtrl: sent: true
   RecoveryCtrl -->> -OtpUI: otpResent: true
   OtpUI -->> User: "A new OTP has been sent to your contact."
end

OtpUI ->> +RecoveryCtrl: verifyOtp(otpCode, accountId)
RecoveryCtrl ->> +OtpEnt: validate() — check code & expiry (BR-05)

alt Exception 8a — Invalid or Expired OTP
   OtpEnt -->> -RecoveryCtrl: result: invalid or expired
   RecoveryCtrl -->> -OtpUI: otpVerificationFailed
   OtpUI -->> -User: "Invalid or expired code.\nTry again or request a new one."

else Normal — OTP is valid
```

```
        OtpEnt -->> -RecoveryCtrl: result: valid
        RecoveryCtrl ->> +OtpEnt: markAsUsed()
        OtpEnt -->> -RecoveryCtrl: marked as used
        RecoveryCtrl -->> -OtpUI: otpVerified: true
        OtpUI -->> -User: renders new password form

        User ->> +NewPwdUI: enters & confirms new password
        NewPwdUI ->> +RecoveryCtrl: submitNewPassword(accountId,\nnewPwd, confirmPwd)
        RecoveryCtrl ->> RecoveryCtrl: validatePasswordPolicy(pwd) (BR-03)

        alt Exception 10a — Password Policy Violation (BR-03)
          RecoveryCtrl -->> -NewPwdUI: policyErrors (specific messages)
          NewPwdUI -->> -User: shows inline validation\nerror messages

        else Normal — Password passes all policy checks
          RecoveryCtrl ->> +AccountEnt: updatePasswordHash(accountId, newHash)
          AccountEnt -->> -RecoveryCtrl: updated: true
          RecoveryCtrl ->> +AuditEnt: recordEvent(accountId,\n"PASSWORD_RECOVERY",
timestamp)
          AuditEnt -->> -RecoveryCtrl: logged
          RecoveryCtrl -->> -NewPwdUI: success: true
          NewPwdUI -->> -User: "Password updated successfully!\nRedirecting to Login page..."
        end
      end
    end
```

UC-05: Change Password — Class Diagram + Sequence Diagram

```
classDiagram
  direction TB

  class AccountSettingsPageUI {
    <<Boundary>>
    +renderChangePasswordForm() void
    +onSubmitForm(currentPwd: String,\nnewPwd: String, confirmPwd: String) void
    +showIncorrectPasswordError() void
    +showPasswordMismatchError() void
    +showPolicyViolationErrors(errors: List) void
    +showAccountLockedNotice() void
    +showSuccess() void
  }

  class ChangePasswordController {
    <<Control>>
    +changePassword(accountId: int,\ncurrentPwd: String, newPwd: String,\nconfirmPwd:
String) void
    +checkPasswordsMatch(newPwd: String,\nconfirmPwd: String) Boolean
    +validatePolicy(pwd: String) Boolean
    -hashPassword(pwd: String) String
  }

  class Account {
    <<Entity>>
```

```
    -account_id: int
    -password_hash: String
    -failed_login_attempts: int
    -locked_until: DateTime
    +verifyCurrentPassword(accountId: int,\ncurrentPwd: String) Boolean
    +incrementFailedAttempts(accountId: int) int
    +lockAccount(accountId: int,\ndurationMinutes: int) void
    +updatePasswordHash(accountId: int,\nnewHash: String) void
}

class UserSession {
    <<Entity>>
    -session_id: int
    -account_id: int
    -session_token: String
    -is_active: Boolean
    +invalidateAllSessions(accountId: int) void
    +invalidateOtherSessions(accountId: int,\ncurrentSessionId: int) void
}

class AuditLog {
    <<Entity>>
    -log_id: int
    -actor_account_id: int
    -action_type: String
    -timestamp: DateTime
```

```
    -ip_address: String
    +recordEvent(actorId: int,\naction: String) void
  }

  AccountSettingsPageUI --> ChangePasswordController : submits change request
  ChangePasswordController --> Account : verifies & updates password
  ChangePasswordController --> UserSession : invalidates sessions
  ChangePasswordController --> AuditLog : records security event

sequenceDiagram
  actor User
  participant ProfileUI as UserProfilePageUI
  participant AddrUI as AddressManagementUI
  participant DeleteUI as DeleteAccountUI
  participant ProfileCtrl as UserProfileController
  participant AddrCtrl as AddressController
  participant DelCtrl as AccountDeletionController
  participant ProfileEnt as UserProfile
  participant AccountEnt as Account
  participant AddrEnt as DeliveryAddress
  participant OtpEnt as OtpToken
  participant AuditEnt as AuditLog
  participant Gateway as EmailSmsGateway

  %% ── Common: Load Profile ──
  User ->>+ ProfileUI: navigates to "My Profile"
```

```
ProfileUI ->>+ ProfileCtrl: loadProfile(accountId)
ProfileCtrl ->>+ ProfileEnt: getByAccountId(accountId)
ProfileEnt -->>- ProfileCtrl: profileData
ProfileCtrl -->>- ProfileUI: profileData
ProfileUI -->>- User: renders current profile data

%% ═══════════════════════════════════════════════════════
%% BRANCH 1 — Normal Flow: Edit & Save
%% ═══════════════════════════════════════════════════════
alt Normal Flow — Update Profile Fields
  User ->>+ ProfileUI: modifies fields,\nclicks "Save Changes"
  ProfileUI ->>+ ProfileCtrl: updateProfile(accountId, updatedData)

  ProfileCtrl ->>+ AccountEnt: checkContactUniqueness(contact, accountId)
  AccountEnt -->>- ProfileCtrl: isUnique: boolean

  alt Exception 5a — Contact already in use
    ProfileCtrl -->> ProfileUI: contactInUseError()
    ProfileUI -->> User: "This contact is already\nassociated with another account."
  else Contact is unique — proceed to update
    ProfileCtrl ->>+ ProfileEnt: update(accountId, updatedData)
    ProfileEnt -->>- ProfileCtrl: updated: true
    ProfileCtrl ->>+ AuditEnt: recordEvent(accountId,\n"SENSITIVE_PROFILE_UPDATED")
    AuditEnt -->>- ProfileCtrl: logged
    ProfileCtrl -->> ProfileUI: success()
    ProfileUI -->> User: "Profile updated successfully."
```

```
    end
    deactivate ProfileCtrl
    deactivate ProfileUI

%% ═══════════════════════════════════════════════════════════
%% BRANCH 2 — Alt 3a: Manage Addresses
%% ═══════════════════════════════════════════════════════════
else Alt 3a — Manage Delivery Addresses
    User ->>+ AddrUI: selects "Manage Addresses" tab
    AddrUI ->>+ AddrCtrl: getAddresses(accountId)
    AddrCtrl ->>+ AddrEnt: getAllByAccount(accountId)
    AddrEnt -->>- AddrCtrl: addressList
    AddrCtrl -->>- AddrUI: addressList
    AddrUI -->>- User: renders address list

    alt Add new address
        User ->>+ AddrUI: fills & submits new address
        AddrUI ->>+ AddrCtrl: addAddress(accountId, addressData)
        AddrCtrl ->>+ AddrEnt: create(accountId, addressData)
        AddrEnt -->>- AddrCtrl: created: true
        AddrCtrl -->> AddrUI: addressAdded()
        AddrUI -->> User: address list refreshed
    else Set as Default address
        User ->>+ AddrUI: clicks "Set as Default"
        AddrUI ->>+ AddrCtrl: setDefaultAddress(addressId, accountId)
        AddrCtrl ->>+ AddrEnt: setDefault(addressId, accountId)
```

```
        AddrEnt -->>- AddrCtrl: updated: true
        AddrCtrl -->> AddrUI: defaultUpdated()
        AddrUI -->> User: address list refreshed
    else Delete an address
        User ->>+ AddrUI: clicks "Delete" on address
        AddrUI ->>+ AddrCtrl: deleteAddress(addressId)
        AddrCtrl ->>+ AddrEnt: delete(addressId)
        AddrEnt -->>- AddrCtrl: deleted: true
        AddrCtrl -->> AddrUI: addressRemoved()
        AddrUI -->> User: address list refreshed
    end
    deactivate AddrCtrl
    deactivate AddrUI

%% ═══════════════════════════════════════════════════════════
%% BRANCH 3 — Alt 3b: Delete Account
%% ═══════════════════════════════════════════════════════════
else Alt 3b — Request Account Deletion
    User ->>+ DeleteUI: selects "Delete Account"
    DeleteUI -->>- User: renders critical data-loss warning\n& current password prompt

    User ->>+ DeleteUI: enters password, clicks "Proceed"
    DeleteUI ->>+ DelCtrl: initDeletion(accountId, password)

    DelCtrl ->>+ AccountEnt: verifyPassword(accountId, password)
    AccountEnt -->>- DelCtrl: isValid: boolean
```

```
alt Exception 3a — Incorrect password (BR-06)
    DelCtrl ->>+ AccountEnt: incrementFailedAttempts(accountId)
    AccountEnt -->>- DelCtrl: failedAttempts: int

    alt failedAttempts >= 5 — Lock profile settings
        DelCtrl -->> DeleteUI: lockAndTerminate()
        DeleteUI -->> User: "Too many failed attempts..."
    else failedAttempts < 5
        DelCtrl -->> DeleteUI: incorrectPasswordError()
        DeleteUI -->> User: "Incorrect password. Please try again."
    end
else Password valid — generate OTP (BR-06)
    DelCtrl ->>+ OtpEnt: create(accountId, "ACCOUNT_DELETION")
    OtpEnt -->>- DelCtrl: otpToken
    DelCtrl ->>+ Gateway: sendOtp(contact, otpCode)
    Gateway -->>- DelCtrl: sent: true
    DelCtrl -->> DeleteUI: otpDispatched()
    DeleteUI -->> User: renders OTP verification prompt
end
deactivate DelCtrl
deactivate DeleteUI

%% Luồng tiếp nối: Tách riêng việc User nhập OTP ra một block opt để không bị dính chùm activation
opt User submits received OTP
```

```
        User ->>+ DeleteUI: enters received OTP
        DeleteUI ->>+ DelCtrl: verifyOtpAndDelete(accountId, otpCode)

        DelCtrl ->>+ OtpEnt: verify(accountId, otpCode)
        OtpEnt -->>- DelCtrl: isValid: boolean

        alt Exception 3a — Invalid or Expired OTP
           DelCtrl -->> DeleteUI: otpInvalidError()
           DeleteUI -->> User: "Invalid or expired code.\nTry again or request a new one."
        else OTP valid — execute account deletion
           DelCtrl ->>+ OtpEnt: markAsUsed()
           OtpEnt -->>- DelCtrl: marked
           DelCtrl ->>+ AccountEnt: markAsDeleted(accountId)
           AccountEnt -->>- DelCtrl: status: "Deleted"
           DelCtrl ->>+ AuditEnt: recordEvent(accountId,\n"ACCOUNT_DELETION_EXECUTED")
           AuditEnt -->>- DelCtrl: logged
           DelCtrl -->> DeleteUI: deletionComplete()
           DeleteUI -->> User: "Account deleted.\nYou will now be logged out."
           Note over DeleteUI: Triggers UC-03 (Logout) to\nclear session & cookies
        end
        deactivate DelCtrl
        deactivate DeleteUI
     end
  end
```

UC-06: Manage User Profile — Class Diagram + Sequence Diagram

```
classDiagram
    direction TB

    class UserProfilePageUI {
        <<Boundary>>
        +renderProfileData(profile: Object) void
        +onSaveChanges(updatedData: Object) void
        +showContactInUseError() void
        +showUpdateSuccess() void
    }

    class AddressManagementUI {
        <<Boundary>>
        +renderAddressList(addresses: List) void
        +onAddAddress(data: Object) void
        +onEditAddress(addressId: int, data: Object) void
        +onDeleteAddress(addressId: int) void
        +onSetDefault(addressId: int) void
    }

    class DeleteAccountUI {
        <<Boundary>>
        +renderDeletionWarning() void
        +renderPasswordPrompt() void
        +renderOtpPrompt() void
        +showVerificationError(msg: String) void
```

```
    +showProfileLockNotice() void
    +showDeletionSuccess() void
}

class EmailSmsGateway {
    <<Boundary>>
    +sendOtp(contact: String,\notpCode: String) Boolean
}

class UserProfileController {
    <<Control>>
    +loadProfile(accountId: int) Object
    +updateProfile(accountId: int,\ndata: Object) void
    +checkContactUniqueness(contact: String,\naccountId: int) Boolean
}

class AddressController {
    <<Control>>
    +getAddresses(accountId: int) List
    +addAddress(accountId: int, data: Object) void
    +editAddress(addressId: int, data: Object) void
    +deleteAddress(addressId: int) void
    +setDefaultAddress(addressId: int,\naccountId: int) void
}

class AccountDeletionController {
```

```
    <<Control>>
    +initDeletion(accountId: int,\npassword: String) void
    +verifyOtpAndDelete(accountId: int,\notpCode: String) void
    +lockProfileSettings(accountId: int) void
}

class UserProfile {
    <<Entity>>
    -profile_id: int
    -account_id: int
    -full_name: String
    -date_of_birth: Date
    +getByAccountId(accountId: int) Object
    +update(accountId: int, data: Object) void
}

class Account {
    <<Entity>>
    -account_id: int
    -email: String
    -phone_number: String
    -status: String
    -failed_login_attempts: int
    +checkContactUniqueness(contact: String,\naccountId: int) Boolean
    +verifyPassword(accountId: int,\npassword: String) Boolean
    +incrementFailedAttempts(accountId: int) int
```

```
    +markAsDeleted(accountId: int) void
}

class DeliveryAddress {
    <<Entity>>
    -address_id: int
    -account_id: int
    -address_line: String
    -is_default: Boolean
    +getAllByAccount(accountId: int) List
    +create(accountId: int, data: Object) void
    +update(addressId: int, data: Object) void
    +delete(addressId: int) void
    +setDefault(addressId: int, accountId: int) void
}

class OtpToken {
    <<Entity>>
    -token_id: int
    -account_id: int
    -otp_code: String
    -purpose: String
    -expires_at: DateTime
    -is_used: Boolean
    +create(accountId: int, purpose: String) OtpToken
    +verify(accountId: int, otpCode: String) Boolean
```

```
    +markAsUsed() void
  }

  class AuditLog {
    <<Entity>>
    -log_id: int
    -actor_account_id: int
    -action_type: String
    -timestamp: DateTime
    -ip_address: String
    +recordEvent(actorId: int, action: String) void
  }

  UserProfilePageUI --> UserProfileController : updates profile
  AddressManagementUI --> AddressController : manages addresses
  DeleteAccountUI --> AccountDeletionController : requests deletion
  UserProfileController --> UserProfile : reads & writes
  UserProfileController --> Account : checks contact uniqueness
  UserProfileController --> AuditLog : logs sensitive changes
  AddressController --> DeliveryAddress : manages addresses
  AccountDeletionController --> Account : verifies & marks deleted
  AccountDeletionController --> OtpToken : generates & verifies
  AccountDeletionController --> EmailSmsGateway : sends deletion OTP
  AccountDeletionController --> AuditLog : logs deletion request
```

Bị mất sequence

UC-32: Manage Users & Approvals — Class Diagram + Sequence Diagram (Version 1)

```
classDiagram
  direction TB

  class UserManagementDashboardUI {
    <<Boundary>>
    +renderDashboard(userList: List) void
    +onApplyRoleFilter(role: String) void
    +onApplyStatusFilter(status: String) void
    +onSearch(keyword: String) void
    +onViewDetails(accountId: int) void
    +showSuccessMessage(msg: String) void
  }

  class DoctorVerificationUI {
    <<Boundary>>
    +renderDoctorProfile(data: Object) void
    +renderCredentialDocViewer(url: String) void
    +disableApproveButton() void
    +showCorruptedDocError() void
    +onApproveClicked(accountId: int) void
    +onRejectClicked(accountId: int) void
  }
```

```
class RejectionModalUI {
    <<Boundary>>
    +renderReasonInput() void
    +onConfirmRejection(accountId: int,\nreason: String) void
}

class UserDetailUI {
    <<Boundary>>
    +renderUserDetail(data: Object) void
    +onSuspendAccount(accountId: int) void
}

class EmailGateway {
    <<Boundary>>
    +sendApprovalNotification(email: String) void
    +sendRejectionNotification(email: String,\nreason: String) void
}

class UserManagementController {
    <<Control>>
    +loadUserList(role: String,\nstatus: String, keyword: String) List
    +loadUserDetail(accountId: int) Object
}

class DoctorApprovalController {
    <<Control>>
```

```
    +loadDoctorProfile(accountId: int) Object
    +checkCredentialDocuments(accountId: int) Boolean
    +approveDoctor(adminId: int, accountId: int) void
    +rejectDoctor(adminId: int,\naccountId: int, reason: String) void
}

class AccountActionController {
    <<Control>>
    +suspendAccount(adminId: int,\ntargetAccountId: int) void
}

class Account {
    <<Entity>>
    -account_id: int
    -email: String
    -status: String
    +findAll(role: String,\nstatus: String, keyword: String) List
    +findById(accountId: int) Object
    +updateStatus(accountId: int,\nnewStatus: String) void
}

class DoctorProfile {
    <<Entity>>
    -doctor_profile_id: int
    -account_id: int
    -medical_license_number: String
```

```
    -credentials_url: String
    -rejection_reason: String
    +getByAccountId(accountId: int) Object
    +hasValidDocuments(accountId: int) Boolean
    +saveRejectionReason(accountId: int,\nreason: String) void
}

class AuditLog {
    <<Entity>>
    -log_id: int
    -actor_account_id: int
    -target_account_id: int
    -action_type: String
    -timestamp: DateTime
    -ip_address: String
    +recordEvent(adminId: int,\ntargetId: int, action: String) void
}

UserManagementDashboardUI --> UserManagementController : load & filter users
UserManagementDashboardUI --> DoctorVerificationUI : opens doctor detail
UserManagementDashboardUI --> UserDetailUI : opens user detail
DoctorVerificationUI --> DoctorApprovalController : approve / reject
RejectionModalUI --> DoctorApprovalController : confirm rejection
UserDetailUI --> AccountActionController : suspend account
UserManagementController --> Account : queries user list
DoctorApprovalController --> Account : updates doctor status
```

```
    DoctorApprovalController --> DoctorProfile : reads & updates profile
    DoctorApprovalController --> AuditLog : records approval action
    DoctorApprovalController --> EmailGateway : sends notification email
    AccountActionController --> Account : updates user status
    AccountActionController --> AuditLog : records suspension action

sequenceDiagram
    actor Admin
    participant DashboardUI as UserManagementDashboardUI
    participant DocVerifyUI as DoctorVerificationUI
    participant RejectModalUI as RejectionModalUI
    participant UserDetailUI as UserDetailUI
    participant UserMgmtCtrl as UserManagementController
    participant DocApprCtrl as DoctorApprovalController
    participant AccActionCtrl as AccountActionController
    participant AccountEnt as Account
    participant DocProfileEnt as DoctorProfile
    participant AuditEnt as AuditLog
    participant EmailGW as EmailGateway

    %% ── Load Dashboard ──
    Admin ->>+ DashboardUI: clicks "User Management"
    DashboardUI ->>+ UserMgmtCtrl: loadUserList(role="All", status="All", keyword="")
    UserMgmtCtrl ->>+ AccountEnt: findAll(role, status, keyword)
    AccountEnt -->>- UserMgmtCtrl: userList
    UserMgmtCtrl -->>- DashboardUI: userList
```

```
    DashboardUI -->>- Admin: renders User Management Dashboard

    %% ── Apply filter: Doctors / Pending Verification ──
    Admin ->>+ DashboardUI: selects role="Doctors", status="Pending Verification"
    DashboardUI ->>+ UserMgmtCtrl: loadUserList(role="Doctor", status="Pending", keyword="")
    UserMgmtCtrl ->>+ AccountEnt: findAll(role, status, keyword)
    AccountEnt -->>- UserMgmtCtrl: filteredList
    UserMgmtCtrl -->>- DashboardUI: filteredList
    DashboardUI -->>- Admin: updates table with pending doctors

    Admin ->>+ DashboardUI: clicks "View Details" (accountId)
    DashboardUI ->>+ DocApprCtrl: loadDoctorProfile(accountId)
    DocApprCtrl ->>+ DocProfileEnt: getByAccountId(accountId)
    DocProfileEnt -->>- DocApprCtrl: doctorProfileData

    %% Rule 4: Check documents BEFORE alt block
    DocApprCtrl ->>+ DocProfileEnt: hasValidDocuments(accountId)
    DocProfileEnt -->>- DocApprCtrl: docsValid: boolean

    alt Exception 7a — Documents missing or corrupted
        DocApprCtrl -->> DashboardUI: disableApproveAndAlert()
        DashboardUI -->> Admin: "Cannot verify credentials.\nDocuments are missing or
corrupted."
    else Documents valid — render verification screen
        DocApprCtrl -->> DashboardUI: doctorProfileData
        DashboardUI ->>+ DocVerifyUI: renderDoctorProfile(data)
```

```
    DocVerifyUI -->>- DashboardUI: rendered
    DashboardUI -->> Admin: renders Doctor Verification screen
end
%% Đặt deactivate ở ĐÂY (ngoài khối alt) để Mermaid không đếm trừ 2 lần
deactivate DocApprCtrl
deactivate DashboardUI

%% Tách phần Approve/Reject thành luồng opt riêng cho sạch sẽ
opt Admin reviews and makes a decision (If Valid)
    Admin ->>+ DocVerifyUI: clicks Approve or Reject

    alt Normal Flow — Approve Doctor Profile
        DocVerifyUI ->>+ DocApprCtrl: approveDoctor(adminId, accountId)
        DocApprCtrl ->>+ AccountEnt: updateStatus(accountId, "Active")
        AccountEnt -->>- DocApprCtrl: updated: true
        DocApprCtrl ->>+ AuditEnt: recordEvent(adminId, accountId, "DOCTOR_APPROVED")
        AuditEnt -->>- DocApprCtrl: logged
        DocApprCtrl ->>+ EmailGW: sendApprovalNotification(email)
        EmailGW -->>- DocApprCtrl: sent: true
        DocApprCtrl -->> DocVerifyUI: approvalComplete()

        %% Gọi trong nhánh Normal thì tắt trong nhánh Normal
        deactivate DocApprCtrl
        DocVerifyUI -->> Admin: "Doctor profile approved successfully."

    else Alt 8a — Reject Doctor Profile
```

```
        DocVerifyUI ->>+ RejectModalUI: renderReasonInput()
        RejectModalUI -->>- DocVerifyUI: modal rendered
        DocVerifyUI -->> Admin: renders Rejection modal (mandatory reason text area)

        Admin ->>+ RejectModalUI: inputs reason, clicks "Confirm Rejection"
        RejectModalUI ->>+ DocApprCtrl: rejectDoctor(adminId, accountId, reason)
        DocApprCtrl ->>+ DocProfileEnt: saveRejectionReason(accountId, reason)
        DocProfileEnt -->>- DocApprCtrl: saved
        DocApprCtrl ->>+ AccountEnt: updateStatus(accountId, "Rejected")
        AccountEnt -->>- DocApprCtrl: updated: true
        DocApprCtrl ->>+ AuditEnt: recordEvent(adminId, accountId, "DOCTOR_REJECTED")
        AuditEnt -->>- DocApprCtrl: logged
        DocApprCtrl ->>+ EmailGW: sendRejectionNotification(email, reason)
        EmailGW -->>- DocApprCtrl: sent: true
        DocApprCtrl -->> RejectModalUI: rejectionComplete()

        %% Gọi trong nhánh Reject thì tắt trong nhánh Reject
        deactivate DocApprCtrl
        RejectModalUI -->> Admin: "Doctor profile rejected.\nEmail notification sent."
        deactivate RejectModalUI
    end
    deactivate DocVerifyUI
end

%% ═══════════════════════════════════════════════════════════
%% Alt 3a — Suspend an Active User
```

```
%% ═══════════════════════════════════════════════════════════
opt Alt 3a — Suspend Active User Account
    Admin ->>+ DashboardUI: filters role="Patients", searches for target user
    DashboardUI ->>+ UserMgmtCtrl: loadUserList(role, status, keyword)
    UserMgmtCtrl ->>+ AccountEnt: findAll(role, status, keyword)
    AccountEnt -->>- UserMgmtCtrl: filteredList
    UserMgmtCtrl -->>- DashboardUI: filteredList
    DashboardUI -->>- Admin: updates table

    Admin ->>+ DashboardUI: clicks "View Details" (accountId)
    DashboardUI ->>+ UserDetailUI: renderUserDetail(accountId)
    UserDetailUI -->>- DashboardUI: rendered
    DashboardUI -->>- Admin: renders User Detail screen\nwith Account Actions dropdown

    Admin ->>+ UserDetailUI: selects "Suspend Account", confirms prompt
    UserDetailUI ->>+ AccActionCtrl: suspendAccount(adminId, targetAccountId)
    AccActionCtrl ->>+ AccountEnt: updateStatus(targetAccountId, "Suspended")
    AccountEnt -->>- AccActionCtrl: updated: true
    AccActionCtrl ->>+ AuditEnt: recordEvent(adminId, targetAccountId,
"ACCOUNT_SUSPENDED")
    AuditEnt -->>- AccActionCtrl: logged
    AccActionCtrl -->> UserDetailUI: suspensionComplete()
    deactivate AccActionCtrl
    UserDetailUI -->>- Admin: "User account has been\nsuccessfully suspended."
End
```

UC14: Find/Search Doctor

```
classDiagram
  direction TB

  class DoctorSearchDashboardUI {
    <<Boundary>>
    +displaySearchInput()
    +displayFilters(specialties)
    +onSearch(keyword, filters)
    +displayDoctorList(results)
    +displayValidationError(msg)
    +displayEmptyState(msg)
    +onViewProfile(doctorId)
    +onBookAppointment(doctorId)
  }

  class DoctorProfileUI {
    <<Boundary>>
    +renderProfile(doctorId)
  }

  class BookingUI {
    <<Boundary>>
    +renderBookingForm(doctorId)
  }
```

```
class DoctorSearchController {
    <<Control>>
    +initDashboard()
    +searchDoctors(criteria)
    +routeToProfile(doctorId)
    +routeToBooking(doctorId)
}

class SearchService {
    <<Control>>
    +getFilterOptions() List
    +validateInput(criteria) bool
    +findDoctors(criteria) List
    -sanitizeInput(input) String
}

class DoctorProfile {
    <<Entity>>
    +doctorId int
    +fullName String
    +specialtyId int
    +averageRating float
    +consultationFee decimal
    +approvalStatus String
    +findActiveByCriteria(criteria) List
```

```
}

class Specialty {
    <<Entity>>
    +specialtyId int
    +specialtyName String
    +getAllSpecialties() List
}

class DoctorSchedule {
    <<Entity>>
    +doctorId int
    +slotDate Date
    +startTime Time
    +status String
    +getNextAvailableSlots(doctorIds) List
}

DoctorSearchDashboardUI ..> DoctorSearchController : calls
DoctorSearchController ..> SearchService : uses
DoctorSearchController ..> DoctorProfileUI : redirects
DoctorSearchController ..> BookingUI : redirects

SearchService ..> DoctorProfile : queries (BR-07)
SearchService ..> Specialty : queries
SearchService ..> DoctorSchedule : queries
```

```
sequenceDiagram
    actor User
    participant SearchUI as <<Boundary>><br>DoctorSearchDashboardUI
    participant ProfileUI as <<Boundary>><br>DoctorProfileUI
    participant BookUI as <<Boundary>><br>BookingUI
    participant Ctrl as <<Control>><br>DoctorSearchController
    participant Svc as <<Control>><br>SearchService
    participant SpecEnt as <<Entity>><br>Specialty
    participant DocEnt as <<Entity>><br>DoctorProfile
    participant SchedEnt as <<Entity>><br>DoctorSchedule

    %% Init Flow
    User->>+SearchUI: navigateToSearch()
    SearchUI->>+Ctrl: initDashboard()
    Ctrl->>+Svc: getFilterOptions()

    %% Quy tắc 4.1: Gọi Entity ngay trước khi xử lý (luồng thẳng)
    Svc->>+SpecEnt: getAllSpecialties()
    SpecEnt-->>-Svc: specialtyList

    Svc-->>-Ctrl: filterData
    Ctrl-->>-SearchUI: renderDashboard(filterData)
    SearchUI-->>-User: display UI (Search Input, Filters)

    %% Search & Filter Flow (Normal Flow + Alt 2a)
```

```
User->>+SearchUI: inputCriteriaAndSearch(keyword, filters)
SearchUI->>+Ctrl: searchDoctors(criteria)
Ctrl->>+Svc: validateInput(criteria)
Svc-->>-Ctrl: isValid

alt Exc 5a: Invalid Search Input (< 2 chars)
   Ctrl-->>SearchUI: displayValidationError("Min 2 valid chars required")
   SearchUI-->>User: show inline validation error
else Normal Flow & Alt 2a: Valid Criteria
   Ctrl->>+Svc: findDoctors(criteria)

   %% Quy tắc 4.1: Gọi Entity và lấy kết quả ngay trước nhánh alt
   Svc->>+DocEnt: findActiveByCriteria(criteria)
   Note over DocEnt: BR-07: Fetch strictly approved doctors
   DocEnt-->>-Svc: doctorList

   Svc->>+SchedEnt: getNextAvailableSlots(doctorIds)
   SchedEnt-->>-Svc: slotsList
   Svc-->>-Ctrl: searchResults

   alt Exc 5b: No Doctors Found
      Ctrl-->>SearchUI: displayEmptyState("No doctors matching criteria")
      SearchUI-->>User: show empty state graphic & Clear Filters
   else Normal Flow: Doctors Found
      Ctrl-->>SearchUI: displayDoctorList(searchResults)
      SearchUI-->>User: display paginated list of doctor cards
```

```
    end
end
%% Quy tắc 4.2: Tắt Ctrl và SearchUI ở ngoài cùng vì được kích hoạt trước alt
deactivate Ctrl
deactivate SearchUI

%% Quy tắc 4.4: Tách luồng rẽ nhánh thao tác UI thành các opt riêng biệt
opt View Doctor Profile (Flow 6a - Variant)
    User->>+SearchUI: clickViewProfile(doctorId)
    SearchUI->>+Ctrl: routeToProfile(doctorId)
    Ctrl->>+ProfileUI: renderProfile(doctorId)
    ProfileUI-->>User: redirect to Doctor Profile Screen

    %% Quy tắc 4.3: Tắt các UI/Ctrl được sinh ra trong opt ngay trước khi end opt
    deactivate ProfileUI
    Ctrl-->>SearchUI: routed
    deactivate Ctrl
    deactivate SearchUI
end

opt Navigate to Book Appointment (Alt Flow 6a)
    User->>+SearchUI: clickBookAppointment(doctorId)
    SearchUI->>+Ctrl: routeToBooking(doctorId)
    Ctrl->>+BookUI: renderBookingForm(doctorId)
    BookUI-->>User: redirect to Book Consultation Screen (UC-17)
```

```
    %% Quy tắc 4.3: Tắt các UI/Ctrl được sinh ra trong opt ngay trước khi end opt
    deactivate BookUI
    Ctrl-->>SearchUI: routed
    deactivate Ctrl
    deactivate SearchUI
  end
```

UC-15: View Doctor Information

```
classDiagram
  direction LR

  class SearchResultsUI {
    <<Boundary>>
    +displaySearchResults(results)
    +onViewProfileClick(doctorId)
  }

  class DoctorProfileUI {
    <<Boundary>>
    +renderFullProfile(data, slots, reviews)
    +updateFeedbackSection(reviews, page)
    +onNextPageClick(page)
    +onBookAppointmentClick(doctorId, slotId)
    +onBackToSearchClick()
  }
```

```
class ErrorPageUI {
    <<Boundary>>
    +displayUnavailableError(message)
    +onReturnToSearchClick()
}

class DoctorProfileController {
    <<Control>>
    +loadProfile(doctorId)
    +loadFeedbackPage(doctorId, page, size)
    +redirectToBooking(doctorId, slotId)
    +navigateBack()
}

class DoctorProfileEntity {
    <<Entity>>
    +getDoctorById(doctorId)
}

class DoctorScheduleEntity {
    <<Entity>>
    +getAvailableSlots(doctorId, from, days)
}

class FeedbackEntity {
    <<Entity>>
```

```
        +getFeedbackPage(doctorId, page, size)
        +maskPatientName(fullName)
    }

    SearchResultsUI --> DoctorProfileController : triggers loadProfile
    DoctorProfileUI --> DoctorProfileController : triggers actions
    DoctorProfileController --> DoctorProfileUI : renders profile
    DoctorProfileController --> ErrorPageUI : renders on EF-2a
    ErrorPageUI --> DoctorProfileController : triggers navigateBack
    DoctorProfileController --> DoctorProfileEntity : queries
    DoctorProfileController --> DoctorScheduleEntity : queries
    DoctorProfileController --> FeedbackEntity : queries

sequenceDiagram
    actor User
    participant SearchResultsUI
    participant DoctorProfileUI
    participant ErrorPageUI
    participant DoctorProfileController
    participant DoctorProfileEntity
    participant DoctorScheduleEntity
    participant FeedbackEntity

    %% NF Step 1: Trigger từ UC-14
    User->>+SearchResultsUI: clickViewProfile(doctorId)
    SearchResultsUI->>+DoctorProfileController: loadProfile(doctorId)
```

```
deactivate SearchResultsUI

%% NF Step 2: Quy tắc 4.1 - Query entity TRƯỚC alt
DoctorProfileController->>+DoctorProfileEntity: getDoctorById(doctorId)
DoctorProfileEntity-->>-DoctorProfileController: doctorResult

alt [BR-07] Profile active & approved - Normal Flow
    DoctorProfileController->>+DoctorScheduleEntity: getAvailableSlots(doctorId, today, 7)
    DoctorScheduleEntity-->>-DoctorProfileController: slotList

    DoctorProfileController->>+FeedbackEntity: getFeedbackPage(doctorId, 1, 5)
    FeedbackEntity-->>-DoctorProfileController: maskedFeedbackList

    %% NF Step 3: Render - Quy tắc 4.3: DoctorProfileUI sinh trong nhánh này
    DoctorProfileController->>+DoctorProfileUI: renderFullProfile(doctorData, slotList, maskedFeedbackList)
    DoctorProfileUI-->>User: displayDoctorProfileScreen
    deactivate DoctorProfileUI

else Profile inactive / not found - EF-2a
    %% Quy tắc 4.3: ErrorPageUI sinh trong nhánh else, tắt ngay trong else
    DoctorProfileController->>+ErrorPageUI: displayUnavailableError("This doctor's profile is currently unavailable or under review by our moderation team.")
    ErrorPageUI-->>User: render404PageWithReturnButton
    deactivate ErrorPageUI
end
%% Quy tắc 4.2: DoctorProfileController mở trước alt, đóng sau end
```

```
deactivate DoctorProfileController

%% Quy tắc 4.4: AF-3a độc lập - tách thành opt riêng
opt AF-3a: User clicks "Next Page" in Patient Feedback section
   User->>+DoctorProfileUI: clickNextPage(page)
   DoctorProfileUI->>+DoctorProfileController: loadFeedbackPage(doctorId, page, 5)
   DoctorProfileController->>+FeedbackEntity: getFeedbackPage(doctorId, page, 5)
   FeedbackEntity-->>-DoctorProfileController: maskedFeedbackList
   DoctorProfileController-->>-DoctorProfileUI: feedbackPageData
   DoctorProfileUI-->>-User: updateFeedbackSection
end

%% Quy tắc 4.4: AF-4a độc lập - tách thành opt riêng
opt AF-4a: User selects slot or clicks "Book Appointment"
   User->>+DoctorProfileUI: clickBookAppointment(doctorId, slotId)
   DoctorProfileUI->>+DoctorProfileController: redirectToBooking(doctorId, slotId)
   DoctorProfileController-->>-DoctorProfileUI: navigateTo(UC-17, doctorId, slotId)
   DoctorProfileUI-->>-User: redirectToBookConsultationScreen
end

%% NF Step 5-6: Quy tắc 4.4 - luồng tuần tự độc lập sau khi user xem xong
opt NF Step 5-6: User clicks "Back to Search"
   User->>+DoctorProfileUI: clickBackToSearch()
   DoctorProfileUI->>+DoctorProfileController: navigateBack()
   DoctorProfileController-->>-DoctorProfileUI: navigateTo(SearchResultsUI)
   DoctorProfileUI-->>-User: redirectToSearchResults
```

End

UC-16: View Patient Medical History

```
classDiagram
direction LR

class MedicalHistoryDashboardUI {
<<Boundary>>
+renderDashboard(recordList)
+renderEmptyState(message)
+onApplyFilter(criteria)
+onViewDetailsClick(recordId)
+onFindDoctorClick()
}

class MedicalRecordDetailUI {
<<Boundary>>
+renderRecordDetail(detail, prescription, attachments)
+onDownloadPDFClick(recordId)
+onBookFollowUpClick(doctorId)
+onBackToHistoryClick()
}

class ErrorPageUI {
<<Boundary>>
```

```
+displayForbiddenError(message)
}

class MedicalHistoryController {
<<Control>>
+loadMedicalHistory(patientId)
+filterRecords(patientId, criteria)
+loadRecordDetail(patientId, recordId)
+generatePDF(recordId, patientDOB)
+redirectToBookConsultation(doctorId, purpose)
+navigateBack()
}

class PDFGeneratorService {
<<Control>>
+generatePasswordProtectedPDF(detail, patientDOB)
}

class MedicalRecordEntity {
<<Entity>>
+getRecordsByPatientId(patientId)
+getFilteredRecords(patientId, criteria)
+verifyRecordOwnership(patientId, recordId)
+getRecordDetail(recordId)
}
```

```
class PrescriptionEntity {
<<Entity>>
+getPrescriptionByRecordId(recordId)
}

class LabAttachmentEntity {
<<Entity>>
+getAttachmentsByRecordId(recordId)
}

class SecurityAuditEntity {
<<Entity>>
+logUnauthorizedAccess(patientId, recordId)
}

MedicalHistoryDashboardUI --> MedicalHistoryController : triggers
MedicalRecordDetailUI --> MedicalHistoryController : triggers
MedicalHistoryController --> MedicalHistoryDashboardUI : renders
MedicalHistoryController --> MedicalRecordDetailUI : renders
MedicalHistoryController --> ErrorPageUI : renders on EF-5a
MedicalHistoryController --> PDFGeneratorService : delegates AF-6b
MedicalHistoryController --> MedicalRecordEntity : queries
MedicalHistoryController --> PrescriptionEntity : queries
MedicalHistoryController --> LabAttachmentEntity : queries
MedicalHistoryController --> SecurityAuditEntity : logs on EF-5a
sequenceDiagram
```

```
actor User
participant MedicalHistoryDashboardUI
participant MedicalRecordDetailUI
participant ErrorPageUI
participant MedicalHistoryController
participant MedicalRecordEntity
participant PrescriptionEntity
participant LabAttachmentEntity
participant SecurityAuditEntity
participant PDFGeneratorService

%% ===== NF Steps 1-3: Load Medical History Dashboard =====
User->>+MedicalHistoryDashboardUI: navigateTo("Medical History")
MedicalHistoryDashboardUI->>+MedicalHistoryController: loadMedicalHistory(patientId)

%% Rule 4.1: Entity query and return BEFORE alt
MedicalHistoryController->>+MedicalRecordEntity: getRecordsByPatientId(patientId)
MedicalRecordEntity-->>-MedicalHistoryController: recordList

alt [Records exist] - NF-3
    MedicalHistoryController-->>MedicalHistoryDashboardUI: renderDashboard(recordList)
else [EF-2a: No records associated with patientId]
    MedicalHistoryController-->>MedicalHistoryDashboardUI: renderEmptyState("You currently have no medical records in the system.")
end
%% Rule 4.2: Both opened before alt, close after end
```

```
deactivate MedicalHistoryController
deactivate MedicalHistoryDashboardUI

%% ===== AF-3a: Filter/Search Records =====
opt AF-3a: User applies date range or Record Type filter
  User->>+MedicalHistoryDashboardUI: applyFilter(criteria)
  MedicalHistoryDashboardUI->>+MedicalHistoryController: filterRecords(patientId, criteria)
  MedicalHistoryController->>+MedicalRecordEntity: getFilteredRecords(patientId, criteria)
  MedicalRecordEntity-->>-MedicalHistoryController: filteredList
  MedicalHistoryController-->>MedicalHistoryDashboardUI: updateRecordList(filteredList)
  deactivate MedicalHistoryController
  MedicalHistoryDashboardUI-->>-User: displayFilteredList
end

%% ===== NF Steps 4-6: View Record Detail =====
User->>+MedicalHistoryDashboardUI: clickViewDetails(recordId)
MedicalHistoryDashboardUI->>+MedicalHistoryController: loadRecordDetail(patientId,
recordId)

%% Rule 4.1: ALL entity reads before alt (including ownership check)
MedicalHistoryController->>+MedicalRecordEntity: verifyRecordOwnership(patientId,
recordId)
MedicalRecordEntity-->>-MedicalHistoryController: ownershipResult

MedicalHistoryController->>+MedicalRecordEntity: getRecordDetail(recordId)
MedicalRecordEntity-->>-MedicalHistoryController: recordDetail
```

```
MedicalHistoryController->>+PrescriptionEntity: getPrescriptionByRecordId(recordId)
PrescriptionEntity-->>-MedicalHistoryController: prescriptionData

MedicalHistoryController->>+LabAttachmentEntity: getAttachmentsByRecordId(recordId)
LabAttachmentEntity-->>-MedicalHistoryController: attachmentList

alt [BR-08: Authorized - recordId belongs to patientId] - NF-5 & NF-6
    %% Rule 4.3: MedicalRecordDetailUI opened inside alt, close inside alt
    MedicalHistoryController->>+MedicalRecordDetailUI: renderRecordDetail(recordDetail, prescriptionData, attachmentList)
    MedicalRecordDetailUI-->>User: displayRecordDetailScreen
    deactivate MedicalRecordDetailUI
else [EF-5a: Unauthorized - recordId does not belong to patientId]
    %% Fire-and-forget write: activate then immediately deactivate (no return arrow)
    %% to comply with Rule 4.1 (no entity return inside alt)
    MedicalHistoryController->>+SecurityAuditEntity: logUnauthorizedAccess(patientId, recordId)
    deactivate SecurityAuditEntity
    %% Rule 4.3: ErrorPageUI opened inside else, close inside else
    MedicalHistoryController->>+ErrorPageUI: displayForbiddenError("You do not have permission to view this medical record.")
    ErrorPageUI-->>User: render403Page
    deactivate ErrorPageUI
end
%% Rule 4.2: Controller and DashboardUI opened before alt, close after end
deactivate MedicalHistoryController
deactivate MedicalHistoryDashboardUI
```

```
%% ===== AF-6b: Download PDF =====
opt AF-6b: User clicks "Download PDF"
    User->>+MedicalRecordDetailUI: clickDownloadPDF(recordId)
    MedicalRecordDetailUI->>+MedicalHistoryController: generatePDF(recordId, patientDOB)
    MedicalHistoryController->>+PDFGeneratorService: generatePasswordProtectedPDF(recordDetail, patientDOB)
    PDFGeneratorService-->>-MedicalHistoryController: pdfFile
    MedicalHistoryController-->>MedicalRecordDetailUI: initiateDownload(pdfFile)
    deactivate MedicalHistoryController
    MedicalRecordDetailUI-->>-User: downloadFile (remains on detail screen)
end

%% ===== AF-6a: Book Follow-up Appointment =====
opt AF-6a: User clicks "Book Follow-up"
    User->>+MedicalRecordDetailUI: clickBookFollowUp(doctorId)
    MedicalRecordDetailUI->>+MedicalHistoryController: redirectToBookConsultation(doctorId, "Follow-up")
    MedicalHistoryController-->>-MedicalRecordDetailUI: navigateTo(UC-17, doctorId, purpose)
    MedicalRecordDetailUI-->>-User: redirectToBookConsultationScreen
end

%% ===== NF: Back to History =====
opt NF: User clicks "Back to History"
    User->>+MedicalRecordDetailUI: clickBackToHistory()
    MedicalRecordDetailUI->>+MedicalHistoryController: navigateBack()
```

```
    MedicalHistoryController-->>-MedicalRecordDetailUI: navigateTo(MedicalHistoryDashboard)
    MedicalRecordDetailUI-->>-User: displayMedicalHistoryDashboard
  end
```

UC-17: Book Consultation

```
classDiagram
  direction TB

  class BookingUI {
    <<Boundary>>
    +clickBookAppointment(doctorId)
    +renderBookingForm()
    +fillFormAndClickContinue(formData)
    +highlightErrors(message)
  }

  class BookingReviewUI {
    <<Boundary>>
    +displayReviewSummary(reviewData)
    +renderReviewScreen()
    +clickConfirmAndPay()
  }

  class AddDependentUI {
    <<Boundary>>
```

```
    +selectAddNewDependent()
    +renderInlineDependentForm()
    +submitDependentForm(data)
    +showProfileAutoSelected()
}

class ConsultRoomUI {
    <<Boundary>>
    +joinRoom(appointmentId)
    +renderVideoCall()
    +clickEndCall()
}

class PostConsultUI {
    <<Boundary>>
    +displaySummary(appointmentData)
    +renderPostConsultationSummary()
    +clickCreatePrescription()
}

class BookingController {
    <<Control>>
    +loadBookingPage(doctorId)
    +validateAndProceed(formData)
    +redirectToPayment(appointmentId, fee)
}
```

```
class DependentController {
    <<Control>>
    +saveNewDependent(data)
}

class ConsultationController {
    <<Control>>
    +startSession(appointmentId)
    +endSession(appointmentId)
    +redirectToPrescription(appointmentId)
}

class PatientProfile {
    <<Entity>>
    +getProfilesWithDependents(userId)
}

class DoctorSchedule {
    <<Entity>>
    +getAvailableSlots(doctorId, next30Days)
    +lockSlot(slotId, durationMin)
}

class Appointment {
    <<Entity>>
```

```
    +createAppointment(formData, status)
    +updateStatus(appointmentId, status)
  }

  class DependentProfile {
    <<Entity>>
    +save(data)
  }

  BookingUI --> BookingController
  BookingReviewUI --> BookingController
  AddDependentUI --> DependentController
  ConsultRoomUI --> ConsultationController
  PostConsultUI --> ConsultationController
  BookingController --> PatientProfile
  BookingController --> DoctorSchedule
  BookingController --> Appointment
  DependentController --> DependentProfile
  ConsultationController --> Appointment


sequenceDiagram
  actor Patient
  actor Doctor
  participant BookingUI as BookingUI<br/><<Boundary>>
  participant AddDependentUI as AddDependentUI<br/><<Boundary>>
```

```
participant ReviewUI as BookingReviewUI<br/><<Boundary>>
participant ConsultRoomUI as ConsultRoomUI<br/><<Boundary>>
participant PostConsultUI as PostConsultUI<br/><<Boundary>>
participant BookingCtrl as BookingController<br/><<Control>>
participant DependentCtrl as DependentController<br/><<Control>>
participant ConsultCtrl as ConsultationController<br/><<Control>>
participant PatientEnt as PatientProfile<br/><<Entity>>
participant ScheduleEnt as DoctorSchedule<br/><<Entity>>
participant ApptEnt as Appointment<br/><<Entity>>
participant DependentEnt as DependentProfile<br/><<Entity>>

%% NF Steps 1-2: Load Booking Page
Patient->>+BookingUI: clickBookAppointment(doctorId)
BookingUI->>+BookingCtrl: loadBookingPage(doctorId)
BookingCtrl->>+PatientEnt: getProfilesWithDependents(userId)
PatientEnt-->>-BookingCtrl: profiles[]
BookingCtrl->>+ScheduleEnt: getAvailableSlots(doctorId, next30Days)
ScheduleEnt-->>-BookingCtrl: availableSlots[]
BookingCtrl-->>-BookingUI: bookingPageData
BookingUI-->>Patient: renderBookingForm()

%% [2a] Alt Flow: Add New Dependent — Rule 4.4: separate opt block
opt [2a] User selects "Add New Dependent"
   Patient->>+AddDependentUI: selectAddNewDependent()
   AddDependentUI-->>Patient: renderInlineDependentForm()
   Patient->>AddDependentUI: submitDependentForm(data)
```

```
    AddDependentUI->>+DependentCtrl: saveNewDependent(data)
    DependentCtrl->>+DependentEnt: save(data)
    DependentEnt-->>-DependentCtrl: newDependentId
    DependentCtrl-->>-AddDependentUI: dependentSaved(newDependentId)
    AddDependentUI-->>-Patient: showProfileAutoSelected()
end

%% NF Steps 3-8: Fill form, validate, lock slot, create appointment
Patient->>BookingUI: fillFormAndClickContinue(formData)
BookingUI->>+BookingCtrl: validateAndProceed(formData)

%% Rule 4.1: Validation is in-memory — no entity call needed before alt
alt [5a] Missing mandatory fields
    BookingCtrl-->>BookingUI: validationFailed(emptyFields)
    BookingUI-->>Patient: highlightErrors(message)
else All fields valid — Normal Flow continues
    BookingCtrl->>+ScheduleEnt: lockSlot(slotId, 10)
    ScheduleEnt-->>-BookingCtrl: slotLocked
    BookingCtrl->>+ApptEnt: createAppointment(formData, "PendingPayment")
    ApptEnt-->>-BookingCtrl: appointmentId
    BookingCtrl-->>BookingUI: reviewData
    BookingUI->>+ReviewUI: displayReviewSummary(reviewData)
    ReviewUI-->>Patient: renderReviewScreen()
    Patient->>ReviewUI: clickConfirmAndPay()
    ReviewUI->>BookingCtrl: redirectToPayment(appointmentId, fee)
    BookingCtrl-->>ReviewUI: handoffToPaymentModule()
```

```
    deactivate ReviewUI
end
%% Rule 4.2: BookingCtrl & BookingUI activated before alt → deactivate after end
deactivate BookingCtrl
deactivate BookingUI

%% [9a] Alt Flow: Conduct and Conclude Consultation — Rule 4.4: separate opt block
opt [9a] At scheduled time — Consultation Session
    Patient->>+ConsultRoomUI: joinRoom(appointmentId)
    Doctor->>ConsultRoomUI: joinRoom(appointmentId)
    ConsultRoomUI->>+ConsultCtrl: startSession(appointmentId)
    ConsultCtrl-->>-ConsultRoomUI: webRTCSessionInitiated
    ConsultRoomUI-->>Patient: renderVideoCall()
    ConsultRoomUI-->>Doctor: renderVideoCall()

    Doctor->>ConsultRoomUI: clickEndCall()
    ConsultRoomUI->>+ConsultCtrl: endSession(appointmentId)

    %% Rule 4.1: Call entity and return result before any branching
    ConsultCtrl->>+ApptEnt: updateStatus(appointmentId, "Completed")
    ApptEnt-->>-ConsultCtrl: statusUpdated

    ConsultCtrl-->>ConsultRoomUI: sessionCompleted
    deactivate ConsultRoomUI

    %% ConsultCtrl stays active to handle post-session redirect
```

```
    ConsultCtrl->>+PostConsultUI: displaySummary(appointmentData)
    PostConsultUI-->>Doctor: renderPostConsultationSummary()
    Doctor->>PostConsultUI: clickCreatePrescription()
    PostConsultUI->>ConsultCtrl: redirectToPrescription(appointmentId)
    ConsultCtrl-->>PostConsultUI: handoffToUC19()
    deactivate PostConsultUI
    deactivate ConsultCtrl
  end
```

UC - 18: Manage Doctor Schedule

```
classDiagram
  direction TB

  class ScheduleDashboardUI {
    <<Boundary>>
    +clickManageSchedule()
    +renderScheduleDashboard(calendarData)
    +clickBlockTimeOff()
    +clickAddNewSchedule()
    +renderScheduleCreationModal()
    +clickGenerateSlots(scheduleConfig)
    +renderSlotPreviewList(slots)
    +clickDeleteSlot(slotIndex)
    +removeSlotFromPreview(slotIndex)
    +clickConfirmAndSave()
    +showSuccessToast(message)
```

```
    +showOverlapError(message)
}

class TimeOffModalUI {
    <<Boundary>>
    +renderTimeOffModal()
    +submitTimeOff(timeOffData)
    +showBookedSlotError(message)
}

class ScheduleController {
    <<Control>>
    +loadScheduleDashboard(doctorId)
    +generateSlots(scheduleConfig)
    +confirmAndSave(slots, doctorId)
    +blockTimeOff(timeOffData, doctorId)
}

class DoctorSchedule {
    <<Entity>>
    +getCurrentSchedule(doctorId)
    +checkBookedSlots(timeRange, doctorId)
    +updateToUnavailable(timeRange, doctorId)
    +checkOverlap(slots, doctorId)
    +saveSlots(slots, doctorId)
}
```

```
    ScheduleDashboardUI --> ScheduleController
    ScheduleDashboardUI --> TimeOffModalUI
    TimeOffModalUI --> ScheduleController
    ScheduleController --> DoctorSchedule

sequenceDiagram
    actor Doctor
    participant ScheduleUI as ScheduleDashboardUI<br/><<Boundary>>
    participant TimeOffUI as TimeOffModalUI<br/><<Boundary>>
    participant ScheduleCtrl as ScheduleController<br/><<Control>>
    participant ScheduleEnt as DoctorSchedule<br/><<Entity>>

    %% NF Steps 1-2: Load Schedule Dashboard
    Doctor->>+ScheduleUI: clickManageSchedule()
    ScheduleUI->>+ScheduleCtrl: loadScheduleDashboard(doctorId)
    ScheduleCtrl->>+ScheduleEnt: getCurrentSchedule(doctorId)
    ScheduleEnt-->>-ScheduleCtrl: calendarData
    ScheduleCtrl-->>-ScheduleUI: calendarData
    ScheduleUI-->>-Doctor: renderScheduleDashboard(calendarData)

    %% [2a] Block Time Off — Rule 4.4: independent opt block
    opt [2a] Doctor clicks Block Time Off
        Doctor->>+ScheduleUI: clickBlockTimeOff()
        ScheduleUI->>+TimeOffUI: renderTimeOffModal()
        TimeOffUI-->>Doctor: renderTimeOffModal()
```

```
    Doctor->>TimeOffUI: submitTimeOff(timeOffData)
    TimeOffUI->>+ScheduleCtrl: blockTimeOff(timeOffData, doctorId)

    %% Rule 4.1: Entity call and return BEFORE nested alt [3a]
    ScheduleCtrl->>+ScheduleEnt: checkBookedSlots(timeRange, doctorId)
    ScheduleEnt-->>-ScheduleCtrl: conflictResult

    alt [3a] Booked slot found in time range
      ScheduleCtrl-->>TimeOffUI: bookedSlotConflict
      TimeOffUI-->>Doctor: showBookedSlotError(message)
    else No conflict — save time off
      %% Rule 4.3: ScheduleEnt spawned inside else → deactivate inside
      ScheduleCtrl->>+ScheduleEnt: updateToUnavailable(timeRange, doctorId)
      ScheduleEnt-->>-ScheduleCtrl: updated
      ScheduleCtrl-->>TimeOffUI: timeOffSaved
    end
    %% Rule 4.2: ScheduleCtrl and TimeOffUI activated before nested alt → deactivate after
end
    deactivate ScheduleCtrl
    deactivate TimeOffUI

    ScheduleUI-->>-Doctor: showSuccessToast(message)
  end

  %% NF Step 3: Add New Schedule modal
  Doctor->>+ScheduleUI: clickAddNewSchedule()
```

```
ScheduleUI-->>-Doctor: renderScheduleCreationModal()

%% NF Steps 5-7: Generate and preview slots
Doctor->>+ScheduleUI: clickGenerateSlots(scheduleConfig)
ScheduleUI->>+ScheduleCtrl: generateSlots(scheduleConfig)
ScheduleCtrl-->>-ScheduleUI: slots[]
ScheduleUI-->>-Doctor: renderSlotPreviewList(slots)

%% [7a] Remove individual slot — Rule 4.4: separate opt, in-memory only
opt [7a] Doctor removes a slot from preview
   Doctor->>+ScheduleUI: clickDeleteSlot(slotIndex)
   ScheduleUI-->>-Doctor: removeSlotFromPreview(slotIndex)
end

%% NF Steps 8-10: Confirm, validate overlap, save
Doctor->>+ScheduleUI: clickConfirmAndSave()
ScheduleUI->>+ScheduleCtrl: confirmAndSave(slots, doctorId)

%% Rule 4.1: Entity call and return BEFORE alt [9a]
ScheduleCtrl->>+ScheduleEnt: checkOverlap(slots, doctorId)
ScheduleEnt-->>-ScheduleCtrl: conflictResult

alt [9a] Overlap detected
   ScheduleCtrl-->>ScheduleUI: overlapConflict
   ScheduleUI-->>Doctor: showOverlapError(message)
else No overlap — Normal Flow
```

```
    %% Rule 4.3: ScheduleEnt spawned inside else → deactivate inside
    ScheduleCtrl->>+ScheduleEnt: saveSlots(slots, doctorId)
    ScheduleEnt-->>-ScheduleCtrl: saved
    ScheduleCtrl-->>ScheduleUI: slotsSaved
  end
  %% Rule 4.2: ScheduleCtrl and ScheduleUI activated before alt → deactivate after end
  deactivate ScheduleCtrl
  ScheduleUI-->>-Doctor: showSuccessToast(message)
```

UC - 19: Provide prescription

```
classDiagram
  direction TB

  class PrescriptionWorkspaceUI {
    <<Boundary>>
    +clickCreatePrescription()
    +renderPrescriptionWorkspace()
    +searchMedication(keyword)
    +renderMedicationAutocomplete(results)
    +clickAddMedication()
    +appendMedicationRow(item)
    +clickRemoveMedication(itemIndex)
    +removeMedicationRow(itemIndex)
    +clickIssueAndSend()
    +validationFailed(message)
    +prescriptionIssued()
```

```
    +showSuccessPrompt()
}

class PrescriptionController {
    <<Control>>
    +loadWorkspace(appointmentId)
    +searchMedication(keyword)
    +validateAndAddMedication(itemData)
    +issueAndSendPrescription(prescriptionData)
}

class PatientProfile {
    <<Entity>>
    +getPatientInfo(patientId)
}

class Medication {
    <<Entity>>
    +searchActiveMedications(keyword)
}

class MedicalRecord {
    <<Entity>>
    +getByAppointmentId(appointmentId)
}
```

```
    class Prescription {
        <<Entity>>
        +createPrescription(data)
    }

    class PrescriptionItem {
        <<Entity>>
        +saveItems(prescriptionId, items)
    }

    PrescriptionWorkspaceUI --> PrescriptionController
    PrescriptionController --> PatientProfile
    PrescriptionController --> Medication
    PrescriptionController --> MedicalRecord
    PrescriptionController --> Prescription
    PrescriptionController --> PrescriptionItem

sequenceDiagram
    actor Doctor
    participant WorkspaceUI as PrescriptionWorkspaceUI<br/><<Boundary>>
    participant PrescriptionCtrl as PrescriptionController<br/><<Control>>
    participant PatientEnt as PatientProfile<br/><<Entity>>
    participant MedicationEnt as Medication<br/><<Entity>>
    participant MedRecordEnt as MedicalRecord<br/><<Entity>>
    participant PrescriptionEnt as Prescription<br/><<Entity>>
    participant PrescriptionItemEnt as PrescriptionItem<br/><<Entity>>
```

```
%% NF Steps 1-2: Load workspace
Doctor->>+WorkspaceUI: clickCreatePrescription()
WorkspaceUI->>+PrescriptionCtrl: loadWorkspace(appointmentId)
PrescriptionCtrl->>+PatientEnt: getPatientInfo(patientId)
PatientEnt-->>-PrescriptionCtrl: patientInfo
PrescriptionCtrl-->>-WorkspaceUI: workspaceData
WorkspaceUI-->>Doctor: renderPrescriptionWorkspace()
deactivate WorkspaceUI

%% NF Step 4: Search medication
Doctor->>+WorkspaceUI: searchMedication(keyword)
WorkspaceUI->>+PrescriptionCtrl: searchMedication(keyword)
PrescriptionCtrl->>+MedicationEnt: searchActiveMedications(keyword)
MedicationEnt-->>-PrescriptionCtrl: medicationList[]
PrescriptionCtrl-->>-WorkspaceUI: medicationList[]
WorkspaceUI-->>-Doctor: renderMedicationAutocomplete(results)

%% NF Steps 5-6: Add first medication
Doctor->>+WorkspaceUI: clickAddMedication()
WorkspaceUI->>+PrescriptionCtrl: validateAndAddMedication(itemData)
PrescriptionCtrl-->>-WorkspaceUI: itemData
WorkspaceUI-->>-Doctor: appendMedicationRow(item)

%% [6a] Alt Flow: Add more medications — Rule 4.4: separate opt block
opt [6a] Repeat to add additional medications
```

```
    Doctor->>+WorkspaceUI: searchMedication(keyword)
    WorkspaceUI->>+PrescriptionCtrl: searchMedication(keyword)
    PrescriptionCtrl->>+MedicationEnt: searchActiveMedications(keyword)
    MedicationEnt-->>-PrescriptionCtrl: medicationList[]
    PrescriptionCtrl-->>-WorkspaceUI: medicationList[]
    WorkspaceUI-->>-Doctor: renderMedicationAutocomplete(results)

    Doctor->>+WorkspaceUI: clickAddMedication()
    WorkspaceUI->>+PrescriptionCtrl: validateAndAddMedication(itemData)
    PrescriptionCtrl-->>-WorkspaceUI: itemData
    WorkspaceUI-->>-Doctor: appendMedicationRow(item)
end

%% [7a] Alt Flow: Remove a medication — Rule 4.4: separate opt block
opt [7a] Doctor removes a medication row
    Doctor->>+WorkspaceUI: clickRemoveMedication(itemIndex)
    WorkspaceUI-->>-Doctor: removeMedicationRow(itemIndex)
end

%% NF Step 8: Issue and send prescription
Doctor->>+WorkspaceUI: clickIssueAndSend()
WorkspaceUI->>+PrescriptionCtrl: issueAndSendPrescription(prescriptionData)

%% Rule 4.1: Validation is in-memory — no entity call needed before alt
alt [8a] Diagnosis empty or medication table has zero items
    PrescriptionCtrl-->>WorkspaceUI: validationFailed(message)
```

```
    WorkspaceUI-->>Doctor: validationFailed(message)
  else All valid — Normal Flow continues
    %% Rule 4.1: All entity calls execute and return before any branching
    PrescriptionCtrl->>+MedRecordEnt: getByAppointmentId(appointmentId)
    MedRecordEnt-->>-PrescriptionCtrl: recordId
    PrescriptionCtrl->>+PrescriptionEnt: createPrescription(data)
    PrescriptionEnt-->>-PrescriptionCtrl: prescriptionId
    PrescriptionCtrl->>+PrescriptionItemEnt: saveItems(prescriptionId, items)
    PrescriptionItemEnt-->>-PrescriptionCtrl: itemsSaved
    PrescriptionCtrl-->>WorkspaceUI: prescriptionIssued()
    WorkspaceUI-->>Doctor: showSuccessPrompt()
  end
  %% Rule 4.2: PrescriptionCtrl and WorkspaceUI activated before alt → deactivate after end
  deactivate PrescriptionCtrl
  deactivate WorkspaceUI
```

UC - 28: View Medical History

```
classDiagram
  direction TB

  class MedicalHistoryDashboardUI {
    <<Boundary>>
    +clickMedicalHistory()
    +renderEmptyState()
    +renderDashboard(records)
    +inputSearchFilter(criteria)
```

```
    +renderFilteredRecords(records)
    +clickViewDetails(recordId)
    +showAccessDenied()
}

class MedicalRecordDetailUI {
    <<Boundary>>
    +renderRecordDetail(recordData)
    +clickDownloadPDF()
    +showDownloadSuccess()
    +clickBookFollowUp(doctorId)
    +clickBackToHistory()
    +navigateToDashboard()
}

class MedicalHistoryController {
    <<Control>>
    +loadMedicalHistory(patientId)
    +filterRecords(patientId, criteria)
    +getRecordDetail(recordId, patientId)
    +downloadPDF(recordId)
    +redirectToBooking(doctorId)
}

class MedicalRecord {
    <<Entity>>
```

```
    +findAllByPatientId(patientId)
    +searchByFilter(patientId, criteria)
    +findById(recordId)
  }

  class Prescription {
    <<Entity>>
    +getByRecordId(recordId)
  }

  MedicalHistoryDashboardUI --> MedicalHistoryController
  MedicalHistoryDashboardUI --> MedicalRecordDetailUI
  MedicalRecordDetailUI --> MedicalHistoryController
  MedicalHistoryController --> MedicalRecord
  MedicalHistoryController --> Prescription

sequenceDiagram
  actor Patient
  participant DashboardUI as MedicalHistoryDashboardUI<br/><<Boundary>>
  participant DetailUI as MedicalRecordDetailUI<br/><<Boundary>>
  participant MedHistoryCtrl as MedicalHistoryController<br/><<Control>>
  participant MedRecordEnt as MedicalRecord<br/><<Entity>>
  participant PrescriptionEnt as Prescription<br/><<Entity>>

  %% NF Steps 1-3: Load Medical History Dashboard
  Patient->>+DashboardUI: clickMedicalHistory()
```

```
DashboardUI->>+MedHistoryCtrl: loadMedicalHistory(patientId)

%% Rule 4.1: Call entity and return result before alt block
MedHistoryCtrl->>+MedRecordEnt: findAllByPatientId(patientId)
MedRecordEnt-->>-MedHistoryCtrl: records[]

alt [2a] No records found
   MedHistoryCtrl-->>DashboardUI: emptyResult
   DashboardUI-->>Patient: renderEmptyState()
else Records found — Normal Flow continues
   MedHistoryCtrl-->>DashboardUI: records[]
   DashboardUI-->>Patient: renderDashboard(records)
end
%% Rule 4.2: Both activated before alt → deactivate after end
deactivate MedHistoryCtrl
deactivate DashboardUI

%% [3a] Alt Flow: Filter/Search records — Rule 4.4: separate opt block
opt [3a] User inputs search or date filter
   Patient->>+DashboardUI: inputSearchFilter(criteria)
   DashboardUI->>+MedHistoryCtrl: filterRecords(patientId, criteria)
   MedHistoryCtrl->>+MedRecordEnt: searchByFilter(patientId, criteria)
   MedRecordEnt-->>-MedHistoryCtrl: filteredRecords[]
   MedHistoryCtrl-->>-DashboardUI: filteredRecords[]
   DashboardUI-->>-Patient: renderFilteredRecords(records)
end
```

```
%% NF Step 4 + [4a] Exception: View Record Detail with authorization check
Patient->>+DashboardUI: clickViewDetails(recordId)
DashboardUI->>+MedHistoryCtrl: getRecordDetail(recordId, patientId)

%% Rule 4.1: Fetch record and return before alt — result determines auth branch
MedHistoryCtrl->>+MedRecordEnt: findById(recordId)
MedRecordEnt-->>-MedHistoryCtrl: record

alt [4a] Unauthorized — record does not belong to patient
   MedHistoryCtrl-->>DashboardUI: accessDenied
   DashboardUI-->>Patient: showAccessDenied()
else Authorized — Normal Flow continues
   %% Secondary entity call inside else branch (downstream, not the branching condition)
   MedHistoryCtrl->>+PrescriptionEnt: getByRecordId(recordId)
   PrescriptionEnt-->>-MedHistoryCtrl: prescriptionData
   MedHistoryCtrl-->>DashboardUI: recordData
   %% DashboardUI hands off to DetailUI inside else — Rule 4.3 applies to DetailUI
   DashboardUI->>+DetailUI: renderRecordDetail(recordData)
   DetailUI-->>Patient: renderRecordDetail(recordData)
   deactivate DetailUI
end
%% Rule 4.2: DashboardUI and MedHistoryCtrl activated before alt → deactivate after end
deactivate MedHistoryCtrl
deactivate DashboardUI
```

```
    %% [5a] Alt Flow: Download PDF — Rule 4.4: separate opt block
    opt [5a] User clicks Download PDF
        Patient->>+DetailUI: clickDownloadPDF()
        DetailUI->>+MedHistoryCtrl: downloadPDF(recordId)
        MedHistoryCtrl-->>-DetailUI: pdfData
        DetailUI-->>-Patient: showDownloadSuccess()
    end

    %% [5b] Alt Flow: Book Follow-up — Rule 4.4: separate opt block
    opt [5b] User clicks Book Follow-up
        Patient->>+DetailUI: clickBookFollowUp(doctorId)
        DetailUI->>+MedHistoryCtrl: redirectToBooking(doctorId)
        MedHistoryCtrl-->>-DetailUI: handoffToUC17
        deactivate DetailUI
    end

    %% NF Steps 6-7: Back to History
    Patient->>+DetailUI: clickBackToHistory()
    DetailUI-->>-Patient: navigateToDashboard()
```

UC - 30: Submit Feedback

```
classDiagram
    direction TB

    class FeedbackModalUI {
```

```
    <<Boundary>>
    +clickLeaveFeedback(appointmentId)
    +renderFeedbackModal(doctorInfo)
    +showIneligibleError()
    +clickStarRating(stars)
    +inputReviewText(text)
    +clickSubmitFeedback()
    +validationFailed(message)
    +showSuccessMessage()
    +redirectToMedicalHistory()
    +clickCancel()
    +closeModal()
}

class FeedbackController {
    <<Control>>
    +loadFeedbackModal(appointmentId)
    +submitFeedback(feedbackData)
    +recalculateDoctorRating(doctorId)
}

class Appointment {
    <<Entity>>
    +verifyCompletedStatus(appointmentId)
}
```

```
    class Feedback {
        <<Entity>>
        +saveFeedback(data)
    }

    class DoctorProfile {
        <<Entity>>
        +updateAverageRating(doctorId)
    }

    FeedbackModalUI --> FeedbackController
    FeedbackController --> Appointment
    FeedbackController --> Feedback
    FeedbackController --> DoctorProfile


sequenceDiagram
    actor Patient
    participant FeedbackUI as FeedbackModalUI<br/><<Boundary>>
    participant FeedbackCtrl as FeedbackController<br/><<Control>>
    participant AppointmentEnt as Appointment<br/><<Entity>>
    participant FeedbackEnt as Feedback<br/><<Entity>>
    participant DoctorProfileEnt as DoctorProfile<br/><<Entity>>

    %% NF Steps 1-3: Load and verify before displaying modal
    Patient->>+FeedbackUI: clickLeaveFeedback(appointmentId)
```

```
FeedbackUI->>+FeedbackCtrl: loadFeedbackModal(appointmentId)

%% Rule 4.1: Call entity and return result right before alt block
FeedbackCtrl->>+AppointmentEnt: verifyCompletedStatus(appointmentId)
AppointmentEnt-->>-FeedbackCtrl: status

alt [2a] Status is not "Completed"
   FeedbackCtrl-->>FeedbackUI: ineligible
   FeedbackUI-->>Patient: showIneligibleError()
else Status is "Completed" — Normal Flow continues
   FeedbackCtrl-->>FeedbackUI: doctorInfo
   FeedbackUI-->>Patient: renderFeedbackModal(doctorInfo)
end
%% Rule 4.2: FeedbackCtrl and FeedbackUI activated before alt → deactivate after end
deactivate FeedbackCtrl
deactivate FeedbackUI

%% [3a] Alt Flow: Cancel — Rule 4.4: separate opt block
opt [3a] User clicks Cancel
   Patient->>+FeedbackUI: clickCancel()
   FeedbackUI-->>-Patient: closeModal()
end

%% NF Steps 4-5: User fills in the form
Patient->>+FeedbackUI: clickStarRating(stars)
FeedbackUI-->>-Patient: renderFeedbackModal(doctorInfo)
```

```
Patient->>+FeedbackUI: inputReviewText(text)
FeedbackUI-->>-Patient: renderFeedbackModal(doctorInfo)

%% NF Step 6: Submit feedback
Patient->>+FeedbackUI: clickSubmitFeedback()
FeedbackUI->>+FeedbackCtrl: submitFeedback(feedbackData)

%% Rule 4.1: Validation is in-memory (no entity call before alt)
alt [6a] Star rating not selected
   FeedbackCtrl-->>FeedbackUI: ratingMissing
   FeedbackUI-->>Patient: validationFailed(message)
else All valid — Normal Flow continues
   %% Rule 4.1: All entity calls execute and return before any branching
   FeedbackCtrl->>+FeedbackEnt: saveFeedback(feedbackData)
   FeedbackEnt-->>-FeedbackCtrl: feedbackId

   %% NF Step 8: Background recalculation of doctor's average rating
   FeedbackCtrl->>+DoctorProfileEnt: updateAverageRating(doctorId)
   DoctorProfileEnt-->>-FeedbackCtrl: ratingUpdated

   FeedbackCtrl-->>FeedbackUI: feedbackSubmitted
   FeedbackUI-->>Patient: showSuccessMessage()
   FeedbackUI-->>Patient: redirectToMedicalHistory()
end
%% Rule 4.2: FeedbackCtrl and FeedbackUI activated before alt → deactivate after end
```

```
    deactivate FeedbackCtrl
    deactivate FeedbackUI
```

UC - 33: Manage booking consultation

```
classDiagram
    direction TB

    class BookingDashboardUI {
        <<Boundary>>
        +clickBookingManagement()
        +renderDashboard(appointments)
        +inputSearchFilter(criteria)
        +renderFilteredAppointments(appointments)
        +clickActionMenu(appointmentId)
        +clickViewAndManage()
        +showSuccessMessage(message)
    }

    class AppointmentDetailUI {
        <<Boundary>>
        +renderAppointmentDetail(appointmentData)
        +clickCancelAppointment()
        +showInvalidStatusError()
        +clickRescheduleAppointment()
```

```
}

class CancellationModalUI {
    <<Boundary>>
    +renderCancellationModal()
    +inputCancellationReason(reason)
    +clickConfirmCancellation()
}

class ReschedulingModalUI {
    <<Boundary>>
    +renderReschedulingModal(availableSlots)
    +selectNewSlot(date, slotId)
    +clickConfirmReschedule()
    +showSlotConflictError()
}

class BookingManagementController {
    <<Control>>
    +loadDashboard()
    +filterAppointments(criteria)
    +getAppointmentDetail(appointmentId)
    +validateStatusForAction(appointmentId)
    +cancelAppointment(appointmentId, reason)
    +initiateRefund(appointmentId)
    +sendCancellationNotification(appointmentId)
```

```
    +loadAvailableSlots(doctorId)
    +confirmReschedule(appointmentId, slotId)
    +sendReschedulingNotification(appointmentId)
}

class Appointment {
    <<Entity>>
    - appointmentId: int
    - doctorId: int
    - patientId: int
    - dependentId: int
    - slotId: int
    - consultationType: String
    - reasonForVisit: String
    - status: String
    - cancellationReason: String
    +findAll()
    +searchByFilter(criteria)
    +findById(appointmentId)
    +updateStatusCancelled(appointmentId, reason)
    +updateSlot(appointmentId, slotId)
}

class DoctorSchedule {
    <<Entity>>
    - slotId: int
```

```
        - doctorId: int
        - slotDate: Date
        - startTime: Time
        - endTime: Time
        - status: String
        +releaseSlot(slotId)
        +getAvailableSlots(doctorId)
        +checkSlotAvailability(slotId)
        +lockSlot(slotId, appointmentId)
    }

    BookingDashboardUI --> BookingManagementController : triggers loadDashboard / filterAppointments
    BookingDashboardUI --> AppointmentDetailUI : renders detail screen
    AppointmentDetailUI --> BookingManagementController : triggers validate / cancel / reschedule
    AppointmentDetailUI --> CancellationModalUI : renders cancellation modal
    AppointmentDetailUI --> ReschedulingModalUI : renders rescheduling modal
    CancellationModalUI --> BookingManagementController : submits cancellation
    CancellationModalUI --> BookingDashboardUI : redirects on success
    ReschedulingModalUI --> BookingManagementController : submits reschedule
    ReschedulingModalUI --> BookingDashboardUI : redirects on success
    BookingManagementController --> Appointment : queries / updates status
    BookingManagementController --> DoctorSchedule : releases / locks / checks slots

sequenceDiagram
    actor Admin
```

```
participant DashboardUI as BookingDashboardUI<br/><<Boundary>>
participant DetailUI as AppointmentDetailUI<br/><<Boundary>>
participant CancelModalUI as CancellationModalUI<br/><<Boundary>>
participant RescheduleModalUI as ReschedulingModalUI<br/><<Boundary>>
participant BookingMgmtCtrl as BookingManagementController<br/><<Control>>
participant AppointmentEnt as Appointment<br/><<Entity>>
participant ScheduleEnt as DoctorSchedule<br/><<Entity>>

%% NF Steps 1-2: Load Dashboard
Admin->>+DashboardUI: clickBookingManagement()
DashboardUI->>+BookingMgmtCtrl: loadDashboard()
BookingMgmtCtrl->>+AppointmentEnt: findAll()
AppointmentEnt-->>-BookingMgmtCtrl: appointments[]
BookingMgmtCtrl-->>-DashboardUI: appointments[]
DashboardUI-->>-Admin: renderDashboard(appointments)

%% [2a] Filter — Rule 4.4: separate opt block
opt [2a] Admin filters by status or search keyword
   Admin->>+DashboardUI: inputSearchFilter(criteria)
   DashboardUI->>+BookingMgmtCtrl: filterAppointments(criteria)
   BookingMgmtCtrl->>+AppointmentEnt: searchByFilter(criteria)
   AppointmentEnt-->>-BookingMgmtCtrl: filteredAppointments[]
   BookingMgmtCtrl-->>-DashboardUI: filteredAppointments[]
   DashboardUI-->>-Admin: renderFilteredAppointments(appointments)
end
```

```
%% NF Steps 3-5: View & Manage → Load Detail
Admin->>+DashboardUI: clickActionMenu(appointmentId)
Admin->>DashboardUI: clickViewAndManage()
DashboardUI->>+BookingMgmtCtrl: getAppointmentDetail(appointmentId)

%% Rule 4.1: Call entity and return before any branching
BookingMgmtCtrl->>+AppointmentEnt: findById(appointmentId)
AppointmentEnt-->>-BookingMgmtCtrl: appointmentData

BookingMgmtCtrl-->>-DashboardUI: appointmentData
DashboardUI->>+DetailUI: renderAppointmentDetail(appointmentData)
DetailUI-->>-Admin: renderAppointmentDetail(appointmentData)
deactivate DashboardUI

%% NF Step 6: Cancel button clicked — [6b] status validation
Admin->>+DetailUI: clickCancelAppointment()
DetailUI->>+BookingMgmtCtrl: validateStatusForAction(appointmentId)
%% Rule 4.1: validateStatusForAction is in-memory — no entity call needed before alt

alt [6b] Status is already Completed or Cancelled
   BookingMgmtCtrl-->>DetailUI: invalidStatus
   DetailUI-->>Admin: showInvalidStatusError()
else Status is valid — Normal Flow continues
   BookingMgmtCtrl-->>DetailUI: statusValid
   DetailUI->>+CancelModalUI: renderCancellationModal()
   CancelModalUI-->>Admin: renderCancellationModal()
```

```
    deactivate CancelModalUI
end
%% Rule 4.2: DetailUI and BookingMgmtCtrl activated before alt → deactivate after end
deactivate BookingMgmtCtrl
deactivate DetailUI

%% NF Steps 7-9: Fill and confirm cancellation
Admin->>+CancelModalUI: inputCancellationReason(reason)
CancelModalUI-->>-Admin: renderCancellationModal()

Admin->>+CancelModalUI: clickConfirmCancellation()
CancelModalUI->>+BookingMgmtCtrl: cancelAppointment(appointmentId, reason)
BookingMgmtCtrl->>+AppointmentEnt: updateStatusCancelled(appointmentId, reason)
AppointmentEnt-->>-BookingMgmtCtrl: updated
BookingMgmtCtrl->>+ScheduleEnt: releaseSlot(slotId)
ScheduleEnt-->>-BookingMgmtCtrl: slotReleased
BookingMgmtCtrl->>BookingMgmtCtrl: initiateRefund(appointmentId)
BookingMgmtCtrl->>BookingMgmtCtrl: sendCancellationNotification(appointmentId)
BookingMgmtCtrl-->>-CancelModalUI: cancellationComplete
CancelModalUI->>+DashboardUI: showSuccessMessage(message)
DashboardUI-->>-Admin: showSuccessMessage(message)
deactivate CancelModalUI

%% [6a] Alt Flow: Reschedule — Rule 4.4: separate opt block
opt [6a] Admin clicks Reschedule Appointment instead
    Admin->>+DetailUI: clickRescheduleAppointment()
```

```
DetailUI->>+BookingMgmtCtrl: validateStatusForAction(appointmentId)
%% Rule 4.1: in-memory check — no entity call before alt

alt [6b] Status is already Completed or Cancelled
   BookingMgmtCtrl-->>DetailUI: invalidStatus
   DetailUI-->>Admin: showInvalidStatusError()
else Status is valid — load available slots
   %% Rule 4.1: getAvailableSlots call AND return are both inside else — no boundary crossing
   BookingMgmtCtrl->>+ScheduleEnt: getAvailableSlots(doctorId)
   ScheduleEnt-->>-BookingMgmtCtrl: availableSlots[]
   BookingMgmtCtrl-->>DetailUI: availableSlots[]
   DetailUI->>+RescheduleModalUI: renderReschedulingModal(availableSlots)
   RescheduleModalUI-->>Admin: renderReschedulingModal(availableSlots)
   deactivate RescheduleModalUI
end
%% Rule 4.2: DetailUI and BookingMgmtCtrl activated before alt → deactivate after end
deactivate BookingMgmtCtrl
deactivate DetailUI

Admin->>+RescheduleModalUI: selectNewSlot(date, slotId)
RescheduleModalUI-->>-Admin: renderReschedulingModal(availableSlots)

Admin->>+RescheduleModalUI: clickConfirmReschedule()
RescheduleModalUI->>+BookingMgmtCtrl: confirmReschedule(appointmentId, slotId)
```

```
    %% Rule 4.1: Call entity and return result before alt [4a]
    BookingMgmtCtrl->>+ScheduleEnt: checkSlotAvailability(slotId)
    ScheduleEnt-->>-BookingMgmtCtrl: availabilityResult

    alt [4a] Selected slot is no longer available
      BookingMgmtCtrl-->>RescheduleModalUI: slotConflict
      RescheduleModalUI-->>Admin: showSlotConflictError()
    else Slot is available — proceed with reschedule
      %% Rule 4.1: updateSlot and lockSlot call+return both inside else — no boundary crossing
      BookingMgmtCtrl->>+AppointmentEnt: updateSlot(appointmentId, slotId)
      AppointmentEnt-->>-BookingMgmtCtrl: updated
      BookingMgmtCtrl->>+ScheduleEnt: lockSlot(slotId, appointmentId)
      ScheduleEnt-->>-BookingMgmtCtrl: slotLocked
      BookingMgmtCtrl->>BookingMgmtCtrl: sendReschedulingNotification(appointmentId)
      BookingMgmtCtrl-->>RescheduleModalUI: rescheduleComplete
      %% Rule 4.3: DashboardUI spawned inside else → deactivate inside else
      RescheduleModalUI->>+DashboardUI: showSuccessMessage(message)
      DashboardUI-->>Admin: showSuccessMessage(message)
      deactivate DashboardUI
    end
    %% Rule 4.2: BookingMgmtCtrl and RescheduleModalUI activated before alt → deactivate after end
    deactivate BookingMgmtCtrl
    deactivate RescheduleModalUI
  end
```

UC20: Manage Shopping Cart

```
classDiagram
  direction TB

  class ShoppingCartUI {
    <<Boundary>>
    +clickCartIcon()
    +renderEmptyState()
    +renderCartWithStockWarning()
    +renderCart(cartData)
    +clickAdjustQuantity(productId, newQty)
    +renderOutOfStockError()
    +renderQuantityUpdated()
    +clickRemove(productId)
    +renderCartRecalculated()
    +clickProceedToCheckout()
  }

  class CartController {
    <<Control>>
    +loadCart(patientId)
    +adjustQuantity(cartId, productId, newQty)
    +removeItem(cartId, productId)
    +redirectToCheckout(cartData)
  }
```

```
class ShoppingCart {
    <<Entity>>
    - cartId: int
    - patientId: int
    - lastUpdated: DateTime
    +findByPatientId(patientId)
}

class CartItem {
    <<Entity>>
    - cartItemId: int
    - cartId: int
    - productId: int
    - quantity: int
    - addedAt: DateTime
    +findByCartId(cartId)
    +updateQuantity(cartId, productId, newQty)
    +deleteItem(cartId, productId)
}

class Inventory {
    <<Entity>>
    - inventoryId: int
    - productId: int
    - qtyAvailable: int
```

```
    - qtyReserved: int
    +checkBulkAvailability(items)
    +checkItemAvailability(productId, newQty)
  }

  ShoppingCartUI --> CartController : triggers loadCart / adjustQuantity / removeItem
  CartController --> ShoppingCart : queries cart session
  CartController --> CartItem : queries / updates / deletes items
  CartController --> Inventory : validates stock availability


sequenceDiagram
  actor Patient
  participant CartUI as ShoppingCartUI<br/><<Boundary>>
  participant CartCtrl as CartController<br/><<Control>>
  participant CartEnt as ShoppingCart<br/><<Entity>>
  participant CartItemEnt as CartItem<br/><<Entity>>
  participant InventoryEnt as Inventory<br/><<Entity>>

  %% NF Steps 1-4: Load and display cart
  Patient->>+CartUI: clickCartIcon()
  CartUI->>+CartCtrl: loadCart(patientId)

  %% Rule 4.1: All entity calls done and returned BEFORE entering alt [2a]
  CartCtrl->>+CartEnt: findByPatientId(patientId)
  CartEnt-->>-CartCtrl: cartData
```

```
CartCtrl->>+CartItemEnt: findByCartId(cartId)
CartItemEnt-->>-CartCtrl: items[]

alt [2a] Cart is empty — zero items found
   CartCtrl-->>CartUI: emptyCart
   CartUI-->>Patient: renderEmptyState()
else Items exist in cart
   %% Rule 4.1: checkBulkAvailability call+return both inside else — no boundary crossing
   CartCtrl->>+InventoryEnt: checkBulkAvailability(items)
   InventoryEnt-->>-CartCtrl: stockResult

   alt [3a] One or more items exceed available stock
      CartCtrl-->>CartUI: stockWarning
      CartUI-->>Patient: renderCartWithStockWarning()
   else All items have sufficient stock
      CartCtrl-->>CartUI: cartData
      CartUI-->>Patient: renderCart(cartData)
   end
end
%% Rule 4.2: CartCtrl and CartUI activated before OUTER alt → deactivate after OUTER end
deactivate CartCtrl
deactivate CartUI

%% [4a] Alt Flow: Adjust quantity — Rule 4.4: separate opt block
opt [4a] User adjusts item quantity via + or - button
```

```
    Patient->>+CartUI: clickAdjustQuantity(productId, newQty)
    CartUI->>+CartCtrl: adjustQuantity(cartId, productId, newQty)

    %% Rule 4.1: Check stock and return before alt [3a]
    CartCtrl->>+InventoryEnt: checkItemAvailability(productId, newQty)
    InventoryEnt-->>-CartCtrl: stockResult

    alt [3a] Requested quantity exceeds current stock
        CartCtrl-->>CartUI: stockExceeded
        CartUI-->>Patient: renderOutOfStockError()
    else Stock is sufficient
        CartCtrl->>+CartItemEnt: updateQuantity(cartId, productId, newQty)
        CartItemEnt-->>-CartCtrl: updated
        CartCtrl-->>CartUI: updatedCartData
        CartUI-->>Patient: renderQuantityUpdated()
    end
    %% Rule 4.2: CartCtrl and CartUI activated before alt → deactivate after end
    deactivate CartCtrl
    deactivate CartUI
end

%% [4b] Alt Flow: Remove item — Rule 4.4: separate opt block
opt [4b] User clicks Remove icon on a product row
    Patient->>+CartUI: clickRemove(productId)
    CartUI->>+CartCtrl: removeItem(cartId, productId)
    CartCtrl->>+CartItemEnt: deleteItem(cartId, productId)
```

```
    CartItemEnt-->>-CartCtrl: deleted
    CartCtrl-->>-CartUI: updatedCartData
    CartUI-->>-Patient: renderCartRecalculated()
  end

  %% NF Step 5: Proceed to checkout
  Patient->>+CartUI: clickProceedToCheckout()
  CartUI->>+CartCtrl: redirectToCheckout(cartData)
  CartCtrl-->>-CartUI: handoffToUC21
  deactivate CartUI
```

UC - 21: Order Medicine

```
classDiagram
  direction TB

  class CheckoutUI {
    <<Boundary>>
    +clickProceedToCheckout()
    +renderOutOfStockError()
    +renderCheckoutScreen(checkoutData)
    +selectDeliveryAddress(addressId)
    +uploadPrescription(file, issueDate)
    +clickUseEPrescription()
    +renderEPrescriptionList(ePrescriptions)
    +selectEPrescription(documentId)
    +autoFillIssueDate()
```

```
    +clickPlaceOrder()
    +showExpiredPrescriptionError()
    +showMissingPrescriptionError()
    +renderOrderSuccess(orderId)
    +redirectToPayment()
}

class OrderController {
    <<Control>>
    +validateStockAndLoadCheckout(cartData)
    +loadEPrescriptions(patientId)
    +placeOrder(orderData)
}

class Inventory {
    <<Entity>>
    - inventoryId: int
    - productId: int
    - qtyAvailable: int
    - qtyReserved: int
    +validateAllCartItems(cartItems)
    +reserveStock(cartItems)
}

class CartItem {
    <<Entity>>
```

```
    - cartItemId: int
    - cartId: int
    - productId: int
    - quantity: int
    - addedAt: DateTime
    +findByCartId(cartId)
    +clearCart(cartId)
}

class ProductMedicine {
    <<Entity>>
    - productId: int
    - productName: String
    - productImageUrl: String
    - unitPrice: Decimal
    - isRxRequired: Boolean
    +checkRxItems(productIds)
}

class PrescriptionDocument {
    <<Entity>>
    - documentId: int
    - patientId: int
    - fileUrl: String
    - issueDate: Date
    - isEPrescription: Boolean
```

```
    +findValidByPatientId(patientId)
    +savePrescriptionDocument(file, issueDate, patientId)
}

class MedicineOrder {
    <<Entity>>
    - orderId: int
    - patientId: int
    - addressId: int
    - documentId: int
    - pharmacistId: int
    - subtotalAmount: Decimal
    - shippingFee: Decimal
    - totalAmount: Decimal
    - status: String
    - cancellationReason: String
    - createdAt: DateTime
    +createOrder(orderData, status)
}

class OrderLineItem {
    <<Entity>>
    - lineItemId: int
    - orderId: int
    - productId: int
    - quantity: int
```

```
    - unitPriceAtPurchase: Decimal
    +saveLineItems(orderId, cartItems)
  }

  CheckoutUI --> OrderController : triggers validateStock / loadEPrescriptions / placeOrder
  OrderController --> Inventory : validates stock / reserves items
  OrderController --> CartItem : queries items / clears cart
  OrderController --> ProductMedicine : checks Rx flags
  OrderController --> PrescriptionDocument : queries / saves prescription
  OrderController --> MedicineOrder : creates order
  OrderController --> OrderLineItem : saves line items with locked price

sequenceDiagram
  actor Patient
  participant CheckoutUI as CheckoutUI<br/><<Boundary>>
  participant OrderCtrl as OrderController<br/><<Control>>
  participant InventoryEnt as Inventory<br/><<Entity>>
  participant CartItemEnt as CartItem<br/><<Entity>>
  participant ProductMedEnt as ProductMedicine<br/><<Entity>>
  participant PrescDocEnt as PrescriptionDocument<br/><<Entity>>
  participant OrderEnt as MedicineOrder<br/><<Entity>>
  participant LineItemEnt as OrderLineItem<br/><<Entity>>

  %% NF Steps 1-4: Stock validation and checkout screen load
  Patient->>+CheckoutUI: clickProceedToCheckout()
  CheckoutUI->>+OrderCtrl: validateStockAndLoadCheckout(cartData)
```

```
%% Rule 4.1: All entity calls return BEFORE entering alt [2a]
OrderCtrl->>+InventoryEnt: validateAllCartItems(cartItems)
InventoryEnt-->>-OrderCtrl: stockResult

alt [2a] One or more items exceed current stock
   OrderCtrl-->>CheckoutUI: stockError
   CheckoutUI-->>Patient: renderOutOfStockError()
else Stock sufficient — Normal Flow continues
   OrderCtrl->>+CartItemEnt: findByCartId(cartId)
   CartItemEnt-->>-OrderCtrl: items[]
   OrderCtrl->>+ProductMedEnt: checkRxItems(productIds)
   ProductMedEnt-->>-OrderCtrl: rxFlags
   OrderCtrl-->>CheckoutUI: checkoutData
   CheckoutUI-->>Patient: renderCheckoutScreen(checkoutData)
end
%% Rule 4.2: OrderCtrl and CheckoutUI activated before alt → deactivate after end
deactivate OrderCtrl
deactivate CheckoutUI

%% NF Step 5: Select delivery address
Patient->>+CheckoutUI: selectDeliveryAddress(addressId)
CheckoutUI-->>-Patient: renderCheckoutScreen(checkoutData)

%% [6a] Alt Flow: Use E-Prescription — Rule 4.4: separate opt block
opt [6a] User clicks Use E-Prescription
```

```
    Patient->>+CheckoutUI: clickUseEPrescription()
    CheckoutUI->>+OrderCtrl: loadEPrescriptions(patientId)
    OrderCtrl->>+PrescDocEnt: findValidByPatientId(patientId)
    PrescDocEnt-->>-OrderCtrl: ePrescriptions[]
    OrderCtrl-->>-CheckoutUI: ePrescriptions[]
    CheckoutUI-->>Patient: renderEPrescriptionList(ePrescriptions)
    Patient->>CheckoutUI: selectEPrescription(documentId)
    CheckoutUI-->>-Patient: autoFillIssueDate()
end

%% NF Step 6: Upload physical prescription (skipped if [3a] no Rx items)
opt Rx items present — physical upload path
    Patient->>+CheckoutUI: uploadPrescription(file, issueDate)
    CheckoutUI-->>-Patient: renderCheckoutScreen(checkoutData)
end

%% NF Steps 8-9: Place Order with in-memory validation
Patient->>+CheckoutUI: clickPlaceOrder()
CheckoutUI->>+OrderCtrl: placeOrder(orderData)

%% Rule 4.1: [7a] and [8a] are in-memory validations — no entity call needed before alt
alt [7a] Issue date is older than 30 days from today
    OrderCtrl-->>CheckoutUI: prescriptionExpired
    CheckoutUI-->>Patient: showExpiredPrescriptionError()
else [8a] Rx items present but no prescription uploaded or selected
    OrderCtrl-->>CheckoutUI: prescriptionMissing
```

```
    CheckoutUI-->>Patient: showMissingPrescriptionError()
else All validations passed — finalize order
    %% Rule 4.1: Entity calls inside else — all call+return contained within this branch
    OrderCtrl->>+PrescDocEnt: savePrescriptionDocument(file, issueDate, patientId)
    PrescDocEnt-->>-OrderCtrl: documentId
    OrderCtrl->>+InventoryEnt: reserveStock(cartItems)
    InventoryEnt-->>-OrderCtrl: reserved
    OrderCtrl->>+OrderEnt: createOrder(orderData, status)
    OrderEnt-->>-OrderCtrl: orderId
    OrderCtrl->>+LineItemEnt: saveLineItems(orderId, cartItems)
    LineItemEnt-->>-OrderCtrl: saved
    OrderCtrl->>+CartItemEnt: clearCart(cartId)
    CartItemEnt-->>-OrderCtrl: cleared

    %% Branch on Rx vs Non-Rx final routing
    alt hasRxItems — status: PendingPrescriptionVerification
        OrderCtrl-->>CheckoutUI: orderCreated(orderId)
        CheckoutUI-->>Patient: renderOrderSuccess(orderId)
    else No Rx — status: PendingPayment
        OrderCtrl-->>CheckoutUI: redirectToPaymentData
        CheckoutUI-->>Patient: redirectToPayment()
    end
end
%% Rule 4.2: OrderCtrl and CheckoutUI activated before outer alt → deactivate after end
deactivate OrderCtrl
deactivate CheckoutUI
```

UC - 23: Verify Prescription

```
classDiagram
  direction TB

  class PrescriptionVerifyUI {
    <<Boundary>>
    +clickVerifyPrescription(orderId)
    +renderFileError()
    +renderVerificationWorkspace(verifyData)
    +clickApprovePrescription()
    +showConcurrencyError()
    +showApproveSuccess()
    +redirectToDashboard()
    +clickRejectPrescription()
  }

  class RejectionModalUI {
    <<Boundary>>
    +renderRejectionModal()
    +selectRejectionReason(reason)
    +clickConfirmRejection()
    +showConcurrencyError()
```

```
    +showRejectSuccess()
}

class VerifyPrescriptionController {
    <<Control>>
    +loadVerificationWorkspace(orderId)
    +approvePrescription(orderId)
    +notifyPatientApproved(patientId)
    +rejectPrescription(orderId, reason)
    +notifyPatientRejected(patientId, reason)
}

class MedicineOrder {
    <<Entity>>
    - orderId: int
    - patientId: int
    - documentId: int
    - pharmacistId: int
    - subtotalAmount: Decimal
    - shippingFee: Decimal
    - totalAmount: Decimal
    - status: String
    - cancellationReason: String
    - createdAt: DateTime
    +findByIdWithDetails(orderId)
    +checkCurrentStatus(orderId)
```

```
    +updateStatusPendingPayment(orderId)
    +updateStatusCancelled(orderId, reason)
  }

  class OrderTracking {
    <<Entity>>
    - trackingId: int
    - orderId: int
    - statusMilestone: String
    - updatedBy: int
    - createdAt: DateTime
    +logMilestone(orderId, status, pharmacistId)
  }

  PrescriptionVerifyUI --> VerifyPrescriptionController : triggers loadVerificationWorkspace /
approvePrescription
  PrescriptionVerifyUI --> RejectionModalUI : renders rejection modal
  RejectionModalUI --> VerifyPrescriptionController : submits rejectPrescription
  VerifyPrescriptionController --> MedicineOrder : queries / updates order status
  VerifyPrescriptionController --> OrderTracking : logs status milestone

sequenceDiagram
  actor Pharmacist
  participant VerifyUI as PrescriptionVerifyUI<br/><<Boundary>>
  participant RejectModalUI as RejectionModalUI<br/><<Boundary>>
  participant VerifyCtrl as VerifyPrescriptionController<br/><<Control>>
```

```
participant OrderEnt as MedicineOrder<br/><<Entity>>
participant TrackingEnt as OrderTracking<br/><<Entity>>

%% NF Steps 1-3: Load Verification Workspace
Pharmacist->>+VerifyUI: clickVerifyPrescription(orderId)
VerifyUI->>+VerifyCtrl: loadVerificationWorkspace(orderId)

%% Rule 4.1: Entity call and return BEFORE alt [2a]
VerifyCtrl->>+OrderEnt: findByIdWithDetails(orderId)
OrderEnt-->>-VerifyCtrl: orderData

alt [2a] Prescription file missing or corrupted
   VerifyCtrl-->>VerifyUI: fileError
   VerifyUI-->>Pharmacist: renderFileError()
else File loaded successfully — Normal Flow continues
   VerifyCtrl-->>VerifyUI: verifyData
   VerifyUI-->>Pharmacist: renderVerificationWorkspace(verifyData)
end
%% Rule 4.2: VerifyCtrl and VerifyUI activated before alt → deactivate after end
deactivate VerifyCtrl
deactivate VerifyUI

%% NF Steps 5-9: Approve Prescription
Pharmacist->>+VerifyUI: clickApprovePrescription()
VerifyUI->>+VerifyCtrl: approvePrescription(orderId)
```

```
%% Rule 4.1: checkCurrentStatus call and return BEFORE alt [5b]
VerifyCtrl->>+OrderEnt: checkCurrentStatus(orderId)
OrderEnt-->>-VerifyCtrl: currentStatus

alt [5b] Order already processed concurrently
   VerifyCtrl-->>VerifyUI: concurrencyError
   VerifyUI-->>Pharmacist: showConcurrencyError()
   VerifyUI-->>Pharmacist: redirectToDashboard()
else Status valid — proceed with approval
   VerifyCtrl->>+OrderEnt: updateStatusPendingPayment(orderId)
   OrderEnt-->>-VerifyCtrl: updated
   VerifyCtrl->>+TrackingEnt: logMilestone(orderId, "PendingPayment", pharmacistId)
   TrackingEnt-->>-VerifyCtrl: logged
   VerifyCtrl->>VerifyCtrl: notifyPatientApproved(patientId)
   VerifyCtrl-->>VerifyUI: approved
   VerifyUI-->>Pharmacist: showApproveSuccess()
   VerifyUI-->>Pharmacist: redirectToDashboard()
end
%% Rule 4.2: VerifyCtrl and VerifyUI activated before alt → deactivate after end
deactivate VerifyCtrl
deactivate VerifyUI

%% [5a] Alt Flow: Reject Prescription — Rule 4.4: separate opt block
opt [5a] Pharmacist clicks Reject Prescription instead
   Pharmacist->>+VerifyUI: clickRejectPrescription()
   VerifyUI->>+RejectModalUI: renderRejectionModal()
```

```
RejectModalUI-->>Pharmacist: renderRejectionModal()

Pharmacist->>RejectModalUI: selectRejectionReason(reason)
RejectModalUI-->>Pharmacist: renderRejectionModal()

Pharmacist->>RejectModalUI: clickConfirmRejection()
RejectModalUI->>+VerifyCtrl: rejectPrescription(orderId, reason)

%% Rule 4.1: Entity call and return BEFORE inner alt [5b]
VerifyCtrl->>+OrderEnt: checkCurrentStatus(orderId)
OrderEnt-->>-VerifyCtrl: currentStatus

alt [5b] Order already processed concurrently
  VerifyCtrl-->>RejectModalUI: concurrencyError
  RejectModalUI-->>Pharmacist: showConcurrencyError()
else Status valid — proceed with rejection
  VerifyCtrl->>+OrderEnt: updateStatusCancelled(orderId, reason)
  OrderEnt-->>-VerifyCtrl: updated
  VerifyCtrl->>+TrackingEnt: logMilestone(orderId, "Cancelled", pharmacistId)
  TrackingEnt-->>-VerifyCtrl: logged
  VerifyCtrl->>VerifyCtrl: notifyPatientRejected(patientId, reason)
  VerifyCtrl-->>RejectModalUI: rejected
  RejectModalUI-->>Pharmacist: showRejectSuccess()
end
%% Rule 4.2: VerifyCtrl and RejectModalUI activated before inner alt → deactivate after inner end
```

```
    deactivate VerifyCtrl
    deactivate RejectModalUI

    %% VerifyUI still active (spawned inside opt) → handles final redirect
    VerifyUI-->>Pharmacist: redirectToDashboard()
    %% Rule 4.3: VerifyUI spawned inside opt → deactivate inside opt
    deactivate VerifyUI
  end
```

UC - 24: Track Order Status

```
classDiagram
  direction TB

  class OrderTrackingUI {
    <<Boundary>>
    +clickTrackOrder(orderId)
    +showAccessDenied()
    +redirectToDashboard()
    +renderTrackingScreen(orderData)
    +clickBackToOrders()
    +redirectToOrderHistory()
    +clickBuyAgain()
    +showAddedToCartSuccess()
    +redirectToCart()
```

```
    +clickRequestReturn()
    +renderReturnModal(returnPolicyData)
    +clickCancel()
    +closeModal()
}

class OrderTrackingController {
    <<Control>>
    +loadOrderTracking(orderId, patientId)
    +reorderItems(orderId, patientId)
    +loadReturnPolicy()
}

class MedicineOrder {
    <<Entity>>
    - orderId: int
    - patientId: int
    - addressId: int
    - documentId: int
    - pharmacistId: int
    - subtotalAmount: Decimal
    - shippingFee: Decimal
    - totalAmount: Decimal
    - status: String
    - cancellationReason: String
    - createdAt: DateTime
```

```
    +findByIdForPatient(orderId, patientId)
}

class OrderTracking {
    <<Entity>>
    - trackingId: int
    - orderId: int
    - statusMilestone: String
    - updatedBy: int
    - createdAt: DateTime
    +findByOrderId(orderId)
}

class CartItem {
    <<Entity>>
    - cartItemId: int
    - cartId: int
    - productId: int
    - quantity: int
    - addedAt: DateTime
    +duplicateItemsToCart(orderId, patientId)
}

OrderTrackingUI --> OrderTrackingController : triggers loadOrderTracking / reorderItems /
loadReturnPolicy
OrderTrackingController --> MedicineOrder : queries order by patient
```

```
    OrderTrackingController --> OrderTracking : retrieves timeline milestones
    OrderTrackingController --> CartItem : duplicates items into active cart



sequenceDiagram
    actor Patient
    participant TrackingUI as OrderTrackingUI<br/><<Boundary>>
    participant TrackingCtrl as OrderTrackingController<br/><<Control>>
    participant OrderEnt as MedicineOrder<br/><<Entity>>
    participant TrackingEnt as OrderTracking<br/><<Entity>>
    participant CartItemEnt as CartItem<br/><<Entity>>

    %% NF Steps 1-3: Load Order Tracking Screen
    Patient->>+TrackingUI: clickTrackOrder(orderId)
    TrackingUI->>+TrackingCtrl: loadOrderTracking(orderId, patientId)

    %% Rule 4.1: All entity calls complete and return BEFORE alt [2a]
    TrackingCtrl->>+OrderEnt: findByIdForPatient(orderId, patientId)
    OrderEnt-->>-TrackingCtrl: orderData

    TrackingCtrl->>+TrackingEnt: findByOrderId(orderId)
    TrackingEnt-->>-TrackingCtrl: milestones[]

    alt [2a] Order ID not linked to authenticated patient
        TrackingCtrl-->>TrackingUI: accessDenied
        TrackingUI-->>Patient: showAccessDenied()
```

```
    TrackingUI-->>Patient: redirectToDashboard()
else Order belongs to patient — Normal Flow continues
    TrackingCtrl-->>TrackingUI: orderData
    TrackingUI-->>Patient: renderTrackingScreen(orderData)
end
%% Rule 4.2: TrackingCtrl and TrackingUI activated before alt → deactivate after end
deactivate TrackingCtrl
deactivate TrackingUI

%% [3a] Alt Flow: Buy Again — Rule 4.4: separate opt block
opt [3a] Patient clicks Buy Again
    Patient->>+TrackingUI: clickBuyAgain()
    TrackingUI->>+TrackingCtrl: reorderItems(orderId, patientId)
    TrackingCtrl->>+CartItemEnt: duplicateItemsToCart(orderId, patientId)
    CartItemEnt-->>-TrackingCtrl: itemsDuplicated
    TrackingCtrl-->>-TrackingUI: itemsAdded
    TrackingUI-->>Patient: showAddedToCartSuccess()
    TrackingUI-->>-Patient: redirectToCart()
end

%% [3b] Alt Flow: Request Return (only visible when status = Delivered)
%% Rule 4.4: separate opt block
opt [3b] Patient clicks Request Return (milestone = Delivered)
    Patient->>+TrackingUI: clickRequestReturn()
    TrackingUI->>+TrackingCtrl: loadReturnPolicy()
    TrackingCtrl-->>-TrackingUI: returnPolicyData
```

```
    TrackingUI-->>Patient: renderReturnModal(returnPolicyData)

    %% User reads policy and clicks Cancel
    Patient->>TrackingUI: clickCancel()
    TrackingUI-->>-Patient: closeModal()
  end

  %% NF Steps 5-6: Back to Orders
  Patient->>+TrackingUI: clickBackToOrders()
  TrackingUI-->>-Patient: redirectToOrderHistory()
```

UC - 25: Make Payment

```
classDiagram
  direction TB

  class PaymentUI {
    <<Boundary>>
    +renderPaymentMethodScreen(transactionData)
    +selectPaymentMethod(method)
    +clickProceedToPay()
    +redirectToPaymentGateway(token)
    +clickCancelPayment()
    +showCancelledMessage()
    +redirectToOriginatingModule()
    +renderPaymentError()
```

```
    +clickRetryPayment()
    +renderTimeoutError()
    +renderPaymentSuccess(receipt)
}

class PaymentController {
    <<Control>>
    +generatePaymentToken(transactionId, paymentMethod)
    +cancelPayment(transactionId)
    +receiveGatewayCallback(callbackData)
}

class TransactionLog {
    <<Entity>>
    - transactionId: int
    - patientId: int
    - appointmentId: int
    - orderId: int
    - paymentMethod: String
    - amount: Decimal
    - gatewayTxnRef: String
    - status: String
    - createdAt: DateTime
    +findByTransactionId(transactionId)
    +updateStatusPaid(transactionId, gatewayRef)
    +updateStatusFailed(transactionId)
```

```
    +updateStatusTimeout(transactionId)
  }

  class EscrowLedger {
    <<Entity>>
    - escrowId: int
    - doctorId: int
    - appointmentId: int
    - inboundTransactionId: int
    - amount: Decimal
    - status: String
    - createdAt: DateTime
    +createHeldEscrow(transactionId, appointmentId, doctorId, amount)
  }

  PaymentUI --> PaymentController : triggers generatePaymentToken / cancelPayment
  PaymentController --> TransactionLog : queries / updates transaction status
  PaymentController --> EscrowLedger : creates held escrow on consultation payment

sequenceDiagram
  actor Patient
  actor PaymentGateway as PaymentGateway
  participant PaymentUI as PaymentUI<br/><<Boundary>>
  participant PaymentCtrl as PaymentController<br/><<Control>>
  participant TransactionEnt as TransactionLog<br/><<Entity>>
  participant EscrowEnt as EscrowLedger<br/><<Entity>>
```

```
%% NF Steps 1-2: System redirect loads the payment screen
Note over Patient,PaymentCtrl: System redirect from UC-26/UC-27 with transactionId and amount

activate PaymentCtrl
PaymentCtrl->>+TransactionEnt: findByTransactionId(transactionId)
TransactionEnt-->>-PaymentCtrl: transactionData

PaymentCtrl->>+PaymentUI: renderPaymentMethodScreen(transactionData)
PaymentUI-->>Patient: renderPaymentMethodScreen(transactionData)
deactivate PaymentUI
deactivate PaymentCtrl

%% [3a] Alt Flow: Cancel Payment — Rule 4.4: separate opt block
opt [3a] Patient clicks Cancel Payment
  Patient->>+PaymentUI: clickCancelPayment()
  PaymentUI->>+PaymentCtrl: cancelPayment(transactionId)
  PaymentCtrl-->>-PaymentUI: cancelled
  PaymentUI-->>Patient: showCancelledMessage()
  PaymentUI-->>Patient: redirectToOriginatingModule()
  deactivate PaymentUI
end

%% NF Step 3: Patient selects method
Patient->>+PaymentUI: selectPaymentMethod(method)
```

```
PaymentUI-->>-Patient: renderPaymentMethodScreen(transactionData)

%% NF Step 3: Patient clicks Proceed to Pay
Patient->>+PaymentUI: clickProceedToPay()
PaymentUI->>+PaymentCtrl: generatePaymentToken(transactionId, paymentMethod)
PaymentCtrl-->>-PaymentUI: token
PaymentUI-->>-Patient: redirectToPaymentGateway(token)

%% NF Steps 5-6: Patient interacts directly on external gateway (PCI-DSS compliant)
Note over Patient,PaymentGateway: Patient inputs financial credentials on external Payment Gateway

%% NF Step 7: Gateway sends callback back to system
PaymentGateway->>+PaymentCtrl: receiveGatewayCallback(callbackData)

%% Rule 4.1: callbackData contains status — no entity call needed before alt
alt [7a] callbackData.status is Failed or Declined
    PaymentCtrl->>+TransactionEnt: updateStatusFailed(transactionId)
    TransactionEnt-->>-PaymentCtrl: updated
    PaymentCtrl->>+PaymentUI: renderPaymentError()
    PaymentUI-->>Patient: renderPaymentError()
    deactivate PaymentUI
else [7b] callbackData.status is Timeout
    PaymentCtrl->>+TransactionEnt: updateStatusTimeout(transactionId)
    TransactionEnt-->>-PaymentCtrl: updated
    PaymentCtrl->>+PaymentUI: renderTimeoutError()
```

```
    PaymentUI-->>Patient: renderTimeoutError()
    deactivate PaymentUI
    %% System automatically returns to step 2 — no user click required
    PaymentCtrl->>+PaymentUI: renderPaymentMethodScreen(transactionData)
    PaymentUI-->>Patient: renderPaymentMethodScreen(transactionData)
    deactivate PaymentUI
  else Success — NF Steps 8-10
    PaymentCtrl->>+TransactionEnt: updateStatusPaid(transactionId, gatewayRef)
    TransactionEnt-->>-PaymentCtrl: updated
    %% Escrow created only for consultation payments (appointmentId present in transaction)
    PaymentCtrl->>+EscrowEnt: createHeldEscrow(transactionId, appointmentId, doctorId,
amount)
    EscrowEnt-->>-PaymentCtrl: escrowCreated
    PaymentCtrl->>+PaymentUI: renderPaymentSuccess(receipt)
    PaymentUI-->>Patient: renderPaymentSuccess(receipt)
    deactivate PaymentUI
  end
  %% Rule 4.2: PaymentCtrl activated before alt (by gateway callback) → deactivate after end
  deactivate PaymentCtrl

  %% [7a Retry] Patient clicks Retry after payment failure — Rule 4.4: separate opt block
  opt [7a Retry] Patient clicks Retry Payment
    Patient->>+PaymentUI: clickRetryPayment()
    PaymentUI-->>Patient: renderPaymentMethodScreen(transactionData)
    deactivate PaymentUI
  end
```

UC - 26: Pay for Doctor Services

```
classDiagram
    direction TB

    class ConsultationPaymentUI {
        <<Boundary>>
        +showPaymentSuccess()
        +redirectToMedicalHistory()
        +showPaymentError()
        +redirectToSearchDoctor()
    }

    class ConsultationPaymentController {
        <<Control>>
        +generatePaymentPayload(appointmentId, doctorId, patientId, fee)
        +receivePaymentCallback(callbackData)
    }

    class Appointment {
        <<Entity>>
        - appointmentId: int
        - doctorId: int
        - patientId: int
        - slotId: int
```

```
    - status: String
    - cancellationReason: String
    +updateStatusConfirmed(appointmentId)
    +updateStatusCancelled(appointmentId)
}

class DoctorSchedule {
    <<Entity>>
    - slotId: int
    - doctorId: int
    - slotDate: Date
    - startTime: Time
    - endTime: Time
    - status: String
    +releaseSlot(slotId)
}

class EscrowLedger {
    <<Entity>>
    - escrowId: int
    - doctorId: int
    - appointmentId: int
    - inboundTransactionId: int
    - amount: Decimal
    - status: String
    - createdAt: DateTime
```

```
    +createHeldEscrow(transactionId, appointmentId, doctorId, amount)
  }

  ConsultationPaymentUI --> ConsultationPaymentController : receives callback / renders result
  ConsultationPaymentController --> Appointment : updates status to Confirmed / Cancelled
  ConsultationPaymentController --> DoctorSchedule : releases slot on payment failure
  ConsultationPaymentController --> EscrowLedger : creates held escrow on success

sequenceDiagram
  actor Patient
  participant ConsultPayUI as ConsultationPaymentUI<br/><<Boundary>>
  participant ConsultPayCtrl as ConsultationPaymentController<br/><<Control>>
  participant AppointmentEnt as Appointment<br/><<Entity>>
  participant ScheduleEnt as DoctorSchedule<br/><<Entity>>
  participant EscrowEnt as EscrowLedger<br/><<Entity>>

  %% NF Steps 1-3: Generate payload and delegate to UC-25
  Note over Patient,ConsultPayCtrl: Triggered by "Confirm and Pay" from UC-17 BookingReviewUI

  activate ConsultPayCtrl
  ConsultPayCtrl->>ConsultPayCtrl: generatePaymentPayload(appointmentId, doctorId, patientId, fee)
  Note over ConsultPayCtrl: Seamlessly redirects Patient to UC-25 (Make Payment)

  %% NF Steps 4-5: Wait for callback from UC-25
  Note over ConsultPayCtrl: System awaits synchronous callback from UC-25
```

```
ConsultPayCtrl->>ConsultPayCtrl: receivePaymentCallback(callbackData)

%% Rule 4.1: callbackData.status determines branch — no entity call needed before alt
alt [5a] Callback is Failed, Timeout, or Cancelled
    ConsultPayCtrl->>+ScheduleEnt: releaseSlot(slotId)
    ScheduleEnt-->>-ConsultPayCtrl: slotReleased
    ConsultPayCtrl->>+AppointmentEnt: updateStatusCancelled(appointmentId)
    AppointmentEnt-->>-ConsultPayCtrl: updated
    ConsultPayCtrl->>+ConsultPayUI: showPaymentError()
    ConsultPayUI-->>Patient: showPaymentError()
    ConsultPayUI-->>Patient: redirectToSearchDoctor()
    deactivate ConsultPayUI
else Callback is Success — NF Steps 6-9
    ConsultPayCtrl->>+AppointmentEnt: updateStatusConfirmed(appointmentId)
    AppointmentEnt-->>-ConsultPayCtrl: updated
    ConsultPayCtrl->>+EscrowEnt: createHeldEscrow(transactionId, appointmentId, doctorId,
amount)
    EscrowEnt-->>-ConsultPayCtrl: escrowCreated
    ConsultPayCtrl->>+ConsultPayUI: showPaymentSuccess()
    ConsultPayUI-->>Patient: showPaymentSuccess()
    ConsultPayUI-->>Patient: redirectToMedicalHistory()
    deactivate ConsultPayUI
end
%% Rule 4.2: ConsultPayCtrl activated before alt → deactivate after end
deactivate ConsultPayCtrl
```

UC - 27: Pay for Medicine

```
classDiagram
  direction TB

  class MedicinePaymentUI {
    <<Boundary>>
    +clickPayNow(orderId)
    +showPaymentSuccess()
    +redirectToTrackOrder()
    +showPaymentError()
  }

  class MedicinePaymentController {
    <<Control>>
    +generatePaymentPayload(orderId, patientId, totalAmount)
    +receivePaymentCallback(callbackData)
  }

  class MedicineOrder {
    <<Entity>>
    - orderId: int
    - patientId: int
    - addressId: int
    - documentId: int
    - subtotalAmount: Decimal
```

```
        - shippingFee: Decimal
        - totalAmount: Decimal
        - status: String
        - createdAt: DateTime
        +updateStatusProcessing(orderId)
    }

    class Inventory {
        <<Entity>>
        - inventoryId: int
        - productId: int
        - qtyAvailable: int
        - qtyReserved: int
        +finalizeDeduction(orderId)
    }

    MedicinePaymentUI --> MedicinePaymentController : triggers generatePaymentPayload / receives callback
    MedicinePaymentController --> MedicineOrder : updates status to Processing / retains Pending
    MedicinePaymentController --> Inventory : finalizes permanent stock deduction

sequenceDiagram
    actor Patient
    participant MedPayUI as MedicinePaymentUI<br/><<Boundary>>
    participant MedPayCtrl as MedicinePaymentController<br/><<Control>>
    participant OrderEnt as MedicineOrder<br/><<Entity>>
```

```
participant InventoryEnt as Inventory<br/><<Entity>>

%% NF Steps 1-3: Patient clicks Pay Now, generate payload, delegate to UC-25
Patient->>+MedPayUI: clickPayNow(orderId)
MedPayUI->>+MedPayCtrl: generatePaymentPayload(orderId, patientId, totalAmount)
MedPayCtrl-->>-MedPayUI: paymentPayload
deactivate MedPayUI
Note over MedPayCtrl: Seamlessly redirects Patient to UC-25 (Make Payment)

%% NF Steps 4-5: Wait for and receive callback from UC-25
Note over MedPayCtrl: System awaits synchronous callback from UC-25

%% FIX: Khởi động lại Controller để xử lý luồng Callback mới
activate MedPayCtrl
MedPayCtrl->>MedPayCtrl: receivePaymentCallback(callbackData)

%% Rule 4.1: callbackData.status determines branch — no entity call needed before alt
alt [5a] Callback is Failed, Timeout, or Cancelled
   %% Order retains "Pending Payment" — no status update entity call
   MedPayCtrl->>+MedPayUI: showPaymentError()
   MedPayUI-->>Patient: showPaymentError()
   MedPayUI-->>Patient: redirectToTrackOrder()
   deactivate MedPayUI
else Callback is Success — NF Steps 6-9
   MedPayCtrl->>+OrderEnt: updateStatusProcessing(orderId)
   OrderEnt-->>-MedPayCtrl: updated
```

```
    MedPayCtrl->>+InventoryEnt: finalizeDeduction(orderId)
    InventoryEnt-->>-MedPayCtrl: deducted
    MedPayCtrl->>+MedPayUI: showPaymentSuccess()
    MedPayUI-->>Patient: showPaymentSuccess()
    MedPayUI-->>Patient: redirectToTrackOrder()
    deactivate MedPayUI
  end
  %% Rule 4.2: MedPayCtrl activated before alt → deactivate after end
  deactivate MedPayCtrl
```

UC - 34: Process Provider Payout

```
classDiagram
  direction TB

  class PayoutDashboardUI {
    <<Boundary>>
    +clickPayoutManagement()
    +renderPayoutDashboard(payoutData)
    +inputSearchFilter(criteria)
    +renderFilteredPayouts(payoutData)
    +clickInitiatePayout(doctorId)
    +showCancelledMessage()
    +showSuccessMessage()
  }

  class PayoutConfirmationModalUI {
```

```
    <<Boundary>>
    +renderPayoutModal(payoutDetails)
    +clickConfirmTransfer()
    +showEscrowMismatchError()
    +showGatewayFailureError()
    +clickCancel()
}

class PayoutController {
    <<Control>>
    +loadPayoutDashboard()
    +filterPayouts(criteria)
    +loadPayoutModal(doctorId)
    +verifyEscrowBalance(doctorId, requestedAmount)
    +submitPayoutToGateway(payoutPayload)
    +receivePayoutCallback(callbackData)
    +notifyDoctorByEmail(doctorId, receipt)
}

class EscrowLedger {
    <<Entity>>
    - escrowId: int
    - doctorId: int
    - appointmentId: int
    - inboundTransactionId: int
    - amount: Decimal
```

```
    - status: String
    - createdAt: DateTime
    +findReleasedByDoctorId(doctorId)
    +verifyBalance(doctorId, requestedAmount)
    +clearEscrowHold(escrowIds)
}

class PayoutRequest {
    <<Entity>>
    - payoutId: int
    - doctorId: int
    - adminId: int
    - totalAmount: Decimal
    - receivingBankName: String
    - receivingAccountNumber: String
    - gatewayPayoutRef: String
    - status: String
    - processedAt: DateTime
    +createPayoutRequest(doctorId, adminId, amount, bankDetails)
    +updateStatusPaidOut(payoutId, gatewayRef)
    +updateStatusFailed(payoutId)
}

class EscrowPayoutItem {
    <<Entity>>
    - payoutId: int
```

```
    - escrowId: int
    +saveBatchItems(payoutId, escrowIds)
  }

  PayoutDashboardUI --> PayoutController : triggers loadPayoutDashboard / filterPayouts / loadPayoutModal
  PayoutDashboardUI --> PayoutConfirmationModalUI : renders payout confirmation modal
  PayoutConfirmationModalUI --> PayoutController : submits verifyEscrowBalance / submitPayoutToGateway
  PayoutController --> EscrowLedger : queries released earnings / verifies balance / clears hold
  PayoutController --> PayoutRequest : creates payout record / updates status
  PayoutController --> EscrowPayoutItem : saves batch linkage items

sequenceDiagram
  actor Admin
  actor PaymentGateway as PaymentGateway
  participant DashboardUI as PayoutDashboardUI<br/><<Boundary>>
  participant ModalUI as PayoutConfirmationModalUI<br/><<Boundary>>
  participant PayoutCtrl as PayoutController<br/><<Control>>
  participant EscrowEnt as EscrowLedger<br/><<Entity>>
  participant PayoutReqEnt as PayoutRequest<br/><<Entity>>
  participant PayoutItemEnt as EscrowPayoutItem<br/><<Entity>>

  %% NF Steps 1-2: Load Payout Dashboard
  Admin->>+DashboardUI: clickPayoutManagement()
  DashboardUI->>+PayoutCtrl: loadPayoutDashboard()
  PayoutCtrl->>+EscrowEnt: findReleasedByDoctorId(doctorId)
```

```
EscrowEnt-->>-PayoutCtrl: releasedEscrows[]
PayoutCtrl-->>-DashboardUI: payoutData
DashboardUI-->>-Admin: renderPayoutDashboard(payoutData)

%% [2a] Alt Flow: Filter payouts — Rule 4.4: separate opt block
opt [2a] Admin filters by status or searches by doctor name
   Admin->>+DashboardUI: inputSearchFilter(criteria)
   DashboardUI->>+PayoutCtrl: filterPayouts(criteria)
   PayoutCtrl->>+EscrowEnt: findReleasedByDoctorId(doctorId)
   EscrowEnt-->>-PayoutCtrl: filteredData
   PayoutCtrl-->>-DashboardUI: filteredData
   DashboardUI-->>-Admin: renderFilteredPayouts(payoutData)
end

%% NF Steps 3-5: Initiate payout → load modal
Admin->>+DashboardUI: clickInitiatePayout(doctorId)
DashboardUI->>+PayoutCtrl: loadPayoutModal(doctorId)
PayoutCtrl-->>-DashboardUI: payoutDetails
DashboardUI->>+ModalUI: renderPayoutModal(payoutDetails)
ModalUI-->>Admin: renderPayoutModal(payoutDetails)
deactivate DashboardUI

%% [5a] Alt Flow: Cancel payout initiation — Rule 4.4: separate opt block
opt [5a] Admin clicks Cancel on modal
   Admin->>ModalUI: clickCancel()
   %% FIX: Bỏ dấu '-' để tránh lỗi deactivate sớm
```

```
    ModalUI-->>Admin: renderPayoutModal(payoutDetails)
    Note over ModalUI: Modal closed
    %% DashboardUI re-activated to show cancel message
    activate DashboardUI
    DashboardUI-->>Admin: showCancelledMessage()
    deactivate DashboardUI
  end

  %% NF Step 6: Admin clicks Confirm Transfer
  Admin->>ModalUI: clickConfirmTransfer()
  ModalUI->>+PayoutCtrl: verifyEscrowBalance(doctorId, requestedAmount)

  %% Rule 4.1: Entity call and return BEFORE alt [7a]
  PayoutCtrl->>+EscrowEnt: verifyBalance(doctorId, requestedAmount)
  EscrowEnt-->>-PayoutCtrl: verificationResult

  alt [7a] Escrow balance mismatch — requested exceeds available
    PayoutCtrl-->>ModalUI: escrowMismatch
    ModalUI-->>Admin: showEscrowMismatchError()
  else Balance verified — NF Steps 8-11
    PayoutCtrl->>+PayoutReqEnt: createPayoutRequest(doctorId, adminId, amount,
bankDetails)
    PayoutReqEnt-->>-PayoutCtrl: payoutId
    PayoutCtrl->>PayoutCtrl: submitPayoutToGateway(payoutPayload)
    Note over PayoutCtrl: Transmits payload to Payment Gateway
```

```
    PaymentGateway->>PayoutCtrl: receivePayoutCallback(callbackData)

    %% Rule 4.1: callbackData.status determines branch — no entity call needed
    alt [9a] Callback is Failed, Rejected, or Invalid Account
        PayoutCtrl->>+PayoutReqEnt: updateStatusFailed(payoutId)
        PayoutReqEnt-->>-PayoutCtrl: updated
        PayoutCtrl-->>ModalUI: gatewayFailure
        ModalUI-->>Admin: showGatewayFailureError()
    else Callback is Success
        PayoutCtrl->>+PayoutReqEnt: updateStatusPaidOut(payoutId, gatewayRef)
        PayoutReqEnt-->>-PayoutCtrl: updated
        PayoutCtrl->>+EscrowEnt: clearEscrowHold(escrowIds)
        EscrowEnt-->>-PayoutCtrl: cleared
        PayoutCtrl->>+PayoutItemEnt: saveBatchItems(payoutId, escrowIds)
        PayoutItemEnt-->>-PayoutCtrl: saved
        PayoutCtrl->>PayoutCtrl: notifyDoctorByEmail(doctorId, receipt)
        PayoutCtrl-->>ModalUI: payoutComplete
        ModalUI-->>Admin: renderPayoutModal(payoutDetails)
        %% Rule 4.3: DashboardUI spawned inside else → deactivate inside else
        activate DashboardUI
        DashboardUI-->>Admin: showSuccessMessage()
        deactivate DashboardUI
    end
end
%% Rule 4.2: PayoutCtrl and ModalUI activated before outer alt → deactivate after end
deactivate PayoutCtrl
```

```
    deactivate ModalUI
```

UC - 35: Process Refund

```
classDiagram
    direction TB

    class RefundDashboardUI {
        <<Boundary>>
        +clickRefundManagement()
        +renderRefundDashboard(refundData)
        +inputSearchFilter(criteria)
        +renderFilteredRefunds(refundData)
        +clickInitiateRefund(transactionId)
        +showCancelledMessage()
        +showSuccessMessage()
    }

    class RefundConfirmationModalUI {
        <<Boundary>>
        +renderRefundModal(refundDetails)
        +inputRefundReason(reason)
        +clickConfirmRefund()
        +showAmountExceededError()
        +showGatewayFailureError()
        +clickCancel()
    }
```

```
class RefundController {
    <<Control>>
    +loadRefundDashboard()
    +filterRefunds(criteria)
    +loadRefundModal(transactionId)
    +submitRefundToGateway(refundPayload)
    +receiveRefundCallback(callbackData)
    +notifyPatientByEmail(patientId, refundDetails)
}

class TransactionLog {
    <<Entity>>
    - transactionId: int
    - patientId: int
    - appointmentId: int
    - orderId: int
    - paymentMethod: String
    - amount: Decimal
    - gatewayTxnRef: String
    - status: String
    - createdAt: DateTime
    +findPendingRefunds()
    +searchByFilter(criteria)
    +findByIdForRefund(transactionId)
    +verifyRefundAmount(transactionId, requestedAmount)
```

```
    +updateStatusRefunded(transactionId)
}

class RefundRequest {
    <<Entity>>
    - refundId: int
    - originalTransactionId: int
    - adminId: int
    - refundAmount: Decimal
    - reason: String
    - gatewayRefundRef: String
    - status: String
    - createdAt: DateTime
    - processedAt: DateTime
    +createRefundRecord(transactionId, adminId, amount, reason)
    +updateStatusRefunded(refundId, gatewayRef)
    +updateStatusFailed(refundId)
}

class EscrowLedger {
    <<Entity>>
    - escrowId: int
    - doctorId: int
    - appointmentId: int
    - inboundTransactionId: int
    - amount: Decimal
```

```
    - status: String
    - createdAt: DateTime
    +updateStatusRefunded(transactionId)
  }

  RefundDashboardUI --> RefundController : triggers loadRefundDashboard / filterRefunds / loadRefundModal
  RefundDashboardUI --> RefundConfirmationModalUI : renders refund confirmation modal
  RefundConfirmationModalUI --> RefundController : submits refund payload / receives result
  RefundController --> TransactionLog : queries pending refunds / verifies amount / updates status
  RefundController --> RefundRequest : creates refund record / updates status
  RefundController --> EscrowLedger : updates escrow to Refunded on consultation cancellations

sequenceDiagram
  actor Admin
  actor PaymentGateway as PaymentGateway
  participant DashboardUI as RefundDashboardUI<br/><<Boundary>>
  participant ModalUI as RefundConfirmationModalUI<br/><<Boundary>>
  participant RefundCtrl as RefundController<br/><<Control>>
  participant TransactionEnt as TransactionLog<br/><<Entity>>
  participant RefundReqEnt as RefundRequest<br/><<Entity>>
  participant EscrowEnt as EscrowLedger<br/><<Entity>>

  %% NF Steps 1-2: Load Refund Dashboard
  Admin->>+DashboardUI: clickRefundManagement()
```

```
DashboardUI->>+RefundCtrl: loadRefundDashboard()
RefundCtrl->>+TransactionEnt: findPendingRefunds()
TransactionEnt-->>-RefundCtrl: pendingRefunds[]
RefundCtrl-->>-DashboardUI: refundData
DashboardUI-->>-Admin: renderRefundDashboard(refundData)

%% [2a filter] Alt Flow: Filter refunds — Rule 4.4: separate opt block
opt [2a] Admin filters by status or search keyword
   Admin->>+DashboardUI: inputSearchFilter(criteria)
   DashboardUI->>+RefundCtrl: filterRefunds(criteria)
   RefundCtrl->>+TransactionEnt: searchByFilter(criteria)
   TransactionEnt-->>-RefundCtrl: filteredData
   RefundCtrl-->>-DashboardUI: filteredData
   DashboardUI-->>-Admin: renderFilteredRefunds(refundData)
end

%% NF Steps 3-4: Initiate Refund → load modal
Admin->>+DashboardUI: clickInitiateRefund(transactionId)
DashboardUI->>+RefundCtrl: loadRefundModal(transactionId)
RefundCtrl->>+TransactionEnt: findByIdForRefund(transactionId)
TransactionEnt-->>-RefundCtrl: refundDetails
RefundCtrl-->>-DashboardUI: refundDetails
DashboardUI->>+ModalUI: renderRefundModal(refundDetails)
ModalUI-->>Admin: renderRefundModal(refundDetails)
deactivate DashboardUI
```

```
%% [5a] Alt Flow: Cancel refund initiation — Rule 4.4: separate opt block
opt [5a] Admin clicks Cancel on modal
   Admin->>ModalUI: clickCancel()
   ModalUI-->>Admin: renderRefundModal(refundDetails)
   activate DashboardUI
   DashboardUI-->>Admin: showCancelledMessage()
   deactivate DashboardUI
end

%% NF Step 5: Admin fills reason and clicks Confirm Refund
Admin->>ModalUI: inputRefundReason(reason)
ModalUI-->>Admin: renderRefundModal(refundDetails)

Admin->>ModalUI: clickConfirmRefund()
ModalUI->>+RefundCtrl: submitRefundToGateway(refundPayload)

%% Rule 4.1: verifyRefundAmount call and return BEFORE alt [7a]
RefundCtrl->>+TransactionEnt: verifyRefundAmount(transactionId, requestedAmount)
TransactionEnt-->>-RefundCtrl: verificationResult

alt [7a] Requested refund amount exceeds original transaction value
   RefundCtrl-->>ModalUI: amountExceeded
   ModalUI-->>Admin: showAmountExceededError()
else Amount valid — NF Steps 6-10
   %% Rule 4.1: createRefundRecord call+return inside else — no boundary crossing
```

```
    RefundCtrl->>+RefundReqEnt: createRefundRecord(transactionId, adminId, amount,
reason)
    RefundReqEnt-->>-RefundCtrl: refundId

    Note over RefundCtrl: Transmits refund payload to Payment Gateway
    PaymentGateway->>RefundCtrl: receiveRefundCallback(callbackData)

    %% Rule 4.1: callbackData.status determines next branch — no entity call needed
    alt [8a] Callback is Failed, Rejected, or Expired
      RefundCtrl->>+RefundReqEnt: updateStatusFailed(refundId)
      RefundReqEnt-->>-RefundCtrl: updated
      RefundCtrl-->>ModalUI: gatewayFailure
      ModalUI-->>Admin: showGatewayFailureError()
    else Callback is Success — NF Steps 8-11
      RefundCtrl->>+TransactionEnt: updateStatusRefunded(transactionId)
      TransactionEnt-->>-RefundCtrl: updated
      RefundCtrl->>+RefundReqEnt: updateStatusRefunded(refundId, gatewayRef)
      RefundReqEnt-->>-RefundCtrl: updated
      RefundCtrl->>+EscrowEnt: updateStatusRefunded(transactionId)
      EscrowEnt-->>-RefundCtrl: updated
      RefundCtrl->>RefundCtrl: notifyPatientByEmail(patientId, refundDetails)
      RefundCtrl-->>ModalUI: refundComplete
      ModalUI-->>Admin: renderRefundModal(refundDetails)
      %% Rule 4.3: DashboardUI spawned inside else → deactivate inside else
      activate DashboardUI
      DashboardUI-->>Admin: showSuccessMessage()
```

```
        deactivate DashboardUI
      end
    end
    %% Rule 4.2: RefundCtrl and ModalUI activated before outer alt → deactivate after end
    deactivate RefundCtrl
    deactivate ModalUI
```

UC - 12: View Insurance Guide

```
classDiagram
    direction TB

    class InsuranceDashboardUI {
      <<Boundary>>
      +clickInsuranceGuide()
      +renderEmptyState()
      +renderDashboard(packages)
      +inputSearchFilter(criteria)
      +renderFilteredPackages(packages)
      +clickAddToCompare(packageId)
      +enableCompareButton()
      +clickComparePackages()
      +renderComparisonTable(comparisonData)
      +clickCloseComparison()
      +clickViewDetails(packageId)
```

```
}

class InsuranceDetailUI {
    <<Boundary>>
    +renderDetailScreen(packageDetail)
    +showLoadingError()
    +clickRetry(packageId)
    +clickPurchaseThisPlan(packageId)
    +redirectToPurchase(packageId)
    +clickBackToList()
}

class InsuranceController {
    <<Control>>
    +loadDashboard()
    +filterPackages(criteria)
    +loadPackageComparison(packageIds)
    +loadPackageDetail(packageId)
}

class InsurancePackage {
    <<Entity>>
    - packageId: int
    - providerId: int
    - packageName: String
    - packageCategory: String
```

```
        - annualPremium: Decimal
        - minAge: int
        - maxAge: int
        - inpatientLimit: Decimal
        - outpatientLimit: Decimal
        - maternityAllowance: Decimal
        - dentalCare: Decimal
        - exclusionsText: String
        +findAllActive()
        +searchByFilter(criteria)
        +findByIds(packageIds)
        +findById(packageId)
    }

    InsuranceDashboardUI --> InsuranceController : triggers loadDashboard / filterPackages /
loadPackageComparison
    InsuranceDashboardUI --> InsuranceDetailUI : renders detail screen
    InsuranceDetailUI --> InsuranceController : triggers loadPackageDetail / retry
    InsuranceController --> InsurancePackage : queries active packages / filters / fetches detail

sequenceDiagram
    actor Patient
    participant DashboardUI as InsuranceDashboardUI<br/><<Boundary>>
    participant DetailUI as InsuranceDetailUI<br/><<Boundary>>
    participant InsuranceCtrl as InsuranceController<br/><<Control>>
    participant PackageEnt as InsurancePackage<br/><<Entity>>
```

```
%% NF Steps 1-3: Load Insurance Dashboard
Patient->>+DashboardUI: clickInsuranceGuide()
DashboardUI->>+InsuranceCtrl: loadDashboard()

%% Rule 4.1: Entity call and return BEFORE alt [2a]
InsuranceCtrl->>+PackageEnt: findAllActive()
PackageEnt-->>-InsuranceCtrl: packages[]

alt [2a] No active insurance packages found
   InsuranceCtrl-->>DashboardUI: emptyResult
   DashboardUI-->>Patient: renderEmptyState()
else Packages found — Normal Flow continues
   InsuranceCtrl-->>DashboardUI: packages[]
   DashboardUI-->>Patient: renderDashboard(packages)
end
%% Rule 4.2: InsuranceCtrl and DashboardUI activated before alt → deactivate after end
deactivate InsuranceCtrl
deactivate DashboardUI

%% [3a] Alt Flow: Filter packages — Rule 4.4: separate opt block
opt [3a] Patient applies filter or search keyword
   Patient->>+DashboardUI: inputSearchFilter(criteria)
   DashboardUI->>+InsuranceCtrl: filterPackages(criteria)
   InsuranceCtrl->>+PackageEnt: searchByFilter(criteria)
   PackageEnt-->>-InsuranceCtrl: filteredPackages[]
```

```
    InsuranceCtrl-->>-DashboardUI: filteredPackages[]
    DashboardUI-->>-Patient: renderFilteredPackages(packages)
end

%% [3b] Alt Flow: Compare packages — Rule 4.4: separate opt block
opt [3b] Patient selects 2-3 packages and clicks Compare
    Patient->>+DashboardUI: clickAddToCompare(packageId)
    DashboardUI-->>Patient: enableCompareButton()

    Patient->>DashboardUI: clickComparePackages()
    DashboardUI->>+InsuranceCtrl: loadPackageComparison(packageIds)
    InsuranceCtrl->>+PackageEnt: findByIds(packageIds)
    PackageEnt-->>-InsuranceCtrl: comparisonData[]
    InsuranceCtrl-->>-DashboardUI: comparisonData[]
    DashboardUI-->>Patient: renderComparisonTable(comparisonData)

    Patient->>DashboardUI: clickCloseComparison()
    DashboardUI-->>-Patient: renderDashboard(packages)
end

%% NF Steps 4-6: View Package Detail
Patient->>+DashboardUI: clickViewDetails(packageId)
DashboardUI->>+InsuranceCtrl: loadPackageDetail(packageId)

%% Rule 4.1: Entity call and return BEFORE alt [5a]
InsuranceCtrl->>+PackageEnt: findById(packageId)
```

```
PackageEnt-->>-InsuranceCtrl: packageDetail

alt [5a] Database timeout or connection error
   InsuranceCtrl-->>DashboardUI: loadingError
   DashboardUI->>+DetailUI: showLoadingError()
   DetailUI-->>Patient: showLoadingError()

   Patient->>DetailUI: clickRetry(packageId)
   DetailUI->>InsuranceCtrl: loadPackageDetail(packageId)
   InsuranceCtrl->>+PackageEnt: findById(packageId)
   PackageEnt-->>-InsuranceCtrl: packageDetail
   InsuranceCtrl-->>DetailUI: packageDetail
   DetailUI-->>Patient: renderDetailScreen(packageDetail)
   deactivate DetailUI
else Detail loaded successfully
   InsuranceCtrl-->>DashboardUI: packageDetail
   DashboardUI->>+DetailUI: renderDetailScreen(packageDetail)
   DetailUI-->>Patient: renderDetailScreen(packageDetail)
   deactivate DetailUI
end
%% Rule 4.2: InsuranceCtrl and DashboardUI activated before alt → deactivate after end
deactivate InsuranceCtrl
deactivate DashboardUI

%% [6a] Alt Flow: Navigate to Purchase — Rule 4.4: separate opt block
opt [6a] Patient clicks Purchase this Plan
```

```
    Patient->>+DetailUI: clickPurchaseThisPlan(packageId)
    DetailUI-->>-Patient: redirectToPurchase(packageId)
  end

  %% NF: Back to List
  Patient->>+DetailUI: clickBackToList()
  DetailUI-->>-Patient: renderDashboard(packages)
```

UC - 13: Purchase Insurance

```
classDiagram
  direction TB

  class PurchaseInsuranceUI {
    <<Boundary>>
    +clickPurchaseThisPlan(packageId)
    +renderBeneficiaryForm(packageData)
    +clickChooseFromSavedProfiles()
    +renderSavedProfiles(profiles)
    +selectSavedProfile(profileId)
    +autoFillBeneficiaryForm(profileData)
    +fillBeneficiaryDetails(formData)
    +clickProceedToPayment()
    +showAgeLimitError()
    +renderOrderSummary(summaryData)
    +checkTermsAndConditions()
```

```
    +clickPayNow()
    +redirectToPaymentGateway()
    +showPaymentCancelledToast()
    +showPaymentFailedScreen()
    +clickRetryPayment()
    +renderPaymentSuccessScreen(certificateData)
}

class PurchaseInsuranceController {
    <<Control>>
    +loadBeneficiaryForm(packageId)
    +loadSavedProfiles(patientId)
    +validateAgeEligibility(dob, packageId)
    +generateOrderSummary(formData, packageId)
    +receivePaymentCallback(callbackData)
    +generateECertificate(policyRecordId)
    +notifyPatientByEmail(patientId, certificateUrl)
}

class InsurancePackage {
    <<Entity>>
    - packageId: int
    - providerId: int
    - packageName: String
    - packageCategory: String
    - annualPremium: Decimal
```

```
    - minAge: int
    - maxAge: int
    - inpatientLimit: Decimal
    - outpatientLimit: Decimal
    - maternityAllowance: Decimal
    - dentalCare: Decimal
    - exclusionsText: String
    +findById(packageId)
}

class DependentProfile {
    <<Entity>>
    - dependentId: int
    - patientId: int
    - fullName: String
    - dateOfBirth: Date
    - relationship: String
    +findByPatientId(patientId)
}

class InsurancePolicyRecord {
    <<Entity>>
    - policyRecordId: int
    - packageId: int
    - purchaserPatientId: int
    - transactionId: int
```

```
    - beneficiaryFullName: String
    - beneficiaryDob: Date
    - beneficiaryNationalId: String
    - beneficiaryGender: String
    - relationship: String
    - eCertificateUrl: String
    - status: String
    +createPolicyRecord(packageId, patientId, transactionId, beneficiarySnapshot)
  }

  class TransactionLog {
    <<Entity>>
    - transactionId: int
    - patientId: int
    - amount: Decimal
    - paymentMethod: String
    - gatewayTxnRef: String
    - status: String
    - createdAt: DateTime
    +createTransactionRecord(patientId, packageId, amount, gatewayRef)
  }

  PurchaseInsuranceUI --> PurchaseInsuranceController : triggers loadBeneficiaryForm / validateAge / generateSummary
  PurchaseInsuranceController --> InsurancePackage : queries package requirements / age limits
  PurchaseInsuranceController --> DependentProfile : retrieves saved family profiles
```

```
    PurchaseInsuranceController --> TransactionLog : creates transaction record on success
    PurchaseInsuranceController --> InsurancePolicyRecord : creates policy record with data snapshot

sequenceDiagram
    actor Patient
    actor PaymentGateway as PaymentGateway
    participant PurchaseUI as PurchaseInsuranceUI<br/><<Boundary>>
    participant PurchaseCtrl as PurchaseInsuranceController<br/><<Control>>
    participant PackageEnt as InsurancePackage<br/><<Entity>>
    participant DependentEnt as DependentProfile<br/><<Entity>>
    participant TransactionEnt as TransactionLog<br/><<Entity>>
    participant PolicyEnt as InsurancePolicyRecord<br/><<Entity>>

    %% NF Steps 1-2: Load Beneficiary Form
    Patient->>+PurchaseUI: clickPurchaseThisPlan(packageId)
    PurchaseUI->>+PurchaseCtrl: loadBeneficiaryForm(packageId)
    PurchaseCtrl->>+PackageEnt: findById(packageId)
    PackageEnt-->>-PurchaseCtrl: packageData
    PurchaseCtrl-->>-PurchaseUI: packageData
    PurchaseUI-->>-Patient: renderBeneficiaryForm(packageData)

    %% [3a] Alt Flow: Choose from Saved Profiles — Rule 4.4: separate opt block
    opt [3a] Patient clicks Choose from Saved Profiles
        Patient->>+PurchaseUI: clickChooseFromSavedProfiles()
        PurchaseUI->>+PurchaseCtrl: loadSavedProfiles(patientId)
```

```
    PurchaseCtrl->>+DependentEnt: findByPatientId(patientId)
    DependentEnt-->>-PurchaseCtrl: profiles[]
    PurchaseCtrl-->>-PurchaseUI: profiles[]
    PurchaseUI-->>Patient: renderSavedProfiles(profiles)
    Patient->>PurchaseUI: selectSavedProfile(profileId)
    PurchaseUI-->>-Patient: autoFillBeneficiaryForm(profileData)
end

%% NF Step 3: Patient fills form manually or continues with autofill
Patient->>+PurchaseUI: fillBeneficiaryDetails(formData)
PurchaseUI-->>-Patient: renderBeneficiaryForm(packageData)

%% NF Steps 4-5: Validate age and render order summary
Patient->>+PurchaseUI: clickProceedToPayment()
PurchaseUI->>+PurchaseCtrl: validateAgeEligibility(dob, packageId)

%% Rule 4.1: in-memory age calculation — no entity call needed before alt
alt [5a] DOB falls outside package age limits (BR-19)
    PurchaseCtrl-->>PurchaseUI: ageLimitViolation
    PurchaseUI-->>Patient: showAgeLimitError()
else Age is eligible — Normal Flow continues
    PurchaseCtrl->>PurchaseCtrl: generateOrderSummary(formData, packageId)
    PurchaseCtrl-->>PurchaseUI: summaryData
    PurchaseUI-->>Patient: renderOrderSummary(summaryData)
end
%% Rule 4.2: PurchaseCtrl and PurchaseUI activated before alt → deactivate after end
```

```
deactivate PurchaseCtrl
deactivate PurchaseUI

%% NF Steps 6-7: Accept terms and redirect to Payment Gateway
Patient->>+PurchaseUI: checkTermsAndConditions()
PurchaseUI-->>-Patient: renderOrderSummary(summaryData)

Patient->>+PurchaseUI: clickPayNow()
PurchaseUI-->>-Patient: redirectToPaymentGateway()

Note over Patient,PaymentGateway: Patient inputs payment details on external Payment Gateway

%% [9a] Alt Flow: Cancel on Gateway — Rule 4.4: separate opt block
opt [9a] Patient clicks Cancel Transaction on Payment Gateway
  PaymentGateway->>+PurchaseUI: showPaymentCancelledToast()
  PurchaseUI-->>-Patient: showPaymentCancelledToast()
end

%% NF Steps 9-11: Receive callback and finalize
PaymentGateway->>+PurchaseCtrl: receivePaymentCallback(callbackData)

%% Rule 4.1: callbackData.status determines branch — no entity call needed before alt
alt [10a] Callback is Failed, Declined, or Timeout
  PurchaseCtrl->>+PurchaseUI: showPaymentFailedScreen()
  PurchaseUI-->>Patient: showPaymentFailedScreen()
```

```
    Patient->>PurchaseUI: clickRetryPayment()


    %% FIX: Bỏ dấu '-' ở đây để dùng lệnh deactivate ở dưới (Rule 4.3)
    PurchaseUI-->>Patient: redirectToPaymentGateway()
    deactivate PurchaseUI


  else Callback is Success — NF Steps 10-11
    %% Rule 4.1: All entity calls inside else — no boundary crossing
    PurchaseCtrl->>+TransactionEnt: createTransactionRecord(patientId, packageId, amount, gatewayRef)
    TransactionEnt-->>-PurchaseCtrl: transactionId
    PurchaseCtrl->>+PolicyEnt: createPolicyRecord(packageId, patientId, transactionId, beneficiarySnapshot)
    PolicyEnt-->>-PurchaseCtrl: policyRecordId
    PurchaseCtrl->>PurchaseCtrl: generateECertificate(policyRecordId)
    PurchaseCtrl->>PurchaseCtrl: notifyPatientByEmail(patientId, certificateUrl)
    PurchaseCtrl->>+PurchaseUI: renderPaymentSuccessScreen(certificateData)
    PurchaseUI-->>Patient: renderPaymentSuccessScreen(certificateData)
    deactivate PurchaseUI
  end
  %% Rule 4.2: PurchaseCtrl activated before alt → deactivate after end
  deactivate PurchaseCtrl
```

UC - 29: Request Ambulance

```
classDiagram
  direction TB
```

```
class AmbulanceDispatchUI {
    <<Boundary>>
    +clickEmergencyButton()
    +promptGpsPermission()
    +renderDispatchScreen(gpsData, profileData)
    +editPreciseAddress(address)
    +clickConfirmDispatch(formData)
    +renderTrackingScreen(dispatchData)
    +displaySuccessMessage()
    +displayCancelledMessage()
    +renderGpsErrorMessage()
    +disableConfirmButton()
    +displayServiceUnavailableMessage()
}

class AmbulanceController {
    <<Control>>
    +requestGpsPermission()
    +retrieveGpsAndProfile()
    +submitDispatchRequest(formData)
    +receiveDispatchConfirmation(response)
    +triggerPriorityNotifications(dispatchId)
    +handleServiceUnavailable()
}

class NotificationService {
```

```
    <<Control>>
    +sendPriorityAlert(patientId, dispatchDetails)
}

class AmbulanceServiceAPI {
    <<Control>>
    +sendDispatchPayload(payload)
}

class AmbulanceDispatch {
    <<Entity>>
    - dispatchId: int
    - patientId: int
    - gpsLatitude: decimal
    - gpsLongitude: decimal
    - preciseAddress: string
    - contactPhone: string
    - situationDescription: string
    - partnerDispatchId: string
    - driverName: string
    - etaMinutes: int
    - status: string
    - requestedAt: datetime
    +create(formData, status)
    +updateWithPartnerData(dispatchId, partnerId, driverName, eta)
}
```

```
    AmbulanceDispatchUI --> AmbulanceController : triggers / submits via
    AmbulanceController --> AmbulanceDispatchUI : renders / updates
    AmbulanceController --> AmbulanceDispatch : creates / updates
    AmbulanceController --> AmbulanceServiceAPI : calls dispatch payload via
    AmbulanceController --> NotificationService : triggers priority alert via
    AmbulanceServiceAPI --> AmbulanceController : returns dispatch confirmation


sequenceDiagram
    participant Patient as Patient
    participant UI as AmbulanceDispatchUI<br/><<Boundary>>
    participant Controller as AmbulanceController<br/><<Control>>
    participant Entity as AmbulanceDispatch<br/><<Entity>>
    participant API as AmbulanceServiceAPI<br/><<Control>>
    participant Notif as NotificationService<br/><<Control>>

    Patient->>+UI: clickEmergencyButton()
    UI->>+Controller: requestGpsPermission()
    Controller-->>-UI: promptGpsPermission()
    UI-->>Patient: promptGpsPermission()

    alt GPS permission granted
        Patient->>+UI: grantPermission()
        UI->>+Controller: retrieveGpsAndProfile()
        Controller-->>-UI: renderDispatchScreen(gpsData, profileData)
```

```
UI-->>-Patient: renderDispatchScreen(gpsData, profileData)

opt Edit auto-filled address (4a)
   Patient->>+UI: editPreciseAddress(address)
   UI-->>-Patient: renderDispatchScreen(gpsData, profileData)
end

opt Cancel request (5a)
   Patient->>+UI: clickCancelRequest()
   UI->>+Controller: handleCancelRequest()
   Controller-->>-UI: displayCancelledMessage()
   UI-->>-Patient: displayCancelledMessage()
end

Patient->>+UI: clickConfirmDispatch(formData)
UI->>+Controller: submitDispatchRequest(formData)
Controller->>+Entity: create(formData, "PENDING")
Entity-->>-Controller: dispatchRecord
Controller->>+API: sendDispatchPayload(payload)

alt Ambulance service available (7)
   API-->>-Controller: confirmationResponse(partnerId, driverName, eta)
   Controller->>+Entity: updateWithPartnerData(dispatchId, partnerId, driverName, eta)
   Entity-->>-Controller: updatedRecord
   Controller->>+Notif: sendPriorityAlert(patientId, dispatchDetails)
   Notif-->>-Controller: alertSent
```

```
        Controller-->>-UI: renderTrackingScreen(dispatchData)
        UI-->>Patient: renderTrackingScreen(dispatchData)
        UI-->>-Patient: displaySuccessMessage()

    else Ambulance service unavailable (7a)
        Controller->>Controller: handleServiceUnavailable()
        Controller-->>UI: displayServiceUnavailableMessage()
        UI-->>Patient: displayServiceUnavailableMessage()
    end

else GPS permission denied (3a)
    Patient->>+UI: denyPermission()
    UI->>+Controller: handleGpsPermissionDenied()
    Controller-->>-UI: renderGpsErrorMessage()
    UI->>UI: disableConfirmButton()
    UI-->>-Patient: renderGpsErrorMessage()
end
```

UC - 07 View Health A-Z

```
classDiagram
    direction TB

    class HealthDirectoryUI {
        <<Boundary>>
        +navigateToHealthAZ()
        +renderDirectory(directoryData)
```

```
    +selectLetter(letter)
    +displayEmptyState(message)
    +renderDiseaseList(diseaseList)
    +selectDisease(diseaseId)
}

class DiseaseDetailUI {
    <<Boundary>>
    +renderDiseaseDetails(diseaseDetailDto)
    +clickRelatedTopic(topicId)
}

class ErrorUI {
    <<Boundary>>
    +display404Error(message)
}

class HealthDirectoryController {
    <<Control>>
    +initDirectory()
    +loadDiseasesByLetter(letter)
    +viewDiseaseDetails(diseaseId)
}

class DiseaseContentService {
    <<Control>>
```

```
    +getInitialDirectory() List
    +getDiseasesByLetter(letter) List
    +getDiseaseDetails(diseaseId) Object
    -appendDisclaimer(diseaseData) Object
}

class DiseaseDictionary {
    <<Entity>>
    -disease_id int
    -author_id int
    -disease_title String
    -symptoms_text String
    -causes String
    -treatments String
    -prevention String
    -status String
    +findPublishedIndex() List
    +findByStartingLetterAndStatus(letter, status) List
    +findByIdAndStatus(diseaseId, status) Object
}

HealthDirectoryUI --> HealthDirectoryController : triggers / submits via
DiseaseDetailUI --> HealthDirectoryController : triggers / submits via
HealthDirectoryController --> DiseaseContentService : delegates to
HealthDirectoryController --> DiseaseDetailUI : renders
HealthDirectoryController --> ErrorUI : renders
```

```
    DiseaseContentService --> DiseaseDictionary : queries

sequenceDiagram
    actor User as Patient/Caregiver
    participant DirUI as <<Boundary>><br>HealthDirectoryUI
    participant DetUI as <<Boundary>><br>DiseaseDetailUI
    participant ErrUI as <<Boundary>><br>ErrorUI
    participant Ctrl as <<Control>><br>HealthDirectoryController
    participant Svc as <<Control>><br>DiseaseContentService
    participant DB as <<Entity>><br>DiseaseDictionary

    %% Normal Flow: Init Directory
    User->>+DirUI: navigateToHealthAZ()
    DirUI->>+Ctrl: initDirectory()
    Ctrl->>+Svc: getInitialDirectory()

    %% Rule 4.1: Call Entity right before returning
    Svc->>+DB: findPublishedIndex()
    DB-->>-Svc: directoryData

    Svc-->>-Ctrl: directoryData
    Ctrl-->>-DirUI: renderDirectory(directoryData)
    DirUI-->>-User: display alphabetical index and list

    %% Normal Flow: Select Letter & Filter
    User->>+DirUI: selectLetter(letter)
```

```
DirUI->>+Ctrl: loadDiseasesByLetter(letter)
Ctrl->>+Svc: getDiseasesByLetter(letter)

%% Rule 4.1: Call Entity and deactivate immediately before alt block
Svc->>+DB: findByStartingLetterAndStatus(letter, "Published")
Note over DB: Apply BR-22: Fetch only published content
DB-->>-Svc: diseaseList
Svc-->>-Ctrl: diseaseList

alt [Exc 4a: No content available for selected letter]
    Ctrl-->>DirUI: displayEmptyState("There are currently no health topics listed under this letter.")
    DirUI-->>User: display empty state message
else [Normal Flow: Content available]
    Ctrl-->>DirUI: renderDiseaseList(diseaseList)
    DirUI-->>User: display list of diseases for selected letter
end
%% Rule 4.2: Deactivate objects opened before the alt block outside the end
deactivate Ctrl
deactivate DirUI

%% Normal Flow: View Disease Details
User->>+DirUI: selectDisease(diseaseId)
DirUI->>+Ctrl: viewDiseaseDetails(diseaseId)
Ctrl->>+Svc: getDiseaseDetails(diseaseId)
```

```
%% Rule 4.1: Call Entity and deactivate before alt
Svc->>+DB: findByIdAndStatus(diseaseId, "Published")
DB-->>-Svc: diseaseData
Note over Svc: Apply BR-24: Append mandatory medical disclaimer
Svc-->>-Ctrl: diseaseDetailDto

alt [Exc 6a: Article not found or unpublished]
    %% Rule 4.3: ErrUI is spawned inside else branch, so deactivate inside
    Ctrl->>+ErrUI: display404Error("The requested health information is currently unavailable or under review by our medical experts.")
    ErrUI-->>User: render 404 error page
    deactivate ErrUI

    Ctrl-->>DirUI: fail()
else [Normal Flow: Article retrieved successfully]
    %% Rule 4.3: DetUI is spawned inside else branch, so deactivate inside
    Ctrl->>+DetUI: renderDiseaseDetails(diseaseDetailDto)
    DetUI-->>User: display detailed article with medical disclaimer
    deactivate DetUI

    Ctrl-->>DirUI: success()
end
%% Rule 4.2: Deactivate objects opened before the alt block outside the end
deactivate Ctrl
deactivate DirUI
```

```
    %% Rule 4.4: Split independent flows using opt block
    opt [Alt 7a: Navigate to related topics]
        User->>+DetUI: clickRelatedTopic(topicId)
        DetUI->>+Ctrl: viewDiseaseDetails(topicId)
        Ctrl->>+Svc: getDiseaseDetails(topicId)

        %% Rule 4.1: Call Entity and deactivate
        Svc->>+DB: findByIdAndStatus(topicId, "Published")
        DB-->>-Svc: topicData
        Note over Svc: Apply BR-24: Append disclaimer
        Svc-->>-Ctrl: topicDetailDto

        Ctrl-->>DetUI: renderDiseaseDetails(topicDetailDto)
        DetUI-->>User: redirect and display related article

        Ctrl-->>-DetUI: routed()
        deactivate DetUI
    end
```

UC - 08: Search Health A-Z

```
classDiagram
    direction TB

    class HealthSearchUI {
        <<Boundary>>
```

```
    +inputKeywordAndSearch(keyword)
    +applyFilters(filters)
    +selectArticle(articleId)
    +displayValidationMessage(message)
    +displayEmptyState(message)
    +renderSearchResults(articleList)
}

class DiseaseDetailUI {
    <<Boundary>>
    +renderDiseaseDetails(articleId)
}

class HealthSearchController {
    <<Control>>
    +searchHealthArticles(keyword)
    +filterSearchResults(keyword, filters)
    +routeToArticleDetail(articleId)
}

class SearchService {
    <<Control>>
    +validateInput(keyword) bool
    +findArticles(keyword, filters) List
}
```

```
    class DiseaseDictionary {
        <<Entity>>
        -disease_id int
        -author_id int
        -disease_title String
        -symptoms_text String
        -causes String
        -treatments String
        -prevention String
        -status String
        +searchByKeywordAndStatus(keyword, status) List
        +searchByKeywordFiltersAndStatus(keyword, filters, status) List
    }

    HealthSearchUI --> HealthSearchController : triggers / submits via
    DiseaseDetailUI --> HealthSearchController : triggered by
    HealthSearchController --> SearchService : delegates to
    HealthSearchController --> DiseaseDetailUI : renders
    SearchService --> DiseaseDictionary : queries


sequenceDiagram
    actor User as Patient/Caregiver
    participant SearchUI as HealthSearchUI<br/><<Boundary>>
    participant DetailUI as DiseaseDetailUI<br/><<Boundary>>
    participant Ctrl as HealthSearchController<br/><<Control>>
```

```
participant Svc as SearchService<br/><<Control>>
participant DB as DiseaseDictionary<br/><<Entity>>

User->>+SearchUI: inputKeywordAndSearch(keyword)
SearchUI->>+Ctrl: searchHealthArticles(keyword)

Ctrl->>+Svc: validateInput(keyword)
Svc-->>-Ctrl: isValid

alt [Exc 4a: Invalid search input]
    Ctrl-->>SearchUI: displayValidationMessage("Please enter at least 2 valid characters to search.")
    SearchUI-->>User: show validation message
else [Normal Flow: Valid input]

    %% Rule 4.1: Call Entity right before checking results in an alt block
    Ctrl->>+Svc: findArticles(keyword, null)
    Svc->>+DB: searchByKeywordAndStatus(keyword, "Published")
    Note over DB: Apply BR-22: Fetch only published content
    DB-->>-Svc: articleList
    Svc-->>-Ctrl: articleList

    alt [Exc 5a: No results found]
        Ctrl-->>SearchUI: displayEmptyState("No results found for '[Keyword]'...")
        SearchUI-->>User: display empty state graphic
    else [Normal Flow: Results found]
```

```
        Ctrl-->>SearchUI: renderSearchResults(articleList)
        SearchUI-->>User: display paginated list of results
    end

end
%% Rule 4.2: Deactivate objects opened before the alt block outside the end
deactivate Ctrl
deactivate SearchUI

%% Rule 4.4: Independent alternate flow wrapped in its own opt block
opt [Alt 6a: Filter search results]
    User->>+SearchUI: applyFilters(filters)
    SearchUI->>+Ctrl: filterSearchResults(keyword, filters)

    Ctrl->>+Svc: findArticles(keyword, filters)
    Svc->>+DB: searchByKeywordFiltersAndStatus(keyword, filters, "Published")
    DB-->>-Svc: refinedArticleList
    Svc-->>-Ctrl: refinedArticleList

    Ctrl-->>SearchUI: renderSearchResults(refinedArticleList)
    SearchUI-->>User: display refined list of search results

    %% Rule 4.3 (applied to opt): Deactivate at the end of the branch
    deactivate Ctrl
    deactivate SearchUI
end
```

```
    %% Rule 4.4: Independent sequential flow wrapped in its own opt block
    opt [Normal Flow Step 7-8: Select specific article]
        User->>+SearchUI: selectArticle(articleId)
        SearchUI->>+Ctrl: routeToArticleDetail(articleId)

        Ctrl->>+DetailUI: renderDiseaseDetails(articleId)
        DetailUI-->>User: redirect to UC-07 detailed article view

        %% Rule 4.3: DetailUI is spawned inside this block, so it is deactivated here
        deactivate DetailUI

        Ctrl-->>SearchUI: routed()
        SearchUI-->>User: close search context

        deactivate Ctrl
        deactivate SearchUI
    end
```

UC - 09: Check Symptoms

```
classDiagram
    direction TB

    class SymptomCheckerUI {
        <<Boundary>>
```

```
    +navigateToModule()
    +renderInputInterface(symptomList)
    +selectSymptoms(symptomIds)
    +clickAnalyze()
    +displayInlineError(message)
    +displayWarning(message)
    +renderAssessmentReport(finalReport)
}

class SymptomCheckerController {
    <<Control>>
    +initSymptomChecker()
    +analyzeSymptoms(symptomIds)
}

class TriageEngineService {
    <<Control>>
    +getAllSymptoms() List
    +validateInput(symptomIds) bool
    +calculateMatchingConditions(symptomIds) List
    +generateRecommendations(matchedConditions) List
    +appendMedicalDisclaimer(reportData) Object
}

class Symptom {
    <<Entity>>
```

```
    -symptom_id int
    -symptom_name String
    +findAllActive() List
}

class DiseaseSymptom {
    <<Entity>>
    -disease_id int
    -symptom_id int
    -matching_probability decimal
    +findBySymptomIds(symptomIds) List
}

class DiseaseDictionary {
    <<Entity>>
    -disease_id int
    -author_id int
    -disease_title String
    -symptoms_text String
    -causes String
    -treatments String
    -prevention String
    -status String
    +findPublishedByIds(diseaseIds, status) List
}
```

```
    SymptomCheckerUI --> SymptomCheckerController : triggers / submits via
    SymptomCheckerController --> SymptomCheckerUI : renders
    SymptomCheckerController --> TriageEngineService : delegates to
    TriageEngineService --> Symptom : queries
    TriageEngineService --> DiseaseSymptom : queries / computes probability
    TriageEngineService --> DiseaseDictionary : queries / checks status

sequenceDiagram
    actor User as Patient/Caregiver
    participant UI as SymptomCheckerUI<br/><<Boundary>>
    participant Ctrl as SymptomCheckerController<br/><<Control>>
    participant Svc as TriageEngineService<br/><<Control>>
    participant SymptomEnt as Symptom<br/><<Entity>>
    participant DiseaseSymEnt as DiseaseSymptom<br/><<Entity>>
    participant DiseaseEnt as DiseaseDictionary<br/><<Entity>>

    %% Initialization Flow
    User->>+UI: navigateToModule()
    UI->>+Ctrl: initSymptomChecker()
    Ctrl->>+Svc: getAllSymptoms()

    %% Rule 4.1 applied implicitly for standard data retrieval
    Svc->>+SymptomEnt: findAllActive()
    SymptomEnt-->>-Svc: symptomList

    Svc-->>-Ctrl: symptomList
```

```
Ctrl-->>-UI: renderInputInterface(symptomList)
UI-->>-User: display symptom input interface

%% Analysis Flow
User->>+UI: selectSymptoms(symptomIds)
UI-->>-User: updates selection state

User->>+UI: clickAnalyze()
UI->>+Ctrl: analyzeSymptoms(symptomIds)

Ctrl->>+Svc: validateInput(symptomIds)
Svc-->>-Ctrl: isValid

alt [Exc 4a: Incomplete input]
    Ctrl-->>UI: displayInlineError("Please select or enter at least one symptom to proceed.")
    UI-->>User: show inline validation error
else [Normal Flow: Valid input]

    %% Rule 4.1: Execute Entity queries and deactivate immediately before evaluating the results in an alt block
    Ctrl->>+Svc: calculateMatchingConditions(symptomIds)

    Svc->>+DiseaseSymEnt: findBySymptomIds(symptomIds)
    DiseaseSymEnt-->>-Svc: matchedData

    Svc->>+DiseaseEnt: findPublishedByIds(diseaseIds, "Published")
```

```
    Note over DiseaseEnt: Apply BR-22: Fetch only approved/published conditions
    DiseaseEnt-->>-Svc: diseaseList

    Svc-->>-Ctrl: matchedConditions

    alt [Exc 5a: Unmatched symptoms combination]
      Ctrl-->>UI: displayWarning("We couldn't find a specific medical match...")
      UI-->>User: show warning message & "Consult a Doctor" button
    else [Normal Flow: Match found]
      Ctrl->>+Svc: generateRecommendations(matchedConditions)
      Svc-->>-Ctrl: actions

      Ctrl->>+Svc: appendMedicalDisclaimer(reportData)
      Note over Svc: Apply BR-24: Append mandatory legal disclaimer
      Svc-->>-Ctrl: finalReport

      Ctrl-->>UI: renderAssessmentReport(finalReport)
      UI-->>User: display assessment, conditions, actions, and disclaimer
    end
  end
  %% Rule 4.2: Deactivate objects opened BEFORE the outermost alt block at the very end
  deactivate Ctrl
  deactivate UI
```

UC - 10: Health Calculator

classDiagram

```
direction TB

class HealthCalculatorUI {
    <<Boundary>>
    +navigateToModule()
    +renderInputForm()
    +inputData(gender, age, height, weight, activityLevel)
    +clickCalculate()
    +displayMissingFieldError(message)
    +displayBoundaryError(message)
    +renderResults(metricsResult, disclaimer)
    +clickSaveToProfile()
    +displayToast(message)
}

class CalculatorController {
    <<Control>>
    +initCalculator()
    +processCalculation(formData)
    +saveMetrics(patientId, formData, metricsResult)
}

class CalculatorService {
    <<Control>>
    +validateRequiredFields(formData) bool
    +validateBoundaries(formData) bool
```

```
    +computeMetrics(formData) Object
    +saveToHistory(patientId, formData, metricsResult) bool
}

class CalculatorHistory {
    <<Entity>>
    -calc_id int
    -patient_id int
    -biological_gender String
    -age int
    -height_cm decimal
    -weight_kg decimal
    -activity_level String
    -bmi_score decimal
    -bmi_classification String
    -bmr_kcal decimal
    -tdee_kcal decimal
    -saved_at datetime
    +createRecord(calcData) Object
}

HealthCalculatorUI --> CalculatorController : triggers / submits via
CalculatorController --> CalculatorService : delegates to
CalculatorController --> HealthCalculatorUI : renders
CalculatorService --> CalculatorHistory : creates / saves
```

```
sequenceDiagram
    actor User as Patient/Caregiver
    participant UI as HealthCalculatorUI<br/><<Boundary>>
    participant Ctrl as CalculatorController<br/><<Control>>
    participant Svc as CalculatorService<br/><<Control>>
    participant DB as CalculatorHistory<br/><<Entity>>

    %% Initialization
    User->>+UI: navigateToModule()
    UI->>+Ctrl: initCalculator()
    Ctrl-->>-UI: renderInputForm()
    UI-->>-User: display input form (Gender, Age, Height, Weight, Activity)

    %% Data Input
    User->>+UI: inputData(gender, age, height, weight, activityLevel)
    UI-->>-User: update input fields

    %% Calculation Flow
    User->>+UI: clickCalculate()
    UI->>+Ctrl: processCalculation(formData)

    %% Rule 4.1: Validate required fields right before branching
    Ctrl->>+Svc: validateRequiredFields(formData)
    Svc-->>-Ctrl: isComplete

    alt [Exc 5b: Missing mandatory fields]
```

```
    %% Rule 4.2: No (-) on return, deactivate outside block
    Ctrl-->>UI: displayMissingFieldError("This field is required.")
    UI-->>User: highlight empty fields
else [Normal Flow: Fields are complete]

    %% Rule 4.1: Validate boundaries right before branching
    Ctrl->>+Svc: validateBoundaries(formData)
    Note over Svc: Apply BR-25 bounds: Age 0-120, Height 40-250, Weight 2-400
    Svc-->>-Ctrl: isValidBounds

    alt [Exc 5a: Invalid input boundaries]
        %% Rule 4.2: No (-) on return, deactivate outside block
        Ctrl-->>UI: displayBoundaryError("Value is out of the acceptable medical range.")
        UI-->>User: highlight invalid fields in red & disable button
    else [Normal Flow: Valid boundaries]
        Ctrl->>+Svc: computeMetrics(formData)
        Note over Svc: Apply BR-26: Standard clinical formulas (BMI, BMR, TDEE)
        Svc-->>-Ctrl: metricsResult

        Note over Ctrl: Apply BR-24: Append medical disclaimer to results
        %% Rule 4.2: No (-) on return, deactivate outside block
        Ctrl-->>UI: renderResults(metricsResult, disclaimer)
        UI-->>User: display BMI, BMR, TDEE outputs & medical disclaimer
    end
end
%% Rule 4.2: Deactivate objects opened before the alt block
```

```
deactivate Ctrl
deactivate UI

%% Rule 4.4: Independent sequential flow wrapped in its own opt block
opt [Alt 8a: User saves metrics to profile]
  User->>+UI: clickSaveToProfile()
  UI->>+Ctrl: saveMetrics(patientId, formData, metricsResult)

  Ctrl->>+Svc: saveToHistory(patientId, formData, metricsResult)

  Svc->>+DB: createRecord(calcData)
  Note over DB: Apply BR-27: Overrides ephemerality rule on explicit save
  DB-->>-Svc: savedResult

  Svc-->>-Ctrl: saveStatus

  Ctrl-->>UI: displayToast("Health metrics saved successfully.")
  UI-->>User: display confirmation toast

  %% Rule 4.3 (applied to opt): Deactivate at the end of the branch
  deactivate Ctrl
  deactivate UI
end
```

UC - 11: View News & Community Programs

classDiagram

```
direction TB

class NewsDashboardUI {
    <<Boundary>>
    +navigateToNewsModule()
    +renderDashboard(dashboardData)
    +selectCategory(categoryId)
    +displayEmptyState(message)
    +renderFilteredContent(articleList)
    +selectArticle(articleId)
    +fail()
    +success()
}

class ArticleDetailUI {
    <<Boundary>>
    +renderDetailedView(articleData)
}

class Error404UI {
    <<Boundary>>
    +display404Page(message)
}

class NewsController {
    <<Control>>
```

```
    +loadDashboard()
    +filterByCategory(categoryId)
    +viewArticleDetails(articleId)
}

class NewsService {
    <<Control>>
    +getDashboardContent() Object
    +getPublishedByCategory(categoryId) List
    +getPublishedArticleDetails(articleId) Object
}

class CmsArticle {
    <<Entity>>
    -articleId int
    -categoryId int
    -authorId int
    -title String
    -coverImageUrl String
    -bodyContent String
    -status String
    -publishedAt datetime
    +findLatestPublished(limit) List
    +findPublishedByCategoryId(categoryId) List
    +findByIdAndStatus(articleId, status) Object
}
```

```
    class ContentCategory {
        <<Entity>>
        -categoryId int
        -categoryName String
        +findAllActive() List
    }

    NewsDashboardUI --> NewsController : triggers / submits via
    NewsController --> NewsService : delegates to
    NewsController --> ArticleDetailUI : renders
    NewsController --> Error404UI : renders
    NewsService --> CmsArticle : queries
    NewsService --> ContentCategory : queries

sequenceDiagram
    actor User as Patient/Caregiver
    participant DashUI as NewsDashboardUI<br/><<Boundary>>
    participant DetUI as ArticleDetailUI<br/><<Boundary>>
    participant ErrUI as Error404UI<br/><<Boundary>>
    participant Ctrl as NewsController<br/><<Control>>
    participant Svc as NewsService<br/><<Control>>
    participant ArtEnt as CmsArticle<br/><<Entity>>
    participant CatEnt as ContentCategory<br/><<Entity>>

    %% Normal Flow Step 1-2: Initialization
```

```
User->>+DashUI: navigateToNewsModule()
DashUI->>+Ctrl: loadDashboard()

Ctrl->>+Svc: getDashboardContent()

%% Rule 4.1: Implicitly applied for straight flows. Entities return before logic.
Svc->>+ArtEnt: findLatestPublished(limit=10)
Note over ArtEnt: Apply BR-22: Fetch only published articles
ArtEnt-->>-Svc: latestArticles

Svc->>+CatEnt: findAllActive()
CatEnt-->>-Svc: categories

Svc-->>-Ctrl: dashboardData
Ctrl-->>-DashUI: renderDashboard(dashboardData)
DashUI-->>-User: display Carousel, Recent, Upcoming, Categories

%% Rule 4.4: Independent Alternative Flow wrapped in opt block
opt [Alt 2a: Filter content by category]
   User->>+DashUI: selectCategory(categoryId)
   DashUI->>+Ctrl: filterByCategory(categoryId)

   Ctrl->>+Svc: getPublishedByCategory(categoryId)

   %% Rule 4.1: Call Entity and deactivate right before entering alt block
   Svc->>+ArtEnt: findPublishedByCategoryId(categoryId)
```

```
    ArtEnt-->>-Svc: articleList
    Svc-->>-Ctrl: articleList

    alt [Exc 2a: No content available in category]
        %% Rule 4.2: Ctrl and DashUI were opened before alt, so NO '-' here
        Ctrl-->>DashUI: displayEmptyState("There are currently no published articles in this category. Please check back later.")
        DashUI-->>User: display empty state & "Clear Filter" button
    else [Normal Flow: Content available]
        %% Rule 4.2: Ctrl and DashUI were opened before alt, so NO '-' here
        Ctrl-->>DashUI: renderFilteredContent(articleList)
        DashUI-->>User: display paginated content (10 items/page)
    end
    %% Rule 4.2: Deactivate objects opened before the alt block
    deactivate Ctrl
    deactivate DashUI
  end

  %% Rule 4.4: Independent Normal Flow wrapped in opt block
  opt [Normal Flow Step 3-7: View Article Details]
    User->>+DashUI: selectArticle(articleId)
    DashUI->>+Ctrl: viewArticleDetails(articleId)

    Ctrl->>+Svc: getPublishedArticleDetails(articleId)

    %% Rule 4.1: Call Entity and deactivate right before entering alt block
```

```
Svc->>+ArtEnt: findByIdAndStatus(articleId, "Published")
Note over ArtEnt: Apply BR-22: Validate content status strictly
ArtEnt-->>-Svc: articleData
Svc-->>-Ctrl: articleData

alt [Exc 4a: Content unavailable or unpublished]
    %% Rule 4.3: ErrUI is spawned exclusively inside this branch, so it must be deactivated inside
    Ctrl->>+ErrUI: display404Page("This content is currently unavailable or under review by our moderation team.")
    ErrUI-->>User: display 404 error page & "Return to News Home"
    deactivate ErrUI

    %% Rule 4.2: NO '-' here
    Ctrl-->>DashUI: fail()
else [Normal Flow: Content retrieved successfully]
    %% Rule 4.3: DetUI is spawned exclusively inside this branch, so it must be deactivated inside
    Ctrl->>+DetUI: renderDetailedView(articleData)
    DetUI-->>User: display full article, cover, author, & share buttons
    deactivate DetUI

    %% Rule 4.2: NO '-' here
    Ctrl-->>DashUI: success()
end
%% Rule 4.2: Deactivate objects opened before the alt block
deactivate Ctrl
```

```
    deactivate DashUI
  end
```

UC - 31: Manage Content

```
classDiagram
  direction TB

  class ContentManagementUI {
    <<Boundary>>
    +navigateToContentManagement()
    +renderContentDashboard(contentList)
    +clickReviewAndEdit(articleId, tab)
    +loadContentEditor(articleId)
    +renderContentEditor(articleData)
    +reviewAndChangeStatus(status)
    +clickSaveChanges(formData)
    +displayValidationError(message)
    +redirectAndShowSuccess(message)
    +clickCreateNewContent(tab)
    +renderBlankDictionaryEditor()
    +inputDictionaryData(dictData)
    +setPublishedAndSave(dictData)
    +selectFeedbackTab()
    +renderFeedbackList(feedbackList)
    +clickHideFeedback(feedbackId)
  }
```

```
class ContentController {
    <<Control>>
    +loadContentDashboard()
    +loadContentEditor(articleId)
    +saveArticleChanges(formData)
    +initDictionaryEditor()
    +saveNewDictionaryEntry(dictData)
    +loadFeedbackList()
    +hidePatientFeedback(feedbackId)
}

class ContentService {
    <<Control>>
    +getDashboardData() List
    +getArticleDetails(articleId) Object
    +validateContent(contentData) bool
    +updateArticleStatusAndSave(formData) bool
    +createNewEntry(dictData) int
    +getRecentFeedback() List
    +hideFeedbackAndUpdateRating(feedbackId) bool
}

class CmsArticle {
    <<Entity>>
    -articleId int
```

```
    -categoryId int
    -authorId int
    -title String
    -coverImageUrl String
    -bodyContent String
    -status String
    -publishedAt datetime
    +fetchAllContentSummaries() List
    +findById(articleId) Object
    +updateStatusAndDetails(formData, adminId) bool
}

class ContentVersionHistory {
    <<Entity>>
    -versionId int
    -referenceId int
    -contentType String
    -contentSnapshotJson String
    -modifiedBy int
    -modifiedAt datetime
    +createSnapshot(oldData, adminId) int
}

class DiseaseDictionary {
    <<Entity>>
    -diseaseId int
```

```
    -authorId int
    -diseaseTitle String
    -symptomsText String
    -causes String
    -treatments String
    -prevention String
    -status String
    +createEntry(dictData, adminId) int
}

class Feedback {
    <<Entity>>
    -feedbackId int
    -appointmentId int
    -ratingScore int
    -reviewText String
    -status String
    -moderatedByAdminId int
    -moderatedAt datetime
    +findRecentReviews() List
    +updateStatusToHidden(feedbackId, adminId) bool
}

class DoctorProfile {
    <<Entity>>
    -doctorId int
```

```
    -averageRating float
    -totalReviews int
    +recalculateAverageRating(doctorId) bool
  }

  ContentManagementUI --> ContentController : triggers / submits via
  ContentController --> ContentService : delegates to
  ContentController --> ContentManagementUI : renders
  ContentService --> CmsArticle : queries / updates
  ContentService --> ContentVersionHistory : creates
  ContentService --> DiseaseDictionary : creates
  ContentService --> Feedback : queries / updates
  ContentService --> DoctorProfile : updates

sequenceDiagram
  actor Admin
  participant AdminUI as ContentManagementUI<br/><<Boundary>>
  participant Ctrl as ContentController<br/><<Control>>
  participant Svc as ContentService<br/><<Control>>
  participant ArtEnt as CmsArticle<br/><<Entity>>
  participant HistEnt as ContentVersionHistory<br/><<Entity>>
  participant DictEnt as DiseaseDictionary<br/><<Entity>>
  participant FbEnt as Feedback<br/><<Entity>>
  participant DocEnt as DoctorProfile<br/><<Entity>>

  %% Normal Flow 1-5: Initialization & Dashboard
```

```
Admin->>+AdminUI: navigateToContentManagement()
AdminUI->>+Ctrl: loadContentDashboard()

Ctrl->>+Svc: getDashboardData()
%% Rule 4.1 applied implicitly for standard retrieval
Svc->>+ArtEnt: fetchAllContentSummaries()
ArtEnt-->>-Svc: contentList
Svc-->>-Ctrl: contentList

Ctrl-->>-AdminUI: renderContentDashboard(contentList)
AdminUI-->>-Admin: display Content Dashboard (Tabs, Search, Table)

%% Normal Flow 6-13 & Exception 8a: Review & Edit Article
Admin->>+AdminUI: clickReviewAndEdit(articleId, tab)
AdminUI->>+Ctrl: loadContentEditor(articleId)

Ctrl->>+Svc: getArticleDetails(articleId)
Svc->>+ArtEnt: findById(articleId)
ArtEnt-->>-Svc: articleData
Svc-->>-Ctrl: articleData

Ctrl-->>-AdminUI: renderContentEditor(articleData)
AdminUI-->>-Admin: display Content Editor screen

Admin->>+AdminUI: reviewAndChangeStatus(status)
AdminUI-->>-Admin: status updated in UI
```

```
Admin->>+AdminUI: clickSaveChanges(formData)
AdminUI->>+Ctrl: saveArticleChanges(formData)

%% Rule 4.1: Execute condition check right before entering alt block
Ctrl->>+Svc: validateContent(formData)
Svc-->>-Ctrl: isValid

alt [Exc 8a: Missing mandatory content fields]
    %% Rule 4.2: Ctrl and AdminUI opened before alt, so no '-' here
    Ctrl-->>AdminUI: displayValidationError("Please complete all required fields before saving or publishing this content.")
    AdminUI-->>Admin: highlight empty fields in red
else [Normal Flow 11-13: Valid content]
    Ctrl->>+Svc: updateArticleStatusAndSave(formData)

    Svc->>+HistEnt: createSnapshot(oldData, adminId)
    Note over HistEnt: Apply BR-22: Save version history
    HistEnt-->>-Svc: snapshotId

    Svc->>+ArtEnt: updateStatusAndDetails(formData, adminId)
    Note over ArtEnt: Update status to Published
    ArtEnt-->>-Svc: updateSuccess

    Svc-->>-Ctrl: saveResult
```

```
    %% Rule 4.2: Ctrl and AdminUI opened before alt, so no '-' here
    Ctrl-->>AdminUI: redirectAndShowSuccess("Content has been successfully published to the public portal.")
    AdminUI-->>Admin: redirect to Dashboard & display success message
  end
  %% Rule 4.2: Deactivate objects opened before the alt block outside the end
  deactivate Ctrl
  deactivate AdminUI


  %% Rule 4.4: Independent sequential flow wrapped in opt block
  opt [Alt 3a: Create new medical dictionary entry]
    Admin->>+AdminUI: clickCreateNewContent(tab)
    AdminUI->>+Ctrl: initDictionaryEditor()
    Ctrl-->>-AdminUI: renderBlankDictionaryEditor()
    AdminUI-->>-Admin: display blank Dictionary Editor

    Admin->>+AdminUI: inputDictionaryData(dictData)
    AdminUI-->>-Admin: data inputted

    Admin->>+AdminUI: setPublishedAndSave(dictData)
    AdminUI->>+Ctrl: saveNewDictionaryEntry(dictData)

    %% Rule 4.1: Validate condition right before entering alt block
    Ctrl->>+Svc: validateContent(dictData)
    Svc-->>-Ctrl: isValid
```

```
    alt [Exc 8a: Missing mandatory dictionary fields]
      %% Rule 4.2: No '-' on return arrows inside block
      Ctrl-->>AdminUI: displayValidationError("Please complete all required fields before saving or publishing this content.")
      AdminUI-->>Admin: highlight empty fields in red
    else [Valid Flow]
      Ctrl->>+Svc: createNewEntry(dictData)

      Svc->>+DictEnt: createEntry(dictData, adminId)
      Note over DictEnt: Apply BR-22: Associate entry with Admin
      DictEnt-->>-Svc: newEntryId

      Svc-->>-Ctrl: createResult

      %% Rule 4.2: No '-' on return arrows inside block
      Ctrl-->>AdminUI: redirectAndShowSuccess("New health dictionary entry created successfully.")
      AdminUI-->>Admin: redirect to Dashboard & display success message
    end
    %% Rule 4.3 (applied to opt): Deactivate at the end of the opt branch
    deactivate Ctrl
    deactivate AdminUI
  end

  %% Rule 4.4: Independent sequential flow wrapped in opt block
```

```
opt [Alt 3b: Moderate abusive patient feedback]
    Admin->>+AdminUI: selectFeedbackTab()
    AdminUI->>+Ctrl: loadFeedbackList()

    Ctrl->>+Svc: getRecentFeedback()
    Svc->>+FbEnt: findRecentReviews()
    FbEnt-->>-Svc: feedbackList
    Svc-->>-Ctrl: feedbackList

    Ctrl-->>-AdminUI: renderFeedbackList(feedbackList)
    AdminUI-->>-Admin: display list of recent feedback

    Admin->>+AdminUI: clickHideFeedback(feedbackId)
    AdminUI->>+Ctrl: hidePatientFeedback(feedbackId)

    Ctrl->>+Svc: hideFeedbackAndUpdateRating(feedbackId)

    Svc->>+FbEnt: updateStatusToHidden(feedbackId, adminId)
    FbEnt-->>-Svc: hideSuccess

    Svc->>+DocEnt: recalculateAverageRating(doctorId)
    DocEnt-->>-Svc: calcSuccess

    Svc-->>-Ctrl: hideResult
```

```
    Ctrl-->>AdminUI: redirectAndShowSuccess("The selected feedback has been hidden from public view.")
    AdminUI-->>Admin: update UI & display success message

    %% Rule 4.3 (applied to opt): Deactivate at the end of the opt branch
    deactivate Ctrl
    deactivate AdminUI
  end
```